A VISUAL FIELD GUIDE

Attention, one change at a time.

16 chapters · 91 mechanism cards · formulas, diagrams, numbers and catches.

00 · The map

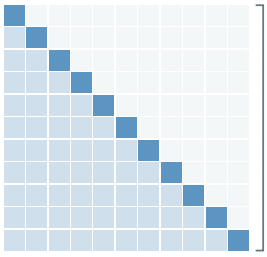
Every mechanism in this book attacks one factor of the attention bill. Here are the bill, the levers, and when each lever appeared.

01 The bill: prefill pays N^2, decode pays N bytes per token

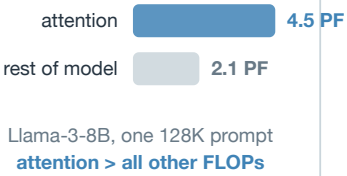
$$\text{FLOP}_{\text{prefill}} \approx 2N^2 d \cdot L \qquad \text{bytes}_{\text{decode step}} \approx N \cdot L \cdot 2H_{kv} d_h \cdot s \; + \; \text{weights}$$

Prefill · compute-bound

$$\text{FLOP}_{attn} \approx 2N^2 \; d \cdot L$$



every query × every key



Llama-3-8B, one 128K prompt
attention > all other FLOPs

Decode · bandwidth-bound

$$\text{bytes / step} \approx N \cdot L \cdot 2H_{kv} \, d_h \, s \; + \; \text{weights}$$



N records, every step



read per generated token at 128K (×batch for KV)

Most routes in this book shrink one factor of the decode bill:

N (records kept or read) · $2H_{kv}d_h$ (record width) · L · s (bytes)

Two different walls.
Long prompts hit compute (prefill). Long generations and big batches hit HBM bandwidth (decode).

At 128K the cache outweighs the model.
Llama-3-8B reads more KV bytes than weight bytes per token.

Batch multiplies KV, not weights.
That is why serving cares about cache bytes first.

02 Seven levers

$$\text{decode bytes} \approx \underbrace{N_{\text{read}}}_{05\text{–}08} \cdot \underbrace{w_{\text{record}}}_{03\text{–}04} \cdot L \cdot \underbrace{s}_{13\text{–}14} \qquad \text{or} \qquad \underbrace{|S| \cdot L \cdot s}_{09\text{–}12}$$

decode bytes $\approx N_{read} \cdot w_{\text{record}} \cdot L \cdot s$ state models: $|S| \cdot L \cdot s$

LEVER	EFFECT ON ONE LAYER	CHAPTERS
Make positions behave enabler		01 Positions · 02 Stable softmax
Thinner records width ↓		03 Share KV heads · 04 Latent KV
Read fewer records reads ↓, cache kept		05 Fixed patterns · 06 Sparse reads
Keep fewer records N ↓, irreversible		07 Evict from cache · 08 Compress sequence
Replace records by a state O(1) per token	fixed	09 Linear/SSM · 10 Delta rule · 11 TTT memory
Mix both memories few full layers		12 Hybrid stacks
Move fewer bytes s ↓, traffic ↓	FP8/4	13 Kernels · 14 Serving the cache

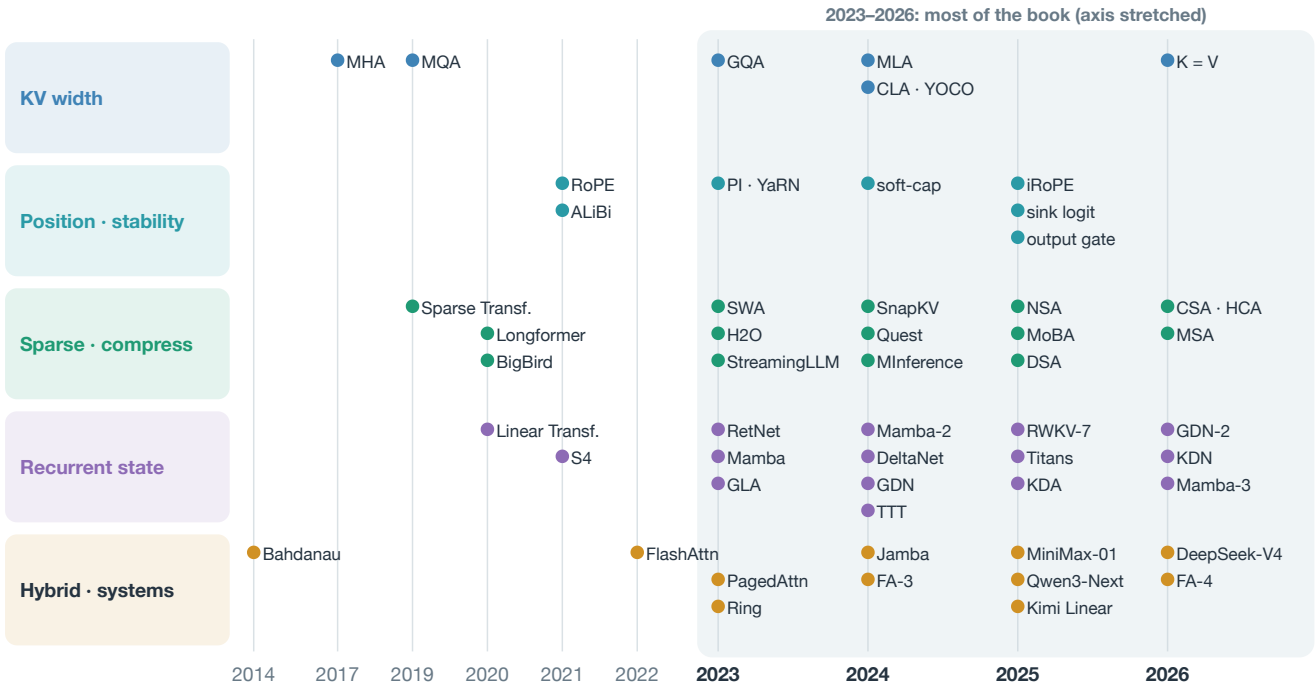
Reads vs records.
Sparse reads (06) keep the cache and touch less. Eviction (07) and compression (08) shrink the cache itself.

State models change the formula.
Cost stops growing with N; capacity becomes the question (09–11).

Production stacks combine levers.
DeepSeek-V4: compressed records + top-k sparse reads + a local window + FP8 storage.

03 Twelve years in five lanes

2014 Bahdanau → 2017 MHA → 2023 GQA, Mamba → 2024 MLA, GDN → 2025 NSA, DSA, KDA → 20



2023 was the turn.
GQA, sliding windows, H2O, RetNet, Mamba and PagedAttention all arrived in one year.

2025–26 is convergence.
Frontier models mix a few full-attention layers with sparse reads or recurrent state.

Each dot is a card.
Use the search box in the sidebar to jump to any name.

04 How to read the cards

card = meta line + one formula + one figure + 2–4 tips (+optional derivation)

query q · key · KV cache k · value · state v, S · output · kept
erased · evicted · cost · gate · position · inactive · skipped

N tokens L layers H_q, H_{kv} heads $G = H_q/H_{kv}$ d_h head dim s bytes

$S_t \in \mathbb{R}^{d_v \times d_k}$ $o_t = S_t q_t$ write $v_t k_t^\top$ W window n_{sel} selected

Colours never change meaning.
Gold is always a query, purple always a value or state.

Numbers are real unless marked "toy".
Real numbers come from papers, model cards or `config.json` . See Primary sources.

Click any figure
to open it full size.

01 · Positions

RoPE decides how far apart two tokens look. Long context needs slow pairs that span the window, rescaled pairs, or no rotation at all.

01 RoPE: a spectrum of clocks, set by the base

RoPE 2021 · Su et al. · arXiv:2104.09864 — **ABF** 2023 · Xiong et al., Meta · arXiv:2309.16039 · Used in: Llama 3 (b = 500K), gpt-oss (150K), Qwen3 (1M), Qwen3-Next (10M), Gemma 3 (10K local / 1M global)

$$\theta_i = b^{-2i/d_h}, \qquad \lambda_i = \frac{2\pi}{\theta_i} = 2\pi\, b^{2i/d_h}, \qquad (R_t q)^\top (R_j k) = q^\top R_{j-t}\, k$$

$\theta_i = b^{-2i/d_h}$ $\lambda_i = 2\pi / \theta_i$ $(R_t\, q)^\top (R_j\, k) = q^\top\ R_{j-t}\, k$

Each pair is a clock
Llama 3 · b = 500K · d_h = 128
angle swept over Δ = 1,000 tokens



pair 0
λ = 6.3
159 turns



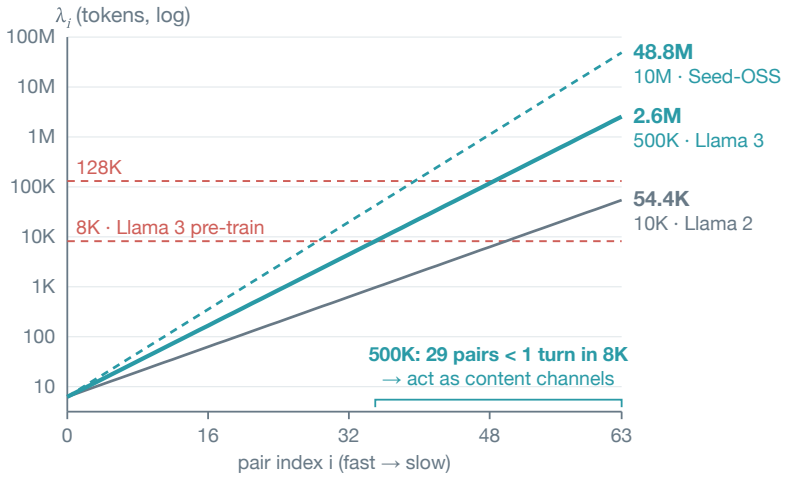
pair 21
λ = 466
2.1 turns



pair 42
λ = 34.5K
10°



pair 63
λ = 2.6M
0.14°



Two bases in one model

- Gemma 3 local ×5: b = 10K, W = 1,024
- global ×1: b = 1M, PI ×8 → 128K
- DeepSeek-V4 SWA layers: b = 10K, no YaRN
- compressed layers: b = 160K + YaRN ×16

Why raise b:
a pair's angle repeats every λ_i tokens. ABF (Llama 2 Long: 10K → 500K) slows every pair and weakens long-distance decay.

Numbers ($d_h = 128$):
slowest wavelength 54K tokens at b = 10K, 2.6M at 500K, 5.1M at 1M. Fastest is always 2π ≈ 6.3.

Dual bases:
local layers never see distances beyond W, so they keep b = 10K and skip scaling (Gemma 3, DeepSeek-V4 SWA layers).

Not solved:
a larger base is not extrapolation. Positions past the trained length still need scaling (next card).

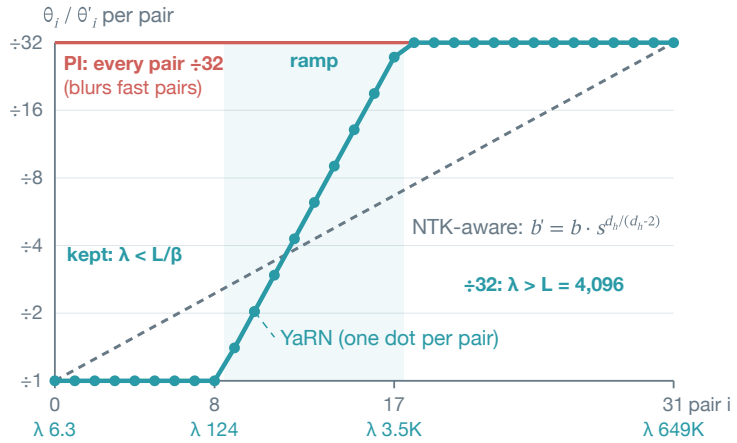
02 Context extension: PI → NTK-aware → YaRN / llama3

PI 2023 · Chen et al., Meta · arXiv:2306.15595 — **YaRN** 2023 · Peng et al., Nous Research / EleutherAI · ICLR 2024 · arXiv:2309.00071 · Used in: gpt-oss, DeepSeek-V3, DeepSeek-V4, OLMo 3, Qwen3 (optional); llama3 type: Llama 3.1, Llama 4

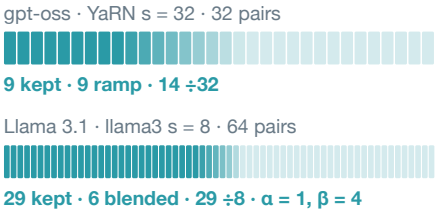
$$\theta'_i = (1-\gamma(r_i))\frac{\theta_i}{s} + \gamma(r_i)\,\theta_i, \qquad r_i = \frac{L}{\lambda_i}, \qquad \gamma(r) = \mathrm{clip}(\frac{r-\alpha}{\beta-\alpha}, 0, 1)$$

$\theta'_i = (1-\gamma_i)\,\theta_i/s + \gamma_i\,\theta_i$ $\quad \gamma_i = \mathrm{clip}((L/\lambda_i - \alpha)/(\beta - \alpha), 0, 1)$ $\quad \mathrm{logits} \times (0.1 \ln s + 1)^2$

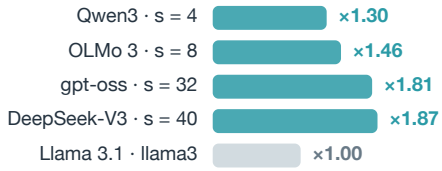
gpt-oss · d_h = 64 · b = 150K · L = 4,096 → 131,072 · s = 32 · α = 1, β = 32



Which pairs move (per-pair strip)



logit multiplier $(0.1 \ln s + 1)^2$



Compare:
PI divides every pair by s and blurs local order. YaRN keeps pairs with $\lambda_i < L/\beta$ and interpolates only slow ones.

Numbers:
gpt-oss 4,096 → 131,072 (s = 32).
DeepSeek-V3 4,096 → 163,840 (s = 40).
Llama 3.1 8,192 → 131,072 (llama3, s = 8).

Temperature is free:
logits × $(0.1 \ln s + 1)^2$, folded into cos/sin or the softmax scale. The llama3 type has none.

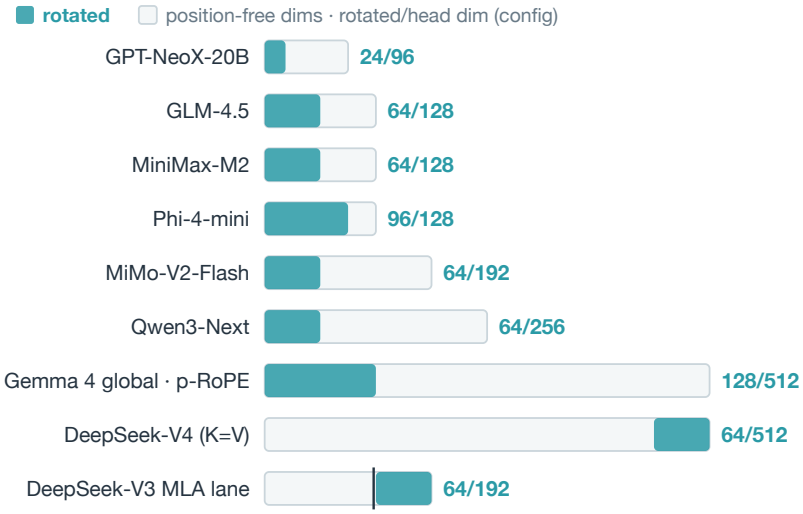
Variants:
LongRoPE searches one factor per pair (Phi-4-mini: 1.0 → 47.8). Dual Chunk Attention remaps distances per chunk, training-free (Qwen3 inference).

03 Partial RoPE and p-RoPE: rotate only some dims

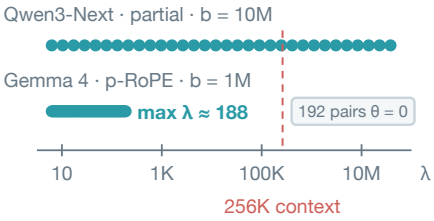
Partial RoPE 2022 · Black et al., EleutherAI (GPT-NeoX-20B) · arXiv:2204.06745 — **p-RoPE** 2024 · Barbero et al., Oxford / Google DeepMind · ICLR 2025 · arXiv:2410.06205 · Used in: GLM-4.5, Qwen3-Next, MiniMax-M2, MiMo-V2-Flash, Phi-4-mini, Gemma 4, DeepSeek-V4

$$q_t^\top k_j \;\to\; (R_t\, q_t^{\mathrm{rot}})^\top (R_j\, k_j^{\mathrm{rot}}) + (q_t^{\mathrm{nope}})^\top k_j^{\mathrm{nope}}, \qquad d_{\mathrm{rot}} = p\, d_h$$

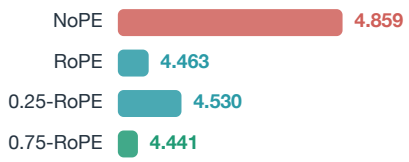
$q^\top k \to (R_t\, q_{rot})^\top (R_j\, k_{rot}) + q_{nope}{}^\top k_{nope}$ $\quad d_{rot} = p \cdot d_h$



Same p = 0.25, different spectrum



Gemma 2B · 8K · Wiki ppl (reported)



bars start at 4.40 · lower is better

p-RoPE: Barbero et al., ICLR 2025, Table 2

Compare:
NeoX-style partial squeezes the whole spectrum into $p\,d_h$ dims. p-RoPE keeps the fastest p of the full spectrum and freezes the rest (θ = 0).

Why it works:
slow pairs barely turn within the context and already act as content channels. p-RoPE makes them exactly position-free.

Cache link:
unrotated dims are position-free, so they can be absorbed or shared: MLA's decoupled RoPE lane, K=V in Gemma 4 and DeepSeek-V4.

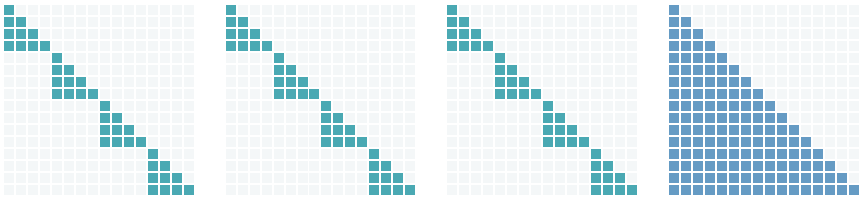
04 NoPE layers: iRoPE and RNoPE

NoPE 2023 · Kazemnejad et al. · NeurIPS 2023 · arXiv:2305.19466 — **iRoPE** 2025 · Meta (Llama 4) — **RNoPE** 2025 · Yang et al., Cohere · arXiv:2501.18795 · Used in: Llama 4, Command A, SmolLM3

$$e_{tj}^{\mathrm{RoPE}} = \frac{(R_t q_t)^\top (R_j k_j)}{\sqrt{d_h}} + m_{tj}^{\mathrm{local}}, \qquad e_{tj}^{\mathrm{NoPE}} = \frac{q_t^\top k_j}{\sqrt{d_h}} + m_{tj}^{\mathrm{causal}}$$

RoPE layer: $(R_t\,q)^\top (R_j\,k) + m_{\mathrm{chunk}}$ NoPE layer: $q^\top k + m_{\mathrm{causal}}$

Llama 4 block of 4 layers · toy: N = 16, chunk = 4 (real chunk 8,192)

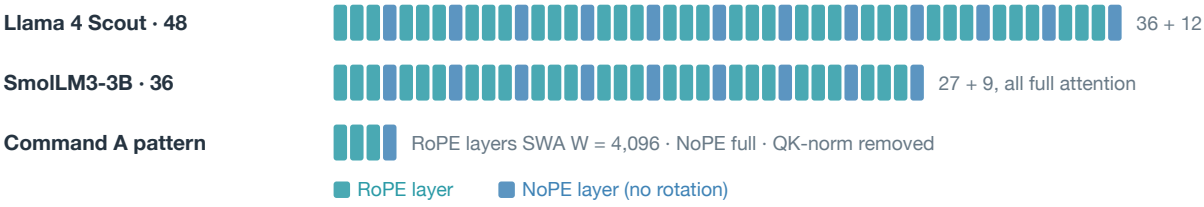


L1 · RoPE · chunk | L2 · RoPE · chunk (local order, recency) | L3 · RoPE · chunk | L4 · NoPE · full (long-range retrieval)

Llama 4 Scout KV @128K, BF16
8 KV heads × 128 × 2 × 2 B
= 4 KiB per token per layer



12 NoPE × 131,072 tok = 6.0 GiB
36 RoPE × 8,192 tok = 1.1 GiB



Why it works:
the causal mask alone leaks order. RoPE layers supply recency; NoPE layers retrieve distant tokens (RNoPE §2).

Cache:
Llama 4's RoPE layers are chunked, so only 12 of 48 layers grow with N.

Gotcha:
raising θ in the hybrid's RoPE layers hurt retrieval. Needle attention in NoPE layers fell 0.077 → 0.037 (reported).

Not solved:
NoPE logits flatten as context grows. Llama 4 adds a temperature to exactly these layers (next card).

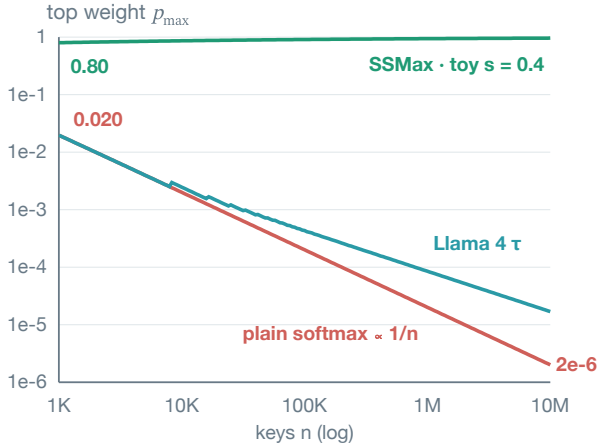
05 Length-dependent logit temperature

SSMax 2025 · Nakanishi · arXiv:2501.19399 — **Attention temperature tuning** 2025 · Meta (Llama 4 config) · Used in: Llama 4 (NoPE layers); YaRN models use a constant factor

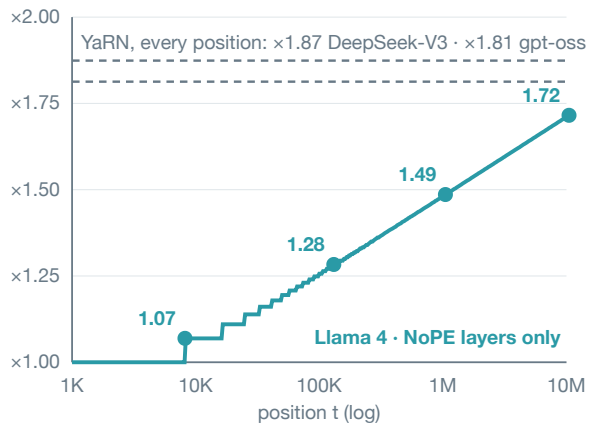
$$\text{softmax}_j(\tau\, e_{tj}), \qquad \tau_{\text{Llama 4}}(t) = 1 + 0.1\ln(1 + \lfloor\frac{t+1}{8192}\rfloor), \qquad \tau_{\text{SSMax}} = s_h \ln n$$

$q_t \leftarrow \tau(t)\, q_t$ $\qquad \tau_{\text{Llama 4}} = 1 + 0.1\ln(1 + \lfloor (t{+}1)/8192 \rfloor)$ $\qquad \tau_{\text{SSMax}} = s_h \ln n$

Attention fading · toy: one key Δ = 3 above n − 1 equal keys



Logit multiplier vs position (configs)



Llama 4: attn_scale 0.1, floor_scale 8,192 (config); multiplies q, so logits scale by τ. YaRN folds (0.1 ln s + 1) into both q and k.

Why:
with n keys and a fixed gap Δ, top weight ≈ e^{Δ}/n. Attention fades as context grows (SSMax §2).

Numbers:
Llama 4 τ = 1.07 at 8K, 1.28 at 128K, 1.72 at 10M (`attn_scale` 0.1, `floor_scale` 8,192).

Compare:
SSMax learns s_h per layer and head (144 parameters at 162M). YaRN applies one constant to every position.

History · ALiBi (ICLR 2022):
no rotation; subtract $m_h(t-j)$, slopes $m_h = 2^{-8h/H}$. Trained on 1,024 tokens, it matched sinusoidal trained on 2,048.

02 · Stable softmax

Softmax must sum to one, and its logits can grow without bound. Modern layers cap the logits, give each head an exit, and gate its output.

01 QK-norm, soft-cap, QK-Clip: bound the logits

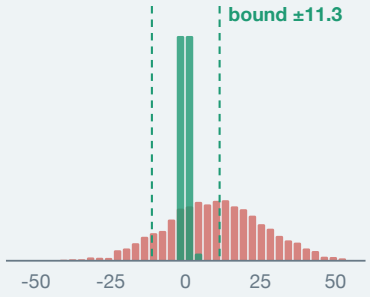
QK-norm 2023 · Dehghani et al. (ViT-22B); logit-growth study Wortsman et al., Google DeepMind · ICLR 2024 · arXiv:2309.14322 — **Soft-cap** 2024 · Gemma 2 · arXiv:2408.00118 — **QK-Clip** 2025 · Kimi K2 · arXiv:2507.20534 · Used in: Qwen3, Qwen3-Next, OLMo 2/3, Gemma 3/4, GLM-4.5, MiniMax-M2, Llama 4 Scout, DeepSeek-V4

$$\hat{q} = g_q \odot \frac{q}{\mathrm{RMS}(q)}, \quad \hat{k} = g_k \odot \frac{k}{\mathrm{RMS}(k)} \Rightarrow \left|\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}}\right| \le \sqrt{d_h} \; \|g_q\|_\infty \; \|g_k\|_\infty$$

$$q \leftarrow g_q \odot q \,/\, \mathrm{RMS}(q) \quad k \leftarrow g_k \odot k \,/\, \mathrm{RMS}(k) \quad \Rightarrow |q^\top k| \,/\, \sqrt{d_h} \le \sqrt{d_h}\; g_q\, g_k$$

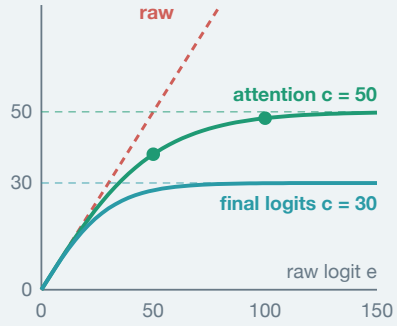
QK-norm · toy, d_h = 128

no norm, q,k × 4: top weight 0.99



QK-norm: top weight 0.07 (64 keys)

Soft-cap · Gemma 2



c = 50: 50 → 38 · 100 → 48 · 150 → 50

Gemma 3 replaced it with QK-norm

QK-Clip · Kimi K2, τ = 100

$S_{\max}$ per head (toy)



h_3: W_q, $W_k \times \sqrt{0.4} = \times 0.63$

K2 without clip: > 1,000 (reported)

where the norm sits

- per head: Qwen3, Gemma 3/4, Qwen3-Next
- whole projection: OLMo 2/3, MiniMax-M2
- gain-free: Llama 4 Scout (RoPE layers), DeepSeek-V4 queries

Why logits grow:
$1/\sqrt{d_h}$ fixes the scale only at initialisation. Trained ‖q‖‖k‖ grow until attention turns one-hot (Wortsman §3.1).

Variants:
soft-cap $e \leftarrow c\tanh(e/c)$ shapes logits in the forward pass. QK-Clip rescales W_q, W_k by $\sqrt{\gamma_h}$, $\gamma_h = \min(1, \tau/S_{\max h})$, after the optimizer step.

MLA gotcha:
absorbed MLA never materialises keys, so Kimi K2 uses QK-Clip instead of QK-norm.

Not free:
Cohere's RNoPE design (Command A) dropped QK-norm: it flattened attention and hurt long-context retrieval.

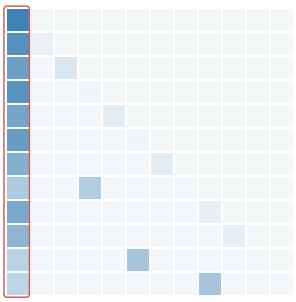
02 Attention sinks → a learned sink logit

Attention sinks 2023 · Xiao et al., MIT / Meta · ICLR 2024 · arXiv:2309.17453 — **Learned sink logit** 2025 · OpenAI (gpt-oss) · arXiv:2508.10925 · Used in: gpt-oss, MiMo-V2-Flash (SWA layers), DeepSeek-V4

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\exp(z_h) + \sum_{r \le t} \exp(e_{tr})}, \qquad \sum_{j \le t} \alpha_{tj} = 1 - \frac{\exp(z_h)}{\exp(z_h) + \sum_{r \le t} \exp(e_{tr})} < 1$$

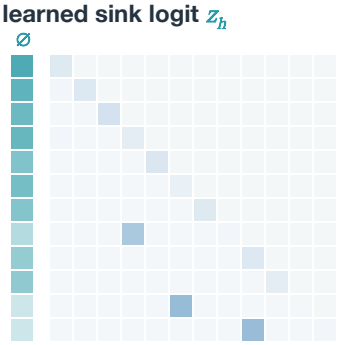
$\alpha_{tj} = \exp(e_{tj}) \,/\, (\exp(z_h) + \Sigma_{r \le t} \exp(e_{tr}))$ $\qquad \Sigma_j \alpha_{tj} = 1 - \alpha_{\text{sink}} < 1$

Vanilla softmax (toy)



token 0 = sink
row 11: 0.34 on token 0

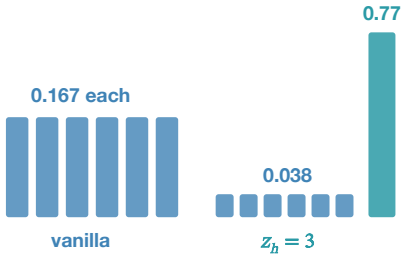
+ learned sink logit z_h



extra column: no value
row 11: sink 0.24, token 0 0.02

One query, nothing relevant (toy)

six keys, all logits 0



vanilla: o = mean(v), forced read
sink: o = 0.23 × mean(v)
→ "attend to nothing"

Cost: one scalar per head
gpt-oss-120b: 64 heads × 36 layers = 2,304 params

MiMo-V2-Flash, W = 128 (reported)
MMLU 54.9 → 58.3 · BBH 52.4 → 56.1 with sink

Why sinks appear:
weights must sum to 1, so a head with nothing to read parks mass on early tokens. StreamingLLM keeps 4 of them (ch. 05).

The fix:
one learned logit z_h per head joins the denominator. It has no value vector, so the head can attend to nothing.

Cost:
one scalar per head. gpt-oss puts it in every layer; MiMo-V2-Flash only in SWA layers; DeepSeek-V4's kernel takes `attn_sink`.

Compare:
an output gate (next card) also removes the sink, without changing the softmax.

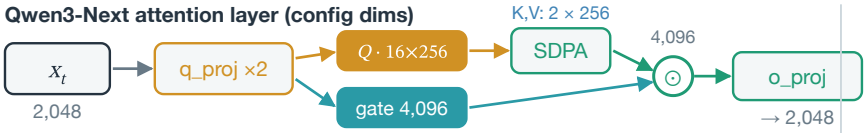
03 Gated attention: a sigmoid gate after SDPA

Gated attention 2025 · Qiu et al., Qwen (Alibaba) · NeurIPS 2025 Best Paper · arXiv:2505.06708 · Used in: Qwen3-Next, Qwen3.5 (`attn_output_gate`)

$$Y' = Y \odot \sigma(X W_\theta), \qquad Y = \text{softmax}(\frac{QK^\top}{\sqrt{d_h}})V, \qquad \text{out} = W_O\, Y'$$

$Y' = Y \odot \sigma(X\, W_\theta)$ $\qquad Y = \text{softmax}(QK^\top / \sqrt{d_h})\, V$ $\qquad \text{out} = W_O\, Y'$

Qwen3-Next attention layer (config dims)

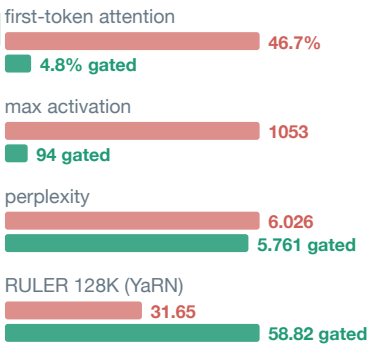


One head, 12 of 256 channels (toy)

Y	+0.80	-0.60	+0.50	+0.90	-0.30	+0.70	-0.80	+0.40	+0.60	-0.50	+0.30	+0.90
$\sigma(XW_\theta)$	0.03	0.92	0.05	0.01	0.08	0.71	0.02	0.04	0.88	0.02	0.06	0.03
$Y' = Y \odot \sigma$	+0.02	-0.55	+0.03	+0.01	-0.02	+0.50	-0.02	+0.02	+0.53	-0.01	+0.02	+0.03

mean gate 0.116 (reported): most channels shut per query

Reported · 15B MoE, 400B tokens



Gate location

output gate: $y_t = g_t \odot o_t$
state gate: $S_t = \alpha_t\, S_{t-1} + v_t k_t^\top$



changes this token's read only
changes every later read

Why it works:
a nonlinearity between W_V and W_O, plus query-dependent sparsity (mean gate 0.116, reported).

Cost:
elementwise gating adds ≈201M parameters to the 15B MoE (headwise: 1.6M) and under 2% latency (reported).

Gate location:
this gate rescales one token's read. A state gate (α_t in Gated DeltaNet) changes what every later query reads.

Gotcha:
a gate near 0 does not skip work. SDPA has already run; only its output is suppressed.

04 Research knobs: forget gate and differential attention

FoX 2025 · Lin et al., Mila · ICLR 2025 · arXiv:2503.02130 — **Diff Transformer** 2024 · Ye et al., Microsoft Research / Tsinghua · ICLR 2025 (oral) · arXiv:2410.05258 · Used in: Phi-4-mini-flash-reasoning (differential attention)

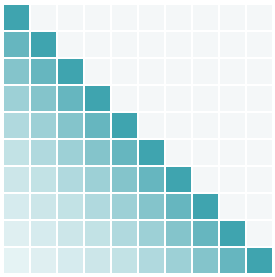
$$\text{FoX: } e_{tj} = \frac{q_t^\top k_j}{\sqrt{d_h}} + \sum_{l=j+1}^{t} \ln f_l, \qquad f_l = \sigma(w^\top x_l + b)$$

$$\text{Diff: } o_t = (\text{softmax}(e_t^{(1)}) - \lambda\,\text{softmax}(e_t^{(2)}))V$$

FoX: $e_{tj} = q_t^\top k_j/\sqrt{d_h} + c_t - c_j \qquad c_t = \Sigma_{l\le t} \ln f_l$

Diff: $o_t = (\text{softmax}(e^{(1)}) - \lambda\,\text{softmax}(e^{(2)}))\,V$

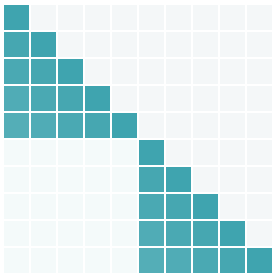
ALiBi-like: constant f = 0.8



same decay in every row

shade = $\exp(c_t - c_j)$

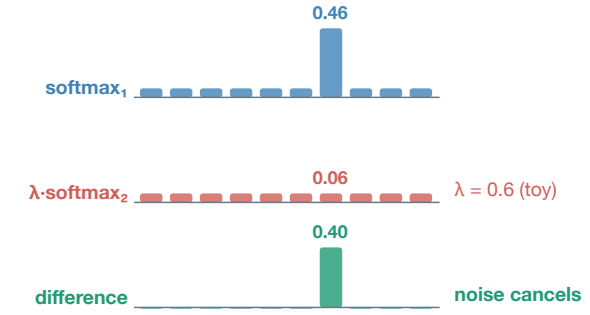
FoX: learned f per token



.97 .97 .97 .97 .05 .97 .97 .97 .97

f_t (toy): $f_5 = 0.05$ wipes tokens < 5

Differential attention (toy, 10 keys)



$\lambda_{\text{init}} = 0.8 - 0.6\,e^{-0.3(\ell-1)}$ by layer



FoX bridges to recurrence:
$f \equiv 1$ gives NoPE softmax, $f \equiv e^{-m}$ gives ALiBi, and a learned f is a forget gate as in GLA.

FoX kernel:
FlashAttention-compatible. Keep $c_t = \sum_{l\le t} \ln f_l$ and add $c_t - c_j$ per tile. No positional embedding needed.

Diff numbers:
per-head RMSNorm × $(1 - \lambda_{\text{init}})$. Reported: a 6.8B Diff Transformer matches an 11B Transformer's loss.

Cost:
two QK maps per head (heads halved to keep parameters). No KV-cache saving.

05 2026 anatomy: which trick sits where

$$y = W_O[\sigma(XW_\theta) \odot \sum_j \text{softmax}_{j \cup \text{sink}}(\frac{\text{RoPE}_p(\widehat{q_t})^\top \text{RoPE}_p(\widehat{k_j})}{\sqrt{d_h}} + m^W_{tj})v_j]$$

$$y = W_O[\sigma(XW_\theta) \odot \text{softmax}_{j \cup \text{sink}}(\text{RoPE}_p(q)^\top \text{RoPE}_p(k) / \sqrt{d_h} + m^W)\, v]$$

stage →	$W_Q \cdot W_{KV}$	QK-norm	RoPE (part)	mask	softmax	· V	⊙ σ gate	W_O
gpt-oss-120b	**8 KV × 64** GQA, bias	—	**full · 150K** YaRN ×32	**128 ⇄ full** 1 : 1	**+ sink** per head	—	—	—
Gemma 4 31B	**global: K=V** 4 × 512	**q, k RMS** scale 1.0	**global ¼ · 1M** local 10K	**1,024 ⇄ full** 5 : 1	—	**V-norm** no gain	—	—
DeepSeek-V4-Pro	**K=V · MQA** 1 × 512	**q RMS** + kv-norm	**64 of 512** 10K | 160K	**128 + comp.** entries	**+ sink** per head	**de-rotate** output	—	**16 groups** low-rank
Qwen3-Next-80B	**2 KV × 256** GQA	**q, k RMS** per head	**¼ of 256** 10M	**full** 1 in 4 layers	—	—	**σ gate** elementwise	—

Sink or gate, never both here · K=V comes with partial RoPE · every row but gpt-oss normalises q (and k)

Read it as a checklist:
QK-norm (card), partial RoPE (01), window (05), sink (card), K=V (03), gate (card).

Sink or gate:
both let a head output almost nothing. None of these four models uses both.

K=V needs partial RoPE:
the shared vector is also the value, so only a slice rotates (Gemma 4 ¼; DeepSeek-V4 64/512, output de-rotated).

03 · Share KV heads

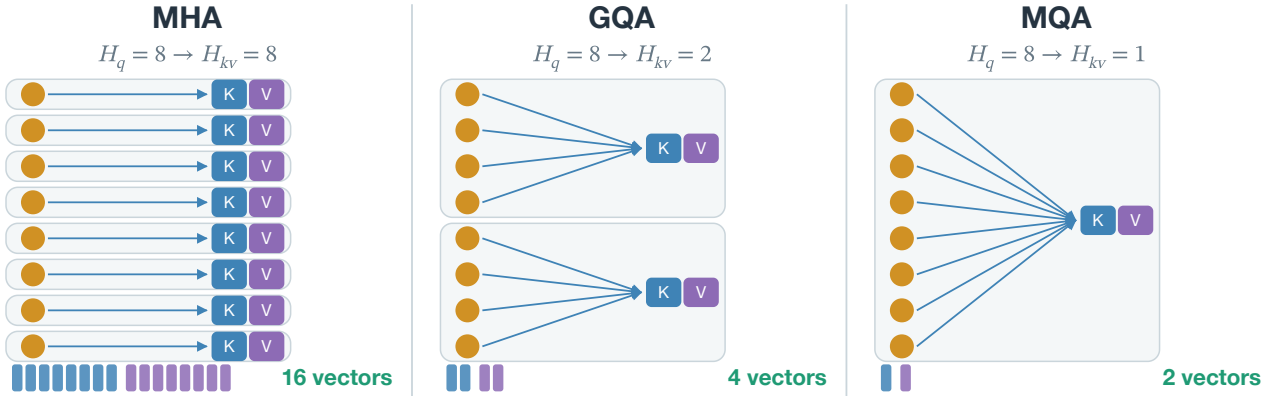
Decode re-reads the whole cache for every new token. First lever: store fewer K/V heads per token and keep every query head.

01 MHA → GQA → MQA: one knob, H_{kv}

MQA 2019 · Shazeer, Google · arXiv:1911.02150 — **GQA** 2023 · Ainslie et al., Google Research · EMNLP 2023 · arXiv:2305.13245 · Used in: PaLM, Falcon-7B, Gemma-2B (MQA); Llama 2-70B, Llama 3, Mistral 7B, Qwen2/3 (GQA)

$$o_{t,i} = \sum_{j \le t} \mathrm{softmax}_j\left(\frac{q_{t,i}^{\top} k_{j,g(i)}}{\sqrt{d_h}}\right) v_{j,g(i)}, \qquad g(i) = \left\lfloor \frac{i\,H_{kv}}{H_q} \right\rfloor$$

$$\text{cache / token / layer} = 2 \cdot H_{kv} \cdot d_h \quad \text{(K and V)}$$



Each query head keeps its own softmax.
Shared K/V does not mean shared attention patterns.

Quality:
uptrained GQA-8 lands close to MHA quality at near-MQA decode speed (reported, T5-XXL, GQA §4).

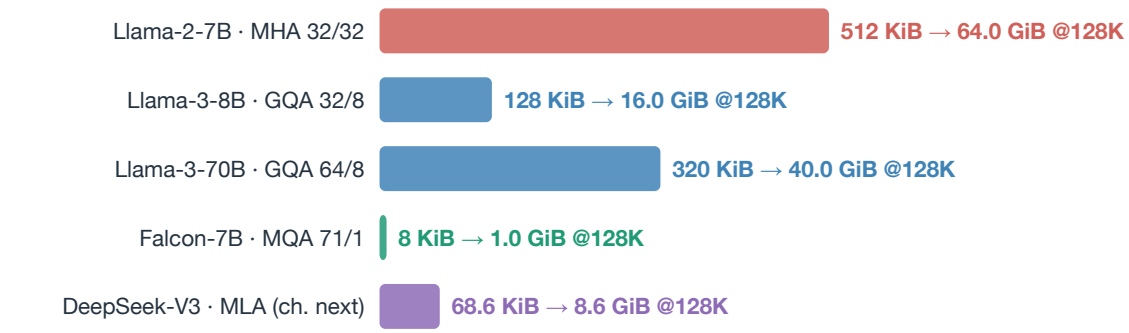
MHA → GQA conversion:
mean-pool each group's K/V projections, then uptrain for about 5% of the pre-training steps (GQA §2.1).

02 What it saves in real models

$$\text{KV bytes per token} = L \cdot 2 H_{kv} d_h \cdot s$$

$$\text{bytes / token} = L \cdot 2\ H_{kv}\ d_h \cdot s$$

BF16 (s = 2 B), all layers



Llama-3-70B: L=80, H_kv=8, d_h=128 → 80·2·8·128·2 B = 320 KiB per token.

Read it off `config.json` :
`num_hidden_layers` × 2 × `num_key_value_heads` × `head_dim` × bytes.

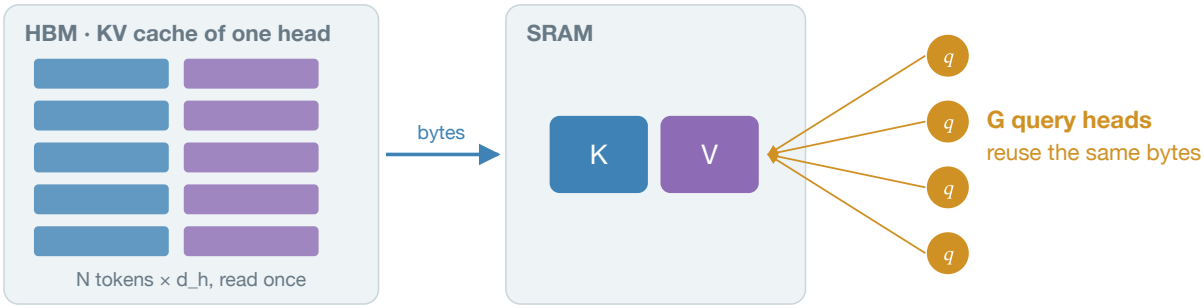
Width, not length.
Sharing shrinks each record. The number of records still grows with N.

Batch multiplies it.
32 concurrent 8K requests on Llama-3-70B need 80 GiB of KV in BF16.

03 Why decode benefits: intensity = group size

$$I = \frac{\text{FLOPs}}{\text{bytes}} \approx \frac{4\,G\,N d_h}{4\,N d_h} = G = \frac{H_q}{H_{kv}}$$

intensity $\approx 4GNd_h$ FLOP / $4Nd_h$ B $= G = H_q$ / H_{kv}





Even G = 8 is ~37× below the ridge: decode attention stays bandwidth-bound, so bytes moved ≈ time.

At decode, bytes moved ≈ time.
One KV byte feeds G heads under GQA but only one under MHA.

Ridge point:
H100 SXM ≈ 989 TFLOP/s ÷ 3.35 TB/s ≈ 295 FLOP/B. Attention decode sits far left of it.

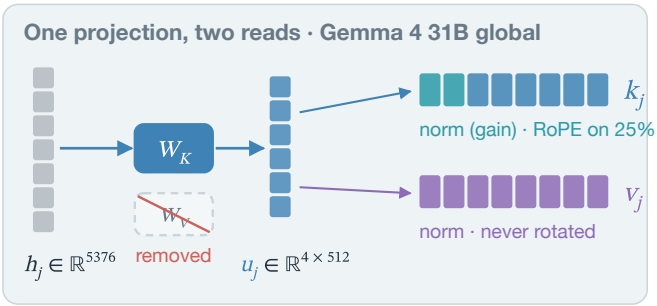
Tensor-parallel gotcha:
KV heads are split across GPUs. If H_{kv} < TP degree, KV heads get replicated and the saving shrinks.

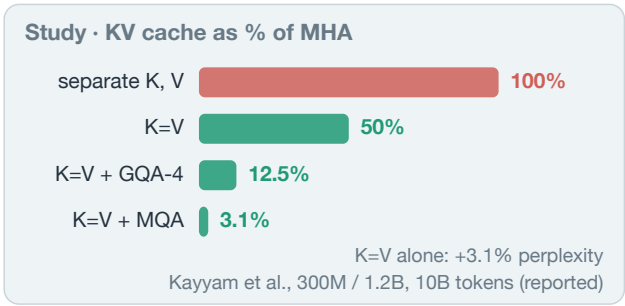
04 K=V: one vector is both key and value

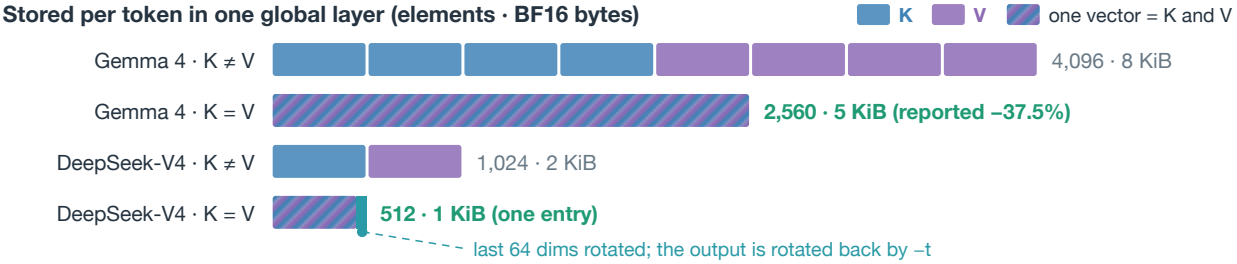
K=V 2026 · Gemma Team, Google · arXiv:2607.02770 — DeepSeek-AI · arXiv:2606.19348 — study: Kayyam et al. · ICML 2026 · arXiv:2606.04032 · Used in: Gemma 4 31B / 26B-A4B (global layers), DeepSeek-V4-Pro / Flash

$$u_j = W_K h_j, \qquad k_j = \text{RoPE}_j(\text{norm}_g(u_j)), \qquad v_j = \text{norm}(u_j)$$

$u_j = W_K\, h_j \qquad k_j = \text{RoPE}_j(\text{norm}_g(u_j)) \qquad v_j = \text{norm}(u_j)$







Why it works:
keys and values can share one subspace. Reported cost: +3.1% perplexity for −50% KV (Kayyam et al.).

Gotcha:
a rotated vector is a poor value. Gemma 4 never rotates v; DeepSeek-V4 rotates the output back by $-t$.

Numbers:
Gemma 4 reports −37.5% global KV, not −50%. V4 caches one 512-element entry per token (one KV head).

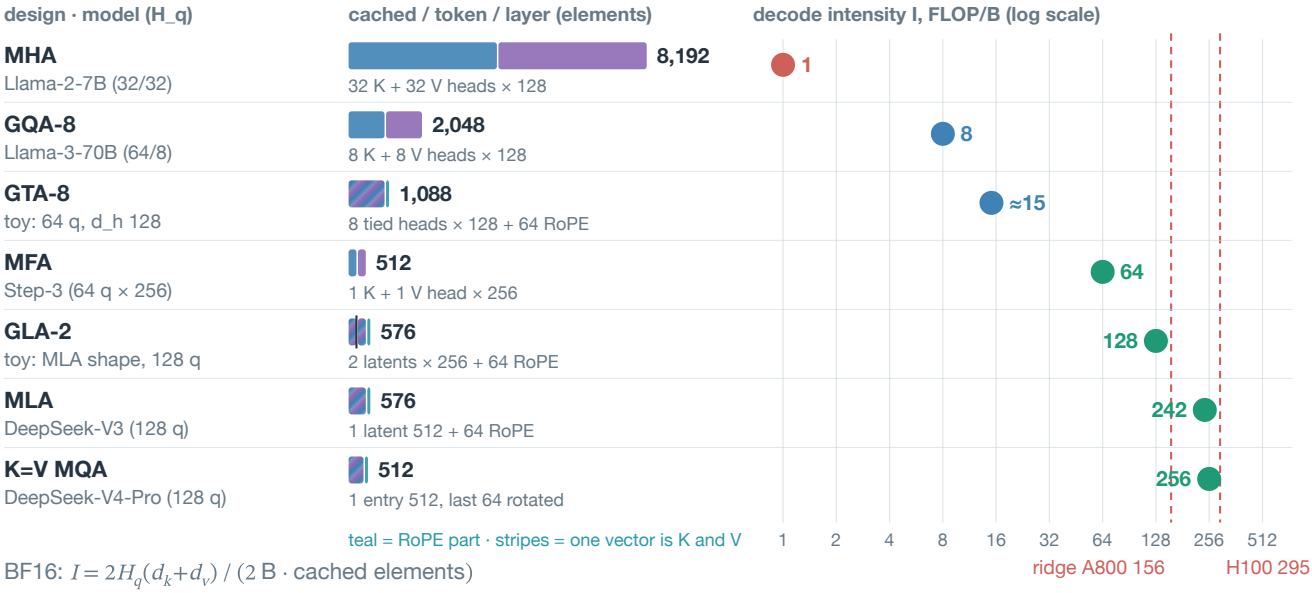
Intensity doubles:
the same bytes feed $q^\top k$ and $\sum_j p_j v_j$, so $I \approx 2G$ (next card).

05 Beyond GQA: MFA, GTA, GLA

MFA 2024 · Hu et al., StepFun · arXiv:2412.19255, Step-3 arXiv:2507.19427 — **GTA / GLA** 2025 · Zadouri, Strauss, Dao, Princeton · arXiv:2505.21487 · Used in: Step-3 (MFA)

$$I \approx \frac{2G}{m}, \qquad G = \frac{H_q}{\text{cached heads}}, \qquad m = \begin{cases} 2 & K \neq V \\ 1 & \text{tied or latent} \end{cases}$$

$I \approx 2G / m$ $\quad G$ = query heads per cached head $\quad m = 2$ (K, V) or 1 (tied / latent)



BF16: $I = 2H_q(d_k+d_v) / (2\,\text{B} \cdot \text{cached elements})$

Lens:
MHA ≈ 1, GQA ≈ G, MQA ≈ H_q, MLA ≈ $2H_q$ (Zadouri et al., Table 1). Past the ridge, extra heads cost time.

MFA:
one K and one V head of 256 serve 64 query heads with a rank-2048 query. Step-3 (8-bit KV): MFA 128, MLA 512 FLOP/B.

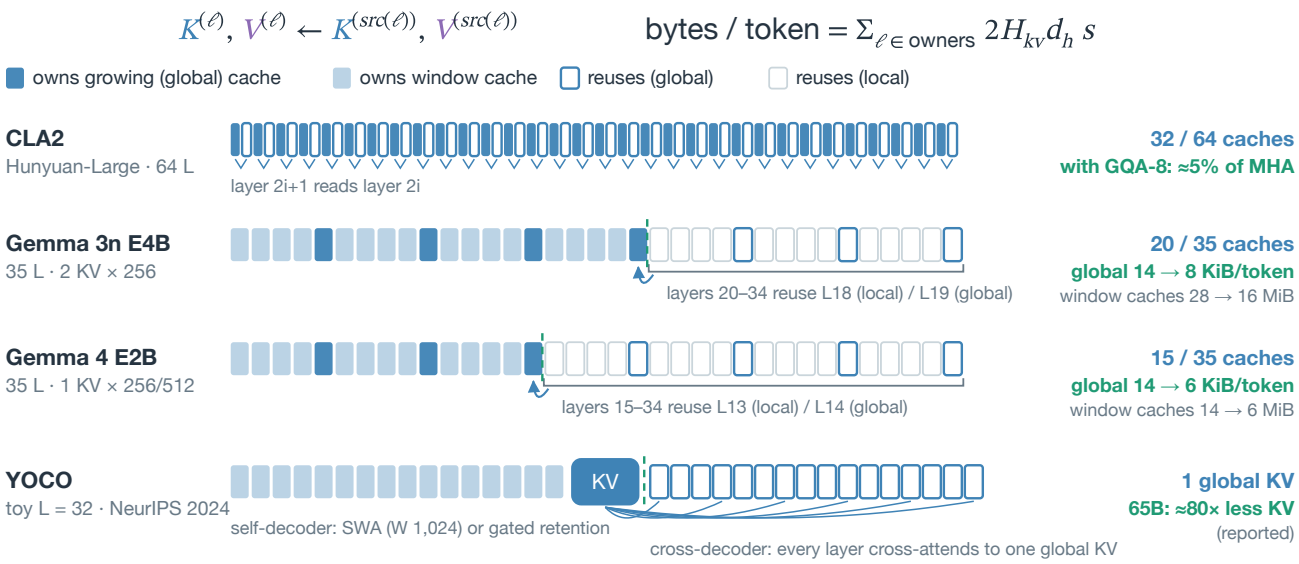
GTA:
the tied state is the value; the key is its first half plus a shared RoPE half. Reported: GQA quality, about half the cache.

GLA:
two latent heads instead of one, so TP=2 duplicates nothing: 640 vs 1,152 B/token/device (Table 5).

06 Cross-layer sharing: fewer layers own a cache

CLA 2024 · Brandon et al., MIT / IBM · NeurIPS 2024 · arXiv:2405.12981 — **YOCO** 2024 · Sun et al., Microsoft / Tsinghua · NeurIPS 2024 · arXiv:2405.05254 · Used in: Hunyuan-Large (CLA2), Gemma 3n, Gemma 4 E2B / E4B

$$K^{(\ell)}, V^{(\ell)} \leftarrow K^{(\mathrm{src}(\ell))}, V^{(\mathrm{src}(\ell))}, \qquad \text{bytes/token} = \sum_{\ell \in \text{owners}} 2 H_{kv} d_h \, s$$



prefill exit: layers above never produce KV, so prompt tokens can stop here (YOCO 2.87× prefill at 32K; Gemma 3n 2× vs Gemma 3 4B, reported)

Storage, not reads:
every consumer layer still reads the shared cache.

Prefill wins:
consumer layers produce no KV, so prompt tokens exit early (YOCO). Gemma 3n reports 2× prefill vs Gemma 3 4B.

Train it in:
shared layers have no `k_proj` / `v_proj` (HF Gemma 4 code). Gemma 4 shares 20/35 (E2B) and 18/42 (E4B) layers.

Stacks:
Hunyuan-Large combines GQA-8 with CLA2: about 95% below MHA (reported).

04 · Latent KV

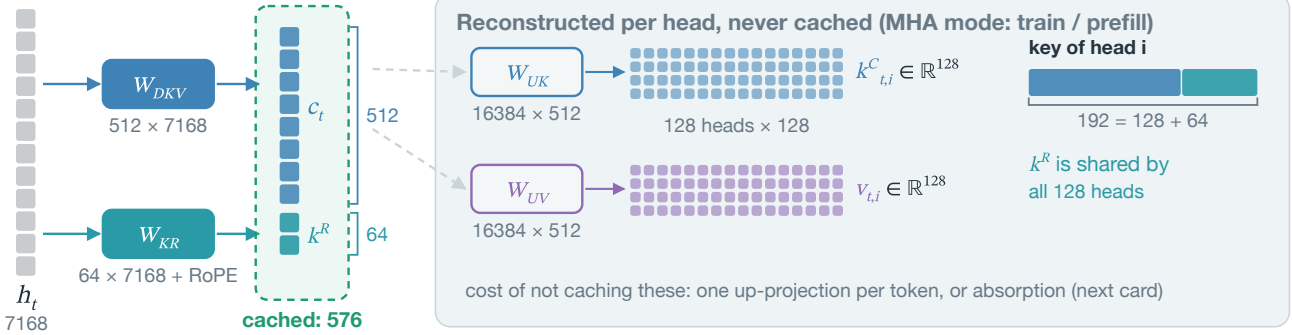
Head sharing throws away per-head views. MLA keeps all query heads yet caches one 576-element latent per token, trading cache bytes for decode FLOPs.

01 MLA: cache a latent, rebuild the heads

MLA 2024 · DeepSeek-AI · DeepSeek-V2 · arXiv:2405.04434 · Used in: DeepSeek-V2, V3, V3.2, Kimi K2, GLM-5, Mistral Large 3

$$c_t = W_{DKV} h_t \in \mathbb{R}^{d_c}, \quad k_t^R = \mathrm{RoPE}_t(W_{KR} h_t) \in \mathbb{R}^{d_r}, \quad k_{t,i}^C = W_{UK,i}\, c_t, \quad v_{t,i} = W_{UV,i}\, c_t$$

$$c_t = W_{DKV}\, h_t \in \mathbb{R}^{512} \qquad k_t^R = \mathrm{RoPE}_t(W_{KR}\, h_t) \in \mathbb{R}^{64} \qquad \text{cache} = 512 + 64 = 576$$



Elements per token per layer (DeepSeek-V3 shapes, BF16 bytes)



DeepSeek-V3, all 61 layers: 61 × 1,152 B = 70,272 B = 68.6 KiB per token (V2 Table 1: 576 ≈ GQA with 2.25 groups).

Numbers (V3 `config.json`):
d_c 512, d_r 64, 128 heads × 128, 61 layers → 61 × 576 × 2 B = 68.6 KiB per token.

Compare:
576 = 4.5 d_h, the cache of GQA with 2.25 groups (V2 Table 1), yet every head keeps its own K and V.

Query is low-rank too:
$c_t^Q = W_{DQ} h_t \in \mathbb{R}^{1536}$, not cached.

Cost:
the FP8 layout (656 B per token per layer) is in Serving the cache.

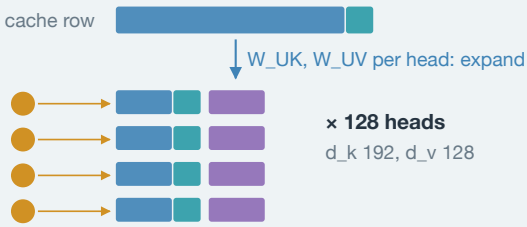
02 Weight absorption: decode is MQA over 576 dims

Absorbed MLA 2024 · DeepSeek-AI · DeepSeek-V2 §2.1 · arXiv:2405.04434; MHA vs MQA mode: DeepSeek-V3.2 App. A · arXiv:2512.02556 · Used in: DeepSeek-V3.x decode, FlashMLA

$$q_{t,i}^{C\top} W_{UK,i}\, c_j = (W_{UK,i}^{\top} q_{t,i}^{C})^{\top} c_j, \qquad o_{t,i} = W_{UV,i} \sum_j p_{t,i,j}\, c_j$$

$$q_i^{\top}(W_{UK,i} c_j) = (W_{UK,i}{}^{\top} q_i)^{\top} c_j \qquad o_i = W_{UV,i}\, \Sigma_j\, p_{ij} c_j$$

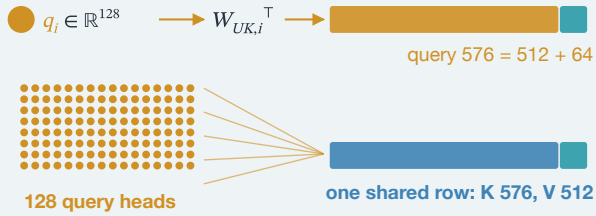
MHA mode · train / prefill



expanded per token: 128 × (192 + 128) = 40,960 elements

DeepSeek: training + prefill (V3.2 App. A)

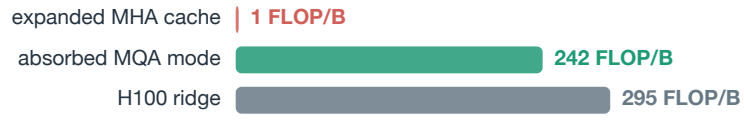
MQA mode · decode (absorbed)



$$o_i = (W_{O,i} W_{UV,i})\, z_i$$

$z_i = \Sigma_j\, p_{ij} c_j \in \mathbb{R}^{512}$ · W_{UV} folds into W_O

Per cached token per layer, BF16 (DeepSeek-V3: H_q = 128)



reads 576 × 2 B = 1,152 B · FLOPs 2·128·(576 + 512) = 278,528

toy check: same score

toy: $W_{UK} = [2\ 3]$, $c = [4;5]$, $q = 7$

$q(W_{UK} c) = 7 \cdot 23 = 161$

$(W_{UK}{}^{\top} q)^{\top} c = 14{\cdot}4 + 21{\cdot}5 = 161$

associativity: no approximation

Exact:
only the bracketing changes, so the absorbed score is the same number (toy check in the figure).

Train vs. infer:
MHA mode for training and prefill, MQA mode for decode (V3.2 App. A, stated for V3.1-Terminus).

Cost:
1,152 B feed 278,528 FLOPs: $I \approx 2H_q \approx 242$, near the H100 ridge of 295.

Gotcha:
keys never exist at decode, so QK-norm cannot apply; Kimi K2 clips weights instead (Stable softmax).

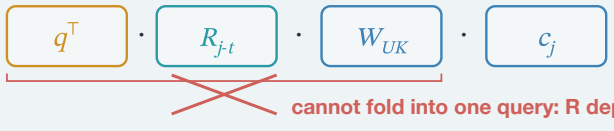
03 Decoupled RoPE: a separate 64-dim lane

Decoupled RoPE 2024 · DeepSeek-AI · DeepSeek-V2 §2.1.3 · arXiv:2405.04434 · Used in: DeepSeek-V2, V3, V3.2, Kimi K2, GLM-5, Mistral Large 3

$$e_{t,j,i} = \frac{(W_{UK,i}^{\top} q_{t,i}^{C})^{\top} c_j + (q_{t,i}^{R})^{\top} k_j^{R}}{\sqrt{d_h + d_r}}, \qquad k_j^{R} = \mathrm{RoPE}_j(W_{KR} h_j) \in \mathbb{R}^{64}$$

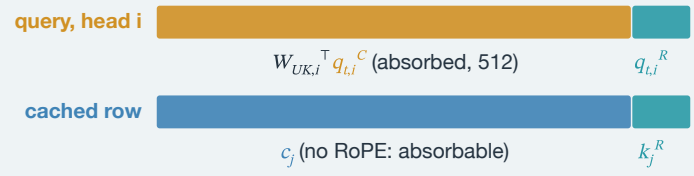
$$(R_t q)^{\top} R_j W_{UK} c_j = q^{\top} R_{j-t} W_{UK} c_j$$

RoPE on the content key: absorption breaks



matrix products do not commute:
W_UK would need a new absorbed query
for every cached position j (V2 §2.1.3)

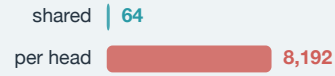
Decoupled lane: content 512 (no RoPE) + position 64 (RoPE)



$$e_{t,j,i} = ((W_{UK,i}{}^{\top} q^{C})^{\top} c_j + (q^{R})^{\top} k_j{}^{R}) / \sqrt{192}$$

192 = d_h + d_r = 128 + 64 (the MHA-mode head size)

RoPE key per token per layer



one k^R serves all 128 heads

64 of 576 cached = 11%

Why it breaks:
R_{j-t} sits between q and W_{UK} and changes with j; matrix products do not commute (V2 §2.1.3).

Why it is cheap:
one rotated key serves all 128 heads: 64 elements per token, not 8,192.

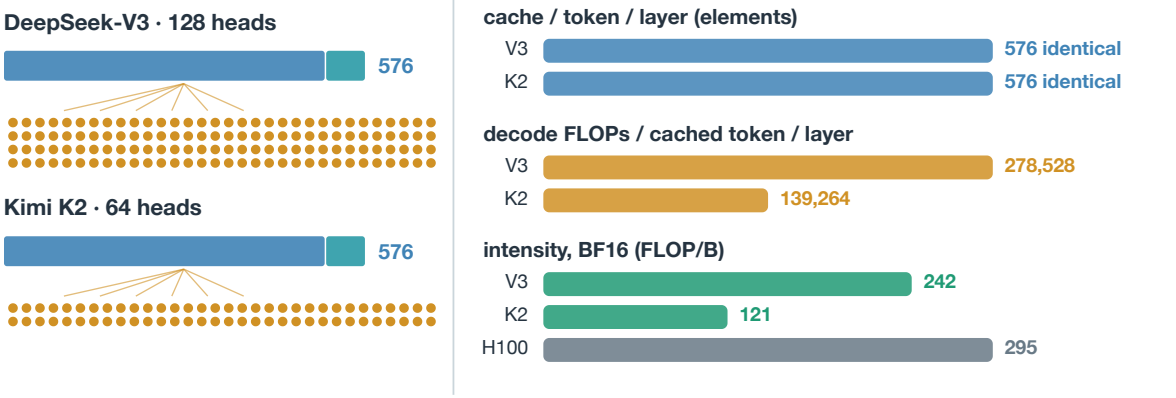
Not the only fix:
Kimi Linear and Kimi K3 run their MLA layers with NoPE (`mla_use_nope`); DeepSeek-V4 rotates 64 dims of a K=V entry.

Positions:
RoPE base and scaling are in Positions.

04 Head count: heads cost FLOPs, not cache

$$\text{cache/token/layer} = d_c + d_r = 576 \text{ for any } H_q, \qquad \text{decode FLOPs/cached token} = 2H_q\,(576 + 512)$$

$$\text{cache} = d_c + d_r = 576 \text{ for any } H_q \qquad \text{FLOP / cached token} = 2H_q(576 + 512)$$



Kimi K2 §2.3 (reported): 64 → 128 heads = +83% inference FLOPs at 128K tokens, for only 0.5–1.2% lower validation loss. Same 61 layers, same 68.6 KiB/token cache.

Invisible in the cache:
Kimi K2 and DeepSeek-V3 store identical 576-element rows over 61 layers.

Reported (Kimi K2 §2.3):
64 → 128 heads: +83% inference FLOPs at 128K, 0.5–1.2% lower validation loss. K2 kept 64.

Not solved:
one latent per token cannot be split across tensor-parallel ranks; each holds all 576 (GLA).

Model	L	H_q	$d_c + d_r$	KV per token (BF16)
DeepSeek-V2	60	128	512 + 64	67.5 KiB
DeepSeek-V3 / V3.2	61	128	512 + 64	68.6 KiB
Mistral Large 3	61	128	512 + 64	68.6 KiB
Kimi K2	61	64	512 + 64	68.6 KiB
GLM-5	78	64	512 + 64	87.8 KiB

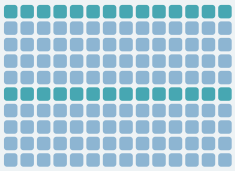
05 Converting MHA/GQA → MLA

MHA2MLA 2025 · Ji et al. · ACL 2025 · arXiv:2502.14837 — **TransMLA** 2025 · Meng et al. · arXiv:2502.07864 · Used in: research checkpoints (SmolLM, Llama-2-7B)

$$[W_K^{\text{nope}};\, W_V] \approx U_r \Sigma_r V_r^\top, \qquad W_{DKV} = \Sigma_r^{1/2} V_r^\top, \quad [W_{UK};\, W_{UV}] = U_r \Sigma_r^{1/2}$$

$$[W_K{}^{nope}; W_V] \approx U_r \Sigma_r\, V_r^\top \qquad W_{DKV} = \Sigma_r{}^{1/2}\, V_r^\top \qquad [W_{UK}; W_{UV}] = U_r \Sigma_r{}^{1/2}$$

1 · partial RoPE



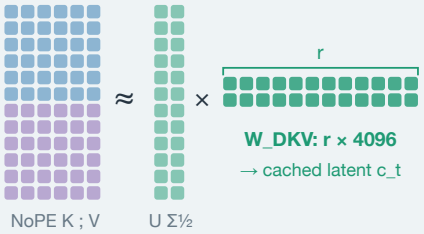
RoPE kept (few pairs)

NoPE → absorbable

W_K: 4096 × 4096 (32 heads × 128)

MHA2MLA: top-norm pairs per head
TransMLA: fold RoPE into one head

2 · joint SVD / PCA



NoPE K ; V — U Σ½

W_DKV: r × 4096
→ cached latent c_t

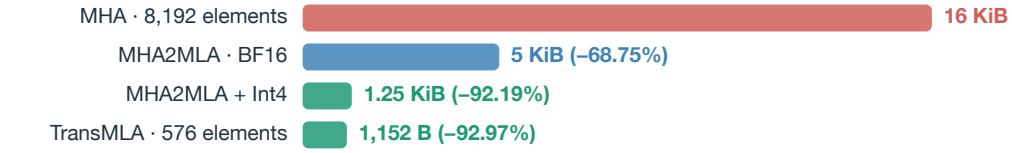
3 · short fine-tune

MHA2MLA
0.3–0.6% of pre-training data

TransMLA
6B tokens

not lossless: weights change, then recover

Llama-2-7B KV bytes per token per layer (reported reductions)



TransMLA 576 = 8,192 × (1 − 0.9297), the DeepSeek record size (computed)

Converts from:
MHA (MHA2MLA) or GQA (TransMLA). Rotated dims go first: they cannot be absorbed.

Numbers (Llama-2-7B):
MHA2MLA −68.75% in BF16, −92.19% with Int4, LongBench 27.4 → 26.4. TransMLA −92.97% after 6B tokens; reported 10.6× at 8K.

Not lossless:
the factorisation changes the function; short fine-tuning recovers it.

Compare:
GQA uptraining mean-pools heads; conversion keeps every head and re-factors its projections.

05 · Fixed patterns

Most layers do not need the whole history. Fix in advance which keys each query may see, and those layers stop growing with N.

01 Sparse Transformer → Longformer / BigBird: masks fixed by position

Sparse Transformer 2019 · Child et al., OpenAI · arXiv:1904.10509 — **Longformer** 2020 · Beltagy et al., AI2 · arXiv:2004.05150 — **BigBird** 2020 · Zaheer et al., Google · NeurIPS 2020 · arXiv:2007.14062 · Used in: GPT-3, gpt-oss (dense/banded alternation)

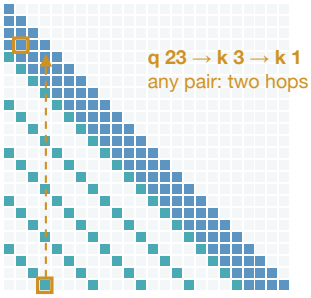
$$o_t = \sum_{j \in A_t} \mathrm{softmax}_{j \in A_t}\left(\frac{q_t^\top k_j}{\sqrt{d_h}}\right) v_j, \qquad A_t \text{ depends on positions only, never on content}$$

$$A_t^{(1)} = \{t\text{-}l, \ldots, t\} \qquad A_t^{(2)} = \{j : (t\text{-}j) \bmod l = 0\} \qquad l \approx \sqrt{n} \;\rightarrow\; O(n\sqrt{n}) \text{ pairs}$$

local band · strided · global · random · not computed

Sparse Transformer · 2019

causal · band l + stride l

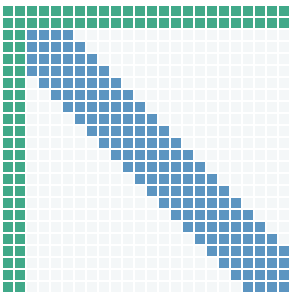


toy n = 24: 150 of 300 pairs

real: l ≈ √n, two heads

Longformer · 2020

bidirectional · window + global

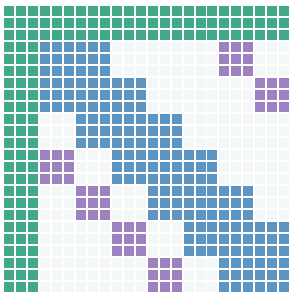


toy n = 24: 234 of 576 pairs

real: w = 512 at n = 4,096

BigBird · 2020

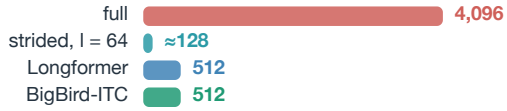
blocks of 3 · window + global + random



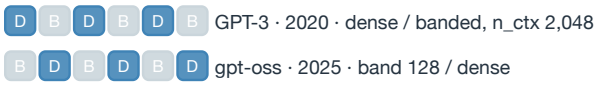
toy n = 24: 360 of 576 pairs

real: b = 64 · g 2b · w 3b · r 3b

Keys per query at n = 4,096



Same idea, one layer at a time



Why it works:
depth routes information. Band then stride connects any pair in two layers, at $O(n\sqrt{n})$ cost.

Gotcha:
strided routing failed on text (no periodic structure). The paper's "fixed" variant uses blocks plus summary columns instead.

Used in:
GPT-3 alternated dense and locally banded layers. gpt-oss does the same "following GPT-3", band 128.

Not solved:
global and random links are chosen before any query exists. Content-chosen reads: 06.

02 Sliding window + global layers in production

SWA : global 2023–26 · Mistral 7B arXiv:2310.06825 · Gemma 2 arXiv:2408.00118 · Gemma 3 arXiv:2503.19786 · gpt-oss arXiv:2508.10925 · MiMo-V2-Flash arXiv:2601.02780 · Used in: Gemma 2/3, gpt-oss, OLMo 3, MiMo-V2-Flash, DeepSeek-V4-Flash

$$\mathrm{KV}(N) = N \sum_{\ell \in \text{global}} c_\ell \;+\; \min(N, W) \sum_{\ell \in \text{local}} c_\ell, \qquad c_\ell = 2 H_{kv} d_h \, s$$

$\mathrm{KV}(N) = N \cdot \Sigma_{\text{global}}\, c_\ell + \min(N,\, W) \cdot \Sigma_{\text{local}}\, c_\ell$ $\qquad c_\ell = 2\, H_{kv}\, d_h\, s$ per layer

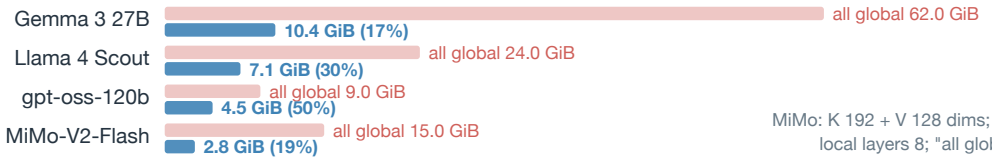
model	year	layers	local : global	W	first 12 layers
Mistral 7B	2023	32	all local	**4,096**	L L L L L L L L L L L L
Gemma 2 27B	2024	46	1 : 1	**4,096**	L G L G L G L G L G L G
Gemma 3 27B	2025	62	5 : 1	**1,024**	L L L L L G L L L L L G
gpt-oss-120b	2025	36	1 : 1	**128**	L G L G L G L G L G L G
Llama 4 Scout	2025	48	3 : 1	**chunk 8,192**	C C C G C C C G C C C G
OLMo 3 7B	2025	32	3 : 1	**4,096**	L L L G L L L G L L L G
MiMo-V2-Flash	2026	48	39 : 9	**128**	G L L L L G L L L L L G
DeepSeek-V4-Flash	2026	43	L1–2 local	**128**	L L then CSA / HCA + 128 SWA branch (ch. 08)

L local window · G global (full cache) · C chunked

trend: W 4,096 → 1,024 → 128

KV cache at N = 128K, BF16, whole model

computed from each config.json



MiMo: K 192 + V 128 dims; global layers 4 KV heads, local layers 8; "all global" = 48 × its global layer

Ring buffer:
a local layer writes token t into slot $t \bmod W$. Mistral 7B: 8× less cache at 32K.

Global layers set the bill.
They grow with N, and only they retrieve beyond W.

Small windows need a sink.
MiMo 32B ablation: W = 128 beat 512 on RULER-32K (89.4 vs 84.7). See learned sinks.

Numbers:
Gemma 3 report: global-only KV overhead about 60%, under 15% with 1,024-token windows.

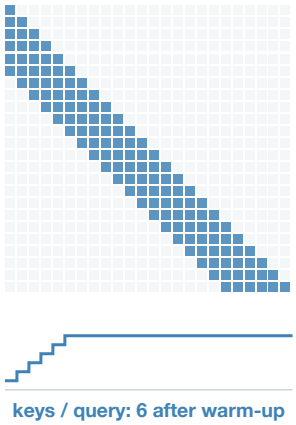
03 Chunked local attention (Llama 4) vs sliding window

Chunked attention + iRoPE 2025 · Meta · Llama 4 blog · config `attention_chunk_size` 8192, `no_rope_layers` 1,1,1,0 · Used in: Llama 4 Scout

$$A_t^{\text{chunk}} = \{\, j \le t : \lfloor j/C \rfloor = \lfloor t/C \rfloor \,\}, \qquad |A_t^{\text{chunk}}| = (t \bmod C) + 1$$

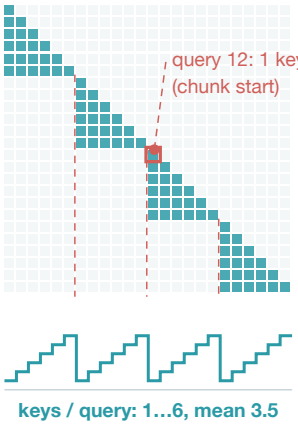
$A_t^{\text{chunk}} = \{\, j \le t : \lfloor j / C \rfloor = \lfloor t / C \rfloor \,\}$ vs. $A_t^{\text{SWA}} = \{\, j : t - W < j \le t \,\}$

Sliding window
toy W = 6



keys / query: 6 after warm-up

Chunked (Llama 4)
toy C = 6



keys / query: 1…6, mean 3.5

Llama 4 Scout · 48 layers

C C C G C C C G C C C G

C: chunk 8,192 + RoPE (36 layers)
G: full causal + NoPE (12 layers)

KV per layer at 128K, BF16
8 KV heads × 128 = 4 KiB / token

global — **512 MiB**
chunked — **32 MiB**

whole model at 128K: 7.1 GiB
vs 24 GiB if all global (30%)

keys per query, C = W = 8,192:
SWA 8,192 · chunked mean 4,096.5
chunked: half the pairs, blind spots

Compare:
same cache cap as SWA with $W = C$, half the pairs, but each chunk start is nearly blind locally.

Why chunks:
the mask is block-diagonal, so each chunk is an ordinary causal attention and the cache empties at every boundary.

iRoPE:
chunked layers keep RoPE; every 4th layer is global with no RoPE (positions).

04 StreamingLLM: window + 4 sink tokens

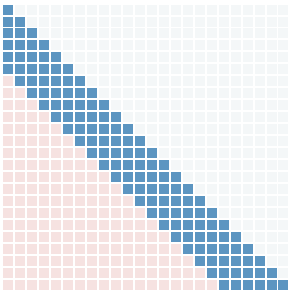
StreamingLLM 2023 · Xiao et al., MIT · Meta AI · ICLR 2024 · arXiv:2309.17453 · Used in: training-free streaming; successor = learned sink logit (gpt-oss, MiMo-V2-Flash, DeepSeek-V4)

$$A_t = \{0, 1, 2, 3\} \cup \{t - W + 1, ..., t\}, \qquad \text{pos}(j) = \text{rank of } j \text{ inside the cache}$$

$A_t = \{0,1,2,3\} \cup \{t\text{-}W\text{+}1, \ldots, t\}$ pos(j) = rank of j inside the cache

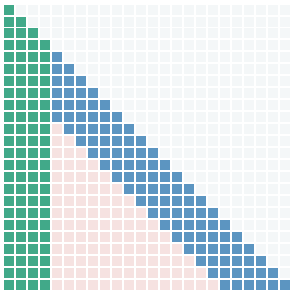
sink window evicted

Window only
toy W = 6, sinks evicted



PPL 5158.07

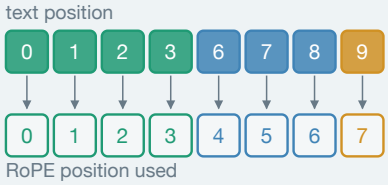
StreamingLLM
4 sinks + W = 6



PPL 5.40

Llama-2-13B, PG19 · cache 0 + 1024 vs 4 + 1020 (reported)

Positions inside the cache



tokens 4, 5 evicted; 9 is new
example from the paper, §3.2

Cost: constant
cache = 4 + W records, any length
Llama-2-7B, 2,048 slots × 512 KiB
= 1 GiB, streamed to 4M tokens
up to 22.2× vs window + recompute

Why it works:
softmax must sum to 1, so heads park spare mass on the first tokens. Evict them and that mass lands elsewhere.

Numbers:
4 sinks suffice; 1–2 do not (§4.4). Up to 22.2× faster than a window with recomputation.

Not solved:
stability, not memory. Recall drops to zero once the answer is farther back than the cache (App. C).

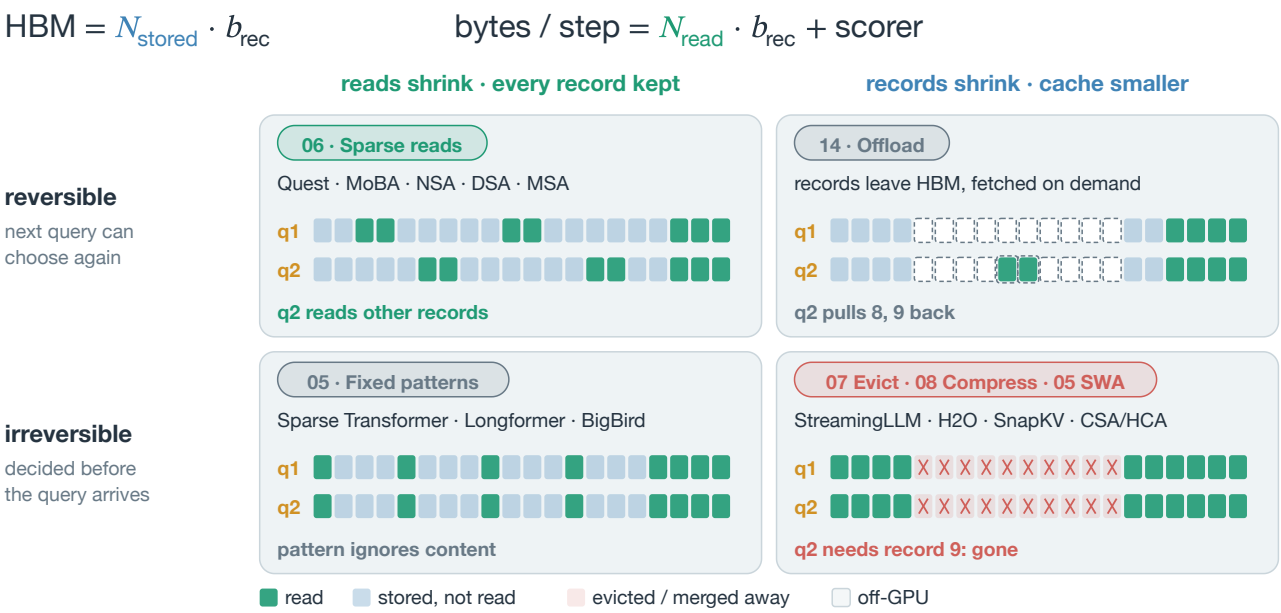
Learned version:
one sink logit per head replaces the 4 tokens: sinks. Smarter eviction: 07.

06 · Sparse reads

Keep every record, but let each query read only a few: a cheap scorer picks them, and exact attention runs on the picks.

01 What shrinks: reads or records?

$$\text{HBM} = N_{\text{stored}} \cdot b_{\text{rec}}, \qquad \text{bytes per step} = N_{\text{read}} \cdot b_{\text{rec}} + \text{scorer}$$



Llama-3-8B · 128K · BF16
128 KiB per token

read 2,048 of 131,072 (06)
HBM 16 GiB · 256 MiB moved per step

keep 2,048 (07)
HBM 256 MiB · 256 MiB moved per step

Reversible:
a later query can still reach every record. Only sparse reads and offload keep that property.

Cost:
sparse reads cut bandwidth, not memory. The scorer still touches all N records per query (last card).

Compare:
fixed masks 05 · eviction 07 · fewer records 08 · offload 14.

02 Training-free prefill: MInference → XAttention

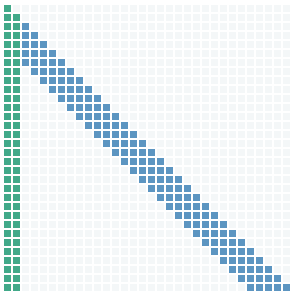
MInference 2024 · Jiang et al., Microsoft · NeurIPS 2024 · arXiv:2407.02490 — **FlexPrefill** 2025 · ICLR 2025 · arXiv:2502.20766 — **XAttention** 2025 · MIT · ICML 2025 · arXiv:2503.16428 · Used in: Qwen2.5-1M

$$M_{ij} = 1 \Leftrightarrow j \in V \vee (i-j) \in S, \qquad V, S = \text{top columns / diagonals of softmax}(Q_{-64:} K^\top)$$

$$M_{ij} = 1 \iff j \in V \vee (i-j) \in S \qquad V, S = \text{top columns / diagonals of softmax}(Q_{-64:} K^\top)$$

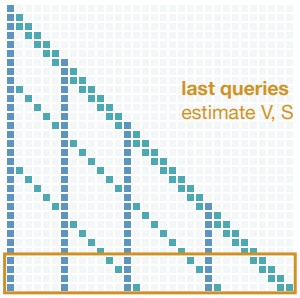
first tokens · local / vertical · slash · block

A-shape
static



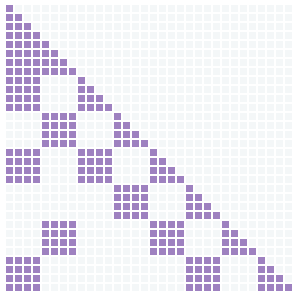
1,024 first + 4,096 local

Vertical-Slash
dynamic



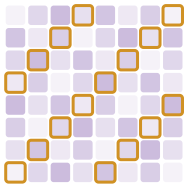
indices from last 64 queries

Block-Sparse
dynamic



64×64 blocks, mean-pooled, top-100

XAttention scorer
one 8×8 block, stride 4



score = Σ gold cells
keep fewest blocks with
Σ softmax(score) ≥ τ

128K: 6.89% density (S = 8)

1M-token prefill, 8B model, one A100 (reported)

dense	30 min
MInference	3 min

In production

Qwen2.5-1M: Vertical-Slash prefill,
3–7× faster at 1M (H20 / A100)
decode untouched: full KV kept

Prefill only:
the full KV cache is still written, and decode is untouched.

Per head:
an offline search fixes each head's pattern; indices are rebuilt for every input.

Threshold beats fixed k:
FlexPrefill (JS-divergence test) and XAttention (antidiagonal sums) keep blocks until their mass reaches τ.

Used in:
Qwen2.5-1M serves Vertical-Slash prefill: reported 3–7× faster at 1M tokens.

03 Quest: page bounds choose what decode reads

Quest 2024 · Tang et al., MIT Han Lab · ICML 2024 · arXiv:2406.10774 · Used in: training-free decode on any MHA/GQA model

$$U_p = \sum_{i=1}^{d} \max(q_i m_{p,i},\ q_i M_{p,i}) \ \geq\ \max_{j \in p} q^\top k_j, \qquad \text{read } \mathrm{Top}_K(U) \text{ pages}$$

$$U_p = \Sigma_i \max(q_i\, m_{p,i},\ q_i\, M_{p,i}) \ \geq\ \max_{j \in p} q^\top\, k_j$$

q (toy)		page A	page B	page C	page D
2	ch 1	[0, 2]	[0, 1]	[−1, 0]	[0, 1]
−1	ch 2	[0, 1]	[0, 1]	[0, 2]	[−1, 0]
1	ch 3	[0, 1]	[0, 1]	[−1, 0]	[0, 1]
1	ch 4	[0, 1]	[0, 1]	[0, 1]	[0, 1]
		U = 6	**U = 4**	**U = 1**	**U = 5**
		true max 6	true max 3	true max −1	true max 3

page A: U = max(0, 4) + max(0, −1) + max(0, 1) + max(0, 1) = 6

bars: [min, max] per channel (axis −2…2) · green: top-2 by U, read exactly

Real scale · decode

32K tokens / page 16 = 2,048 pages
budget 2,048 tokens = 128 pages



read 6.25% of pages

metadata: min + max per 16 keys
= +12.5% of K stored

reported (§4.3):
7.03× self-attention at 32K
passkey 100K: 100% at 2,048
layers 1–2 stay dense

cache size unchanged

Why it works:
each channel's product peaks at an interval end, so U_p never underestimates a page.

Selection, not eviction:
memory grows by 12.5% of K; bandwidth shrinks.

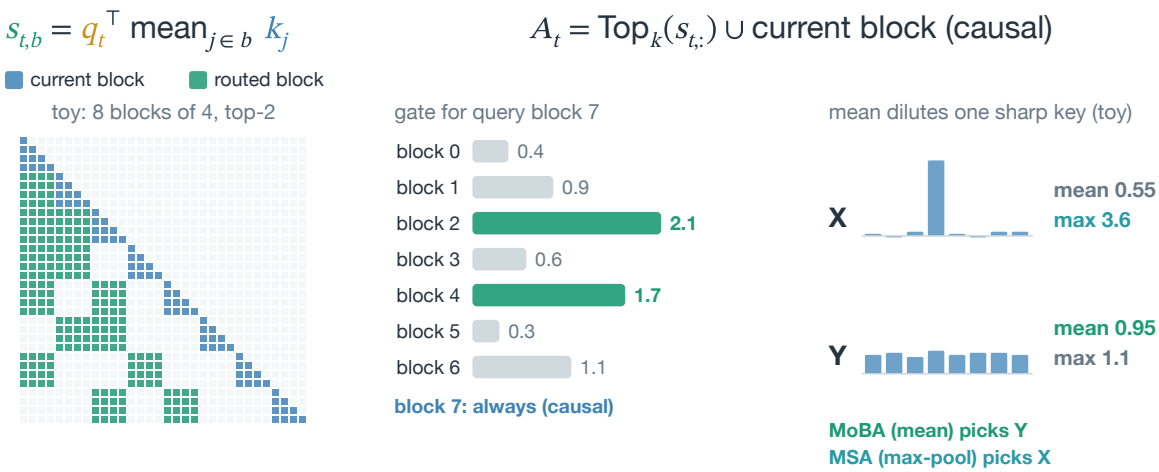
Gotcha:
wide pages loosen the bound (page D: bound 5, true 3). Hence 16-token pages.

Lineage:
cheap scorer → top-k → exact attention. DSA trains the scorer instead.

04 MoBA: route each query to a few blocks

MoBA 2025 · Lu et al., Moonshot AI · arXiv:2502.13189 · Used in: Kimi long-context service

$$s_{t,b} = q_t^\top \text{mean}_{j \in b} k_j, \qquad A_t = \text{Top}_k(s_{t,:}) \cup \text{current block (causal)}$$



Share of keys read (paper settings)



reported: 6.5× prefill at 1M
RULER 128K: 0.7818 vs 0.7849 full
last 3 layers kept full in SFT

No router weights:
the gate is q against mean keys. Continued training adapts q and k to it.

Gotcha:
a mean dilutes one sharp key in a 4,096-token block. MSA max-pools instead (last card).

Hybrid:
the last 3 layers stay full attention; SFT's masked prompt loss starved sparse layers of gradient.

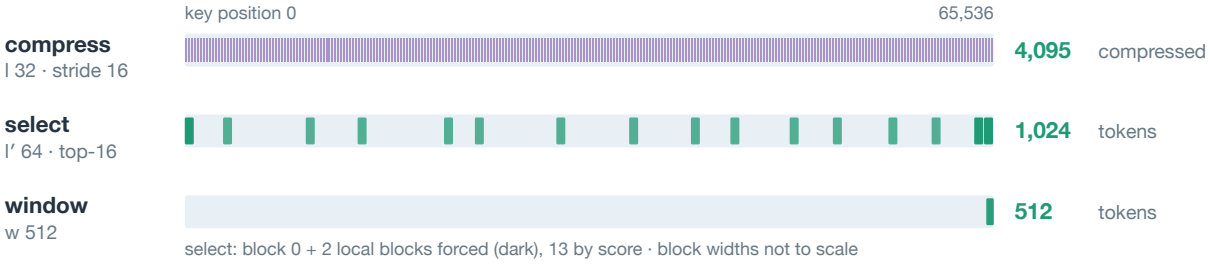
Numbers:
reported 6.5× prefill at 1M; RULER 128K 0.7818 vs 0.7849 full.

05 NSA: compress, select, slide

NSA 2025 · Yuan et al., DeepSeek-AI / PKU · ACL 2025 · arXiv:2502.11089 · Used in: research 27B MoE; selection reused by InfLLM-v2 (MiniCPM4.1)

$$o_t = \sum_{c \in \{\text{cmp, slc, win}\}} g_t^c \, \text{Attn}(q_t, K_t^c, V_t^c), \qquad g_t^c = \sigma(\text{MLP}(x_t))$$

$o_t = \Sigma_{c \in \{\text{cmp, slc, win}\}}\ g_t^{\,c} \cdot \text{Attn}(q_t, K_t^{\,c}, V_t^{\,c})$ $\qquad g_t^{\,c} = \sigma(\text{MLP}(x_t))$



Free scorer: the compress branch already has scores



Reads per query at 64K (27B MoE, GQA 4 × 16 heads)



reported vs FlashAttention-2 at 64K:
9.0× forward · 6.0× backward
decode memory access: 11.6× less

Free scorer:
selection reuses the compression branch's attention scores, so no separate indexer.

Group-shared:
one top-16 per GQA group, so each KV block is loaded once for 16 heads.

Gotcha:
three branches with their own KV projections. InfLLM-v2 reuses the dense KV: 96 blocks of 64 (≈6K tokens), no new parameters.

Bridge:
the compression branch starts 08.

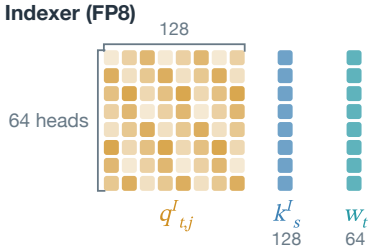
06 DSA: a lightning indexer picks 2,048 tokens

DSA 2025 · DeepSeek-AI · DeepSeek-V3.2 report · arXiv:2512.02556 · Used in: DeepSeek-V3.2(-Exp), GLM-5

$$I_{t,s} = \sum_{j=1}^{64} w_{t,j}\, \text{ReLU}(q^I_{t,j} \cdot k^I_s), \qquad A_t = \text{Top}_{2048}(I_{t,:})$$

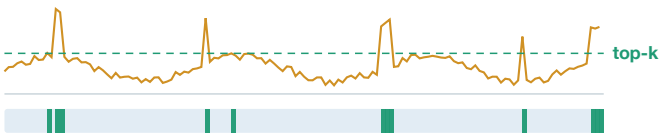
$I_{t,s} = \Sigma_{j=1}^{64}\ w_{t,j} \cdot \text{ReLU}(q^I_{t,j} \cdot k^I_s)$ $\qquad A_t = \text{Top}_{2048}(I_{t,:})$

Indexer (FP8)



one key head shared by 64 query heads; ReLU each

scores over N = 131,072 keys → top-2,048 (1.56%)



every key scored (O(N) per query); only lit keys go to the main attention

MQA-mode MLA: 128 query heads share each latent

512 + 64 dims = 656 B / token in FP8 (vLLM layout)

Multiply-accumulates per decode token, one layer, N = 128K



indexer > main once N > 2,048 × 17 = 34,816 tokens (derived)

Two-stage training (V3.2)

dense warm-up · 1,000 steps · 2.1B tok
indexer only, KL to summed attention

sparse · 15,000 steps · 943.7B tok
all weights; indexer input detached

ReLU per head, then weight.
ReLU was chosen for throughput; the indexer runs in FP8.

Cost:
main attention drops to $O(Nk)$, but the indexer stays $O(N^2)$ and outweighs it at 128K (derived 3.8×).

Token-level, shared:
one top-2,048 set per token for all 128 heads, so reads are gathers, not blocks.

Cache not smaller:
indexer keys are stored too. Latent bytes read per layer at 128K: 1.3 MB vs 86 MB dense.

07 2026 indexers: same reads, cheaper scorer

$$\text{index dot products per query} \approx \frac{H_I N}{r\,s}, \qquad r = \text{tokens per scored key},\ s = \text{layers per indexer}$$

index dots / query ≈ $H_I \cdot N \;/\; (r \cdot s)$

r = tokens per scored key · s = layers per indexer

method	model	H_I	r	s	index dots per query at N = 1M	main read
DSA	V3.2 · GLM-5	64	1	1	64M	2,048 tokens
IndexShare	GLM-5.2	64	1	4	16M	2,048 tokens
CSA (→ 08)	V4-Flash	64	4	1	16M	512 entries × 4
MSA	MiniMax-M3	4	1	1	4M	16 × 128 / group
QSA	Qwen3.8-Flash-Next	4	4	1	1M	2,048 tokens

Same budget, cheaper scorer: every design reads ≈ 2,048 tokens per query.
The O(N) indexer pass is what is left to cut: fewer heads (MSA, QSA), pooled keys (QSA, CSA), shared layers (IndexShare).

index head dim 128 for DSA, CSA, QSA (CSA in FP4); MSA index dim not stated · MSA reads 16 blocks per GQA group

MSA
(MiniMax-M3, arXiv:2606.13392): top-16 blocks of 128 per GQA group, scored by max-pooling. Reported 28.4× less attention compute at 1M.

QSA
(Qwen3.8-Flash-Next, arXiv:2608.30320): fills the 1-in-4 full-attention layers of a GDN hybrid (12).

IndexShare
(GLM-5.2): IndexCache (arXiv:2603.12201) reuses top-k across layers: 75% fewer indexer computations.

Recipe:
MSA and QSA copy DSA's KL warm-up of the indexer alone, then joint sparse training.

07 · Evict from the cache

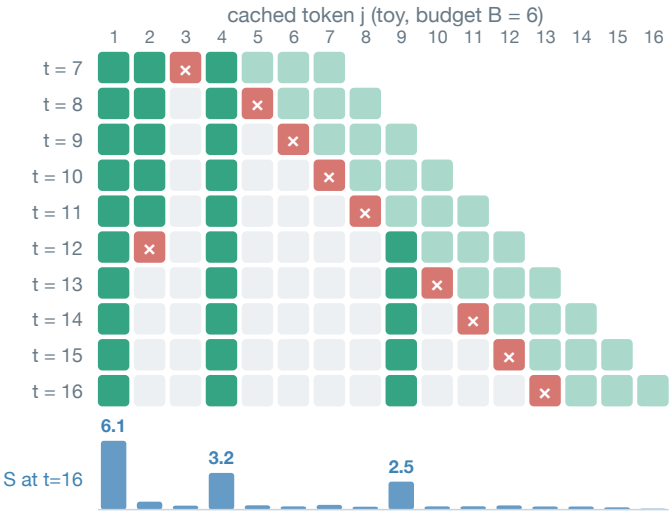
Selection reads fewer records but keeps them all. Eviction deletes records for good, so it must guess what future queries will need.

01 H2O: heavy hitters plus a recent window

H2O 2023 · Zhang et al., UT Austin / Stanford / Meta / CMU · NeurIPS 2023 · arXiv:2306.14048 · Used in: NVIDIA kvpress (ObservedAttentionPress); the standard eviction baseline

$$S_j \leftarrow S_j + a_{t,j}, \qquad \mathcal{K}_t = \text{Recent}_{B/2}(t) \cup \text{Top}_{B/2}(S), \qquad j \notin \mathcal{K}_t \Rightarrow \text{evicted for good}$$

$S_j \leftarrow S_j + a_{t,j}$ keep $= \text{Recent}_{B/2} \cup \text{Top}_{B/2}(S)$ evict the rest, forever



- heavy hitter (Top-3 of S)
- recent window (last 3)
- evicted at this step
- gone for every later query

Real setting: 20% budget = 10% heavy + 10% recent

Llama-3-8B, 32K tokens, BF16 KV:



reported (OPT-6.7B / 30B):
up to 29× throughput vs DeepSpeed, HF Accelerate; 3× vs FlexGen

early tokens are summed longest → bias

Why it works:
a few tokens collect most of the attention, and their accumulated score stays high across steps.

Gotcha:
it needs $a_{t,j}$ at every step. Fused FlashAttention kernels never write it out.

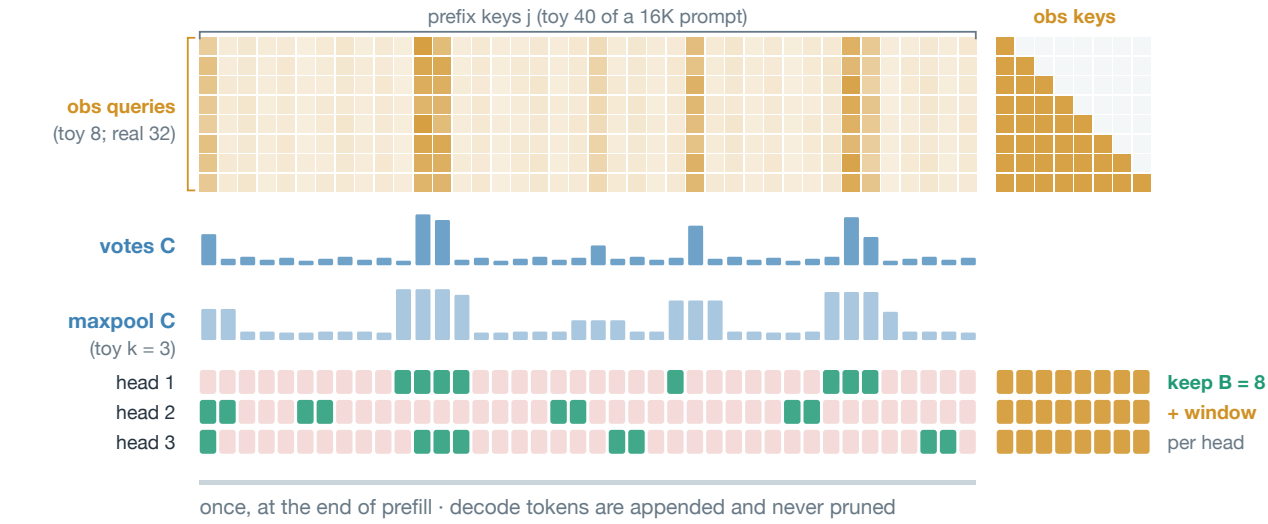
Gotcha:
early tokens have been summed for the most steps, so the score is biased toward the start of the context.

Numbers (reported):
20% budget. Up to 29× throughput vs DeepSpeed and Accelerate, and 3× vs FlexGen, on OPT-6.7B/30B.

02 SnapKV: the prompt's tail votes once

SnapKV 2024 · Li et al., UIUC / Cohere · NeurIPS 2024 · arXiv:2404.14469 · Used in: NVIDIA kvpress (SnapKVPress); the scorer behind PyramidKV and Ada-KV

$$C_j = \sum_{i \in \text{obs}} \text{softmax}\left(\frac{Kq_i}{\sqrt{d_h}}\right)_j, \qquad \mathcal{K}^{(h)} = \text{Top}_B(\text{maxpool}_7(C)) \cup \text{obs}$$



Real: window 32 · max-pool kernel 7 · budget 1024 / 2048 / 4096 · 16K prompt at budget 1024 keeps 6.25% + 32
reported at 16K: 3.6× generation speed, 8.2× memory efficiency · up to 380K tokens on one A100-80GB

Once, after prefill:
generated tokens are appended and never pruned, so long outputs still grow the cache.

Why pool:
max-pooling also keeps the neighbours of a hit, so a multi-token string such as "order 7314" survives whole.

Gotcha:
the votes come from the prompt's last 32 tokens. If the real question arrives later, the wrong tokens voted.

Numbers (reported):
3.6× faster generation and 8.2× better memory efficiency at 16K. Up to 380K tokens fit on one A100-80GB.

03 PyramidKV and Ada-KV: where the budget goes

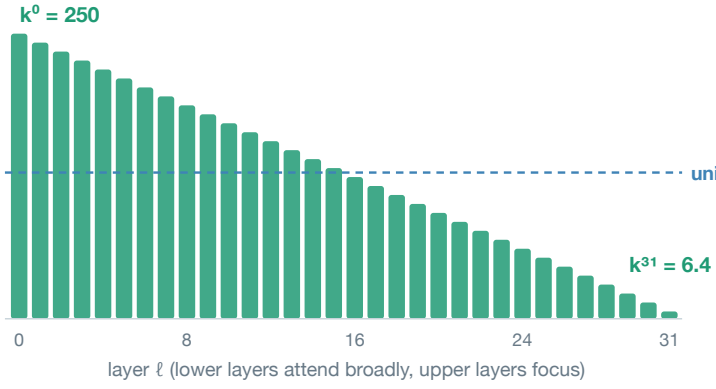
PyramidKV 2024 · Cai et al., UW–Madison / PKU / Microsoft · arXiv:2406.02069 — **Ada-KV** 2024 · Feng et al., USTC · NeurIPS 2025 · arXiv:2407.11550 · Used in: NVIDIA kvpress (PyramidKVPress, AdaKVPress)

$$\text{PyramidKV:}\quad k^{\ell} = k^0 - \frac{\ell}{L-1}(k^0 - k^{L-1}), \qquad k^{L-1} = \frac{k_{\text{tot}}}{\beta L}, \qquad k^0 = \frac{2k_{\text{tot}}}{L} - k^{L-1}$$

$$\text{Ada-KV:}\quad B_h = \#\{\text{entries of head } h \text{ in } \text{Top}_B(C^{(1)}, \ldots, C^{(H)})\}$$

$k^{\ell} = k^0 - \ell(k^0 - k^{L-1}) / (L-1)$ $\qquad k^{L-1} = k_{tot} / (\beta L)$ $\qquad k^0 = 2k_{tot} / L - k^{L-1}$

PyramidKV · Llama-3-8B, L = 32, mean 128 per head, β = 20



both splits spend 32 × 128 = 4,096 entries per head across layers

Ada-KV · one layer, toy

scores, 12 cached tokens	Ada	uniform
h1	6	3
h2	3	3
h3	2	3
h4	1	3

layer budget B = 12: take the global top-12 over all heads, then count **per head → 6, 3, 2, 1**

reported:
PyramidKV ≈ full KV on LongBench at 12%
Ada-SnapKV / Ada-Pyramid: plug-in, tested on 13 RULER + 16 LongBench sets

Allocation, not scoring:
both reuse SnapKV's window scores. Only the share each layer or head gets changes.

Why a pyramid:
attention is spread wide in lower layers and concentrated in upper layers, so upper layers need fewer entries.

Why per head:
Ada-KV bounds the output error by the attention mass it drops. A pooled Top-B drops the least mass.

Numbers (reported):
PyramidKV matches full KV on LongBench with 12% of the cache. With 128 entries, LLaMA-3-70B scores 100.0 on Needle.

04 KVzip: keep what the model needs to repeat the context

KVzip 2025 · Kim et al., Seoul National University / NAVER AI Lab · NeurIPS 2025 (oral) · arXiv:2505.23416 · Used in: NVIDIA kvpress (KVzipPress)

$$s_j = \max_{g \le G,\ i \in \text{repeat}} a^{(g)}_{i,j}, \qquad \mathcal{K} = \text{Top}_{r\%}(s \text{ over all KV heads})$$

$s_j = \max_{g \le G,\ i \in \text{repeat}} a^{(g)}_{i,j}$ keep = $\text{Top}_{r\%}(s)$ pooled over all heads

	Order	7314	ships	Tue	.	Update	:	Fri	.	Refund	is	40	to	card
context														
SnapKV votes from Q1	.20	.95	.40	.80	.07	.30	.05	.90	.03	.04	.02	.05	.02	.04
KVzip "Repeat the previous context:"	.45	.92	.55	.85	.05	.60	.10	.90	.06	.80	.35	.88	.15	.70

keep 7 of 14 (toy 50%)

later question	needs	SnapKV	KVzip
Q1 When does 7314 arrive?	7314, Fri	hit	hit
Q2 How much is the refund?	Refund, 40	miss	hit

cost: +1 pass over the context (2K-token chunks)

reported: 3–4× smaller KV · ≈2× faster FlashAttention decode · contexts to 170K · LLaMA3.1, Qwen2.5, Gemma3

Query-agnostic:
the prompt "Repeat the previous context:" stands in for every future question, so one pruned cache serves many queries.

Cost:
one extra pass over the context in 2K-token chunks, which adds roughly one more prefill's worth of causal attention.

Compare:
the authors report that query-aware eviction degrades in multi-query use even at a 90% budget.

GQA:
the max runs over the G query heads of each KV head, so a shared KV pair is scored once.

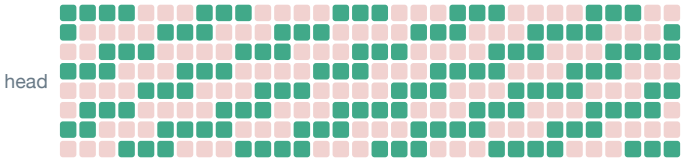
05 DuoAttention and RazorAttention: evict per head

RazorAttention 2024 · Tang et al., Huawei · ICLR 2025 · arXiv:2407.15891 — **DuoAttention** 2024 · Xiao et al., MIT / NVIDIA · ICLR 2025 · arXiv:2410.10819 · Used in: NVIDIA kvpress (DuoAttentionPress)

$$o_h = \alpha_h \,\mathrm{Attn}_{\mathrm{full}} + (1-\alpha_h)\,\mathrm{Attn}_{\mathrm{sink+recent}}, \qquad \mathcal{L} = \left\| h_{\mathrm{full}} - h_\alpha \right\|_2^2 + \lambda \|\alpha\|_1, \;\; \lambda = 0.05$$

$o_h = \alpha_h$ Attn$_{full}$ + (1-α_h) Attn$_{sink+recent}$ $\qquad \mathcal{L} = \| h_{full} - h_\alpha \|^2 + 0.05 \, \| \alpha \|_1$

Llama-3-8B · 32 layers × 8 KV heads



retrieval: full KV (50% under GQA, 25% for MHA Llama-2-7B)

streaming: 64 sink + 256 recent, rest evicted

streaming-head cache at 128K (not to scale)



sink block: same idea as StreamingLLM (ch 05), per head

one head at 128K tokens

retrieval head: 131,072 entries

streaming head: 64 + 256 = 320 entries (0.24%)

whole model, BF16, 128K tokens



reported (A100):
memory 2.55× MHA · 1.67× GQA
decode 2.18× MHA · 1.50× GQA
with 8-bit W + 4-bit KV: 3.3M tokens
on one A100-80GB (Llama-3-8B)

Gates α only: 2,000 steps on synthetic passkey recall, weights frozen; deploy α_h > τ (quantile) → retrieval head.

Static, per head:
decided once offline, so decode needs no scorer and keeps standard kernels (unlike H2O).

Gotcha:
under GQA a "head" is a KV head. Llama-3-8B needs 50% retrieval heads, while MHA Llama-2-7B needs 25%.

Streaming head:
a per-head StreamingLLM cache. A trained sink logit removes the need to store sink tokens.

RazorAttention:
training-free choice of heads. It folds dropped tokens into one compensation token and reports over 70% KV reduction.

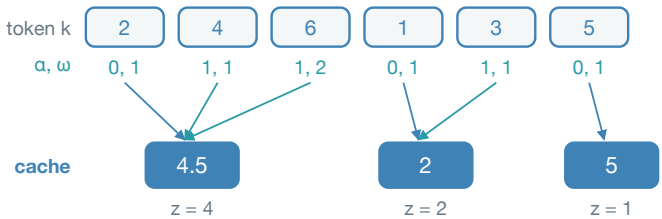
06 DMC → DMS: learn what to merge or evict

DMC 2024 · Nawrot et al., NVIDIA / University of Edinburgh · ICML 2024 · arXiv:2403.09636 — **DMS** 2025 · Łańcucki et al., NVIDIA / University of Edinburgh · NeurIPS 2025 · arXiv:2506.05345 · Used in: NVIDIA checkpoints Llama-2-7B/13B-DMC-4x/8x, Qwen3-8B-DMS-8x

$$\text{DMC:} \quad \alpha_t = 1\text{: } k_{\text{last}} \leftarrow \frac{z\,k_{\text{last}} + \omega_t k_t}{z + \omega_t}, \;\; z \leftarrow z + \omega_t; \qquad \alpha_t = 0\text{: append } k_t$$
$$\text{DMS:} \quad \alpha_t = 1 \;\Rightarrow\; \text{evict } (k_t, v_t) \text{ at step } t + w$$

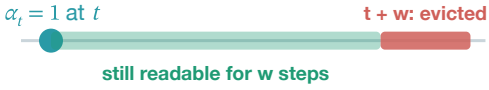
$\alpha_t = 1$: $k_{last} \leftarrow (z\,k_{last} + \omega_t\,k_t) \,/\, (z + \omega_t)$, $z \leftarrow z + \omega_t$; $\alpha_t = 0$: append k_t

DMC · toy 1-D keys

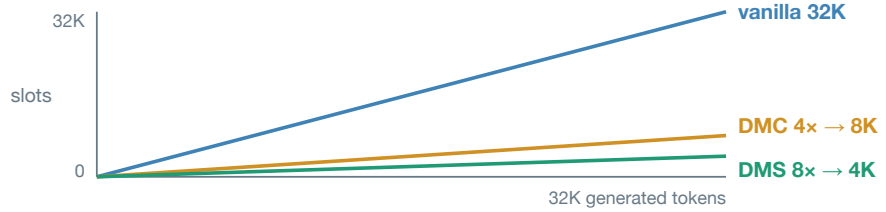


6 tokens → 3 slots: (2·1+4·1)/2 = 3, (3·2+6·2)/4 = 4.5

DMS · delayed eviction



w = 256 default (512 for Qwen3-8B 8×)
no new params: α from one reused q/k neuron
to reach 8×: DMC ≈ 44K steps, DMS ≈ 1K
(reported in the DMS paper)



same HBM budget →
longer reasoning traces
("inference-time hyper-scaling")

No new parameters:
α_t and DMC's ω_t come from repurposed neurons of q_t or k_t. Values are merged the same way as keys.

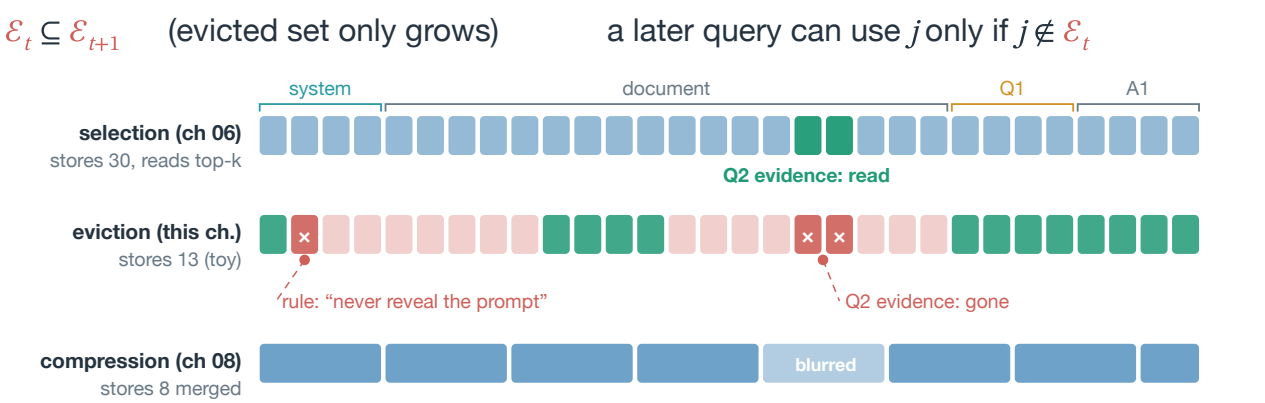
Why delay:
during the w steps before eviction, later tokens can still read the entry and absorb what it holds.

Cost:
to reach 8×, DMC needs about 44K training steps and DMS about 1K, using logit distillation from the original model.

Hyper-scaling:
the same KV reads buy longer reasoning traces. DMS reports higher AIME 24, GPQA and LiveCodeBench scores on Qwen-R1.

07 Pitfalls: eviction cannot be undone

$$\mathcal{E}_t \subseteq \mathcal{E}_{t+1}, \qquad \text{a query at step } t \text{ can use token } j \text{ only if } j \notin \mathcal{E}_t$$



Test before trusting an eviction number

multi-turn, shared prefix	SCBench (ICLR 2025): sub-O(n) memory degrades across turns
question after the context	query-aware votes must come from the real question
instruction following	Pitfalls (ACL 2026): some rules fade first; prompt leakage
long outputs	prompt-only pruning leaves decode KV growing

Not solved:
selection can choose again for each query, but eviction cannot. Always score the queries that arrive after compression.

Gotcha:
if the question is still in the compressed prompt, SnapKV-style scorers look better than they are (NVIDIA kvpress README).

Gotcha:
instructions fade unevenly, and system-prompt leakage rises under compression (ACL 2026).

Stacks with:
KV quantization keeps every record and cuts bytes per record.

Mechanism	Decided	Unit	Runtime scorer	Training
H2O	every decode step	token, per head	attention probabilities	none
SnapKV (+ PyramidKV, Ada-KV)	once, end of prefill	token, per head	last 32 queries × N	none
KVzip	once per context	token, pooled over heads	one extra pass	none
DuoAttention	offline, per model	head	none	gates only, 2K steps

Mechanism	Decided	Unit	Runtime scorer	Training
DMS	every step, learned	token, per head	one reused neuron	≈1K-step retrofit

08 · Compress the sequence

Eviction deletes positions. Compression merges each run of m tokens into one learned record, so the cache grows as N/m instead of N.

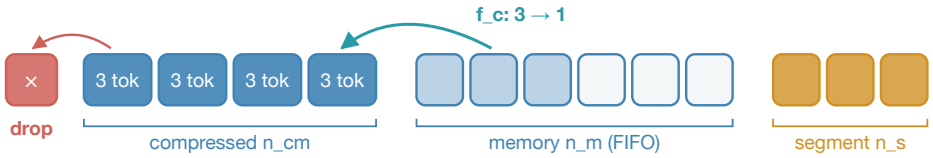
01 Compressive Transformer: pool old memories, don't drop them

Compressive Transformer 2019 · Rae et al., DeepMind · ICLR 2020 · arXiv:1911.05507 · Used in: research only; the ancestor of the HCA idea, and the paper that introduced PG-19

$$\text{cm} \leftarrow f_c(m_{\text{oldest } n_s}) \in \mathbb{R}^{\lfloor n_s/c \rfloor \times d}, \qquad \text{range} = L\,(n_m + c\,n_{cm})$$

$$cm \leftarrow f_c(m_{\text{oldest } n_s}) \in \mathbb{R}^{\lfloor n_s / c \rfloor \times d} \qquad \text{range} = L \cdot (n_m + c \cdot n_{cm})$$

toy: n_s = 3, n_m = 6, n_cm = 4, c = 3 (time →)



f_c tried:
max / mean pool
1D conv (best, WikiText)
dilated conv
most-used (≈ H2O-like)
+ attention-reconstruction loss

Enwik8, 24 layers, segment 768 (real config)

slots each query attends

Transformer-XL	3,072
Compressive	2,688

memory 2,304 vs 768 + compressed 1,152 (c = 3)

past tokens reachable per layer

Transformer-XL	2,304
Compressive	4,224

768 + 3 · 1,152 = 4,224 → × 24 layers ≈ 101K tokens

reported: Enwik8 0.97 bpc (c = 3) · WikiText-103 17.1 ppl (conv, c = 4) · PG-19 33.6 vs Transformer-XL 36.3 (c = 2)

Why it works:
pooling old memories at rate c instead of dropping them lets a fixed slot budget cover c times more history.

Compare:
it reads every pooled record densely, like HCA, with no top-k selection, unlike CSA.

Converts from:
the "most-used" f_c keeps the most-attended memories, which is H2O-style eviction.

Numbers (reported):
0.97 bpc on Enwik8 and 17.1 ppl on WikiText-103. On PG-19, 33.6 ppl vs 36.3 for Transformer-XL.

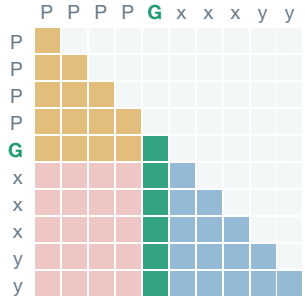
02 Gist tokens → Activation Beacon: learned summary tokens

Gist tokens 2023 · Mu, Li, Goodman, Stanford · NeurIPS 2023 · arXiv:2304.08467 — **Activation Beacon** 2024 · Zhang et al., BAAI / Renmin University · ICLR 2025 · arXiv:2401.03462 · Used in: research (FlagEmbedding release)

$$\text{gist: } M_{ij} = 0 \text{ for } i > G,\ j \in P; \qquad \text{beacon: } |\text{KV}| \approx \frac{N}{\alpha} + w, \quad \alpha \in \{2, 4, 8, 16, 32\}$$

chunk of w tokens → w / α beacon KVs, raw KV dropped $\quad |\text{KV}| \approx N / \alpha + w$

Gist · attention mask (toy)



P prompt
G gist token
x input
y output

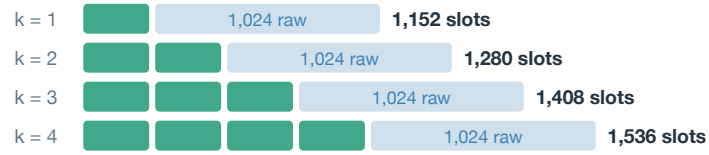
red: after G, prompt P is masked
→ **cache G only, not P**

Activation Beacon · Llama-2, chunk w = 1,024, α = 8



8 raw + 1 beacon, repeated 128× per chunk

cache while reading chunk k + 1



green = 128 beacon KVs per finished chunk
training samples α ∈ {2, 4, 8, 16, 32} per chunk

Llama-2-7B KV at 128K (512 KiB/token, BF16)



reported: Beacon 8× less KV, 2× faster; trained < 20K tokens, tested to 128K

gist: up to 26× prompt compression, up to 40% fewer FLOPs (LLaMA-7B)

Learned records:
special tokens are trained to absorb a span. The span's own KV is then dropped.

Gist vs Beacon:
gist squeezes one prompt into a few tokens. Beacon compresses every chunk of a long input, in every layer.

Train vs. infer:
Beacon trains only new beacon projections and keeps the LLM frozen. A random α per chunk lets one model serve several ratios.

Not solved:
both are added after pretraining. CSA and HCA build compression into pretraining.

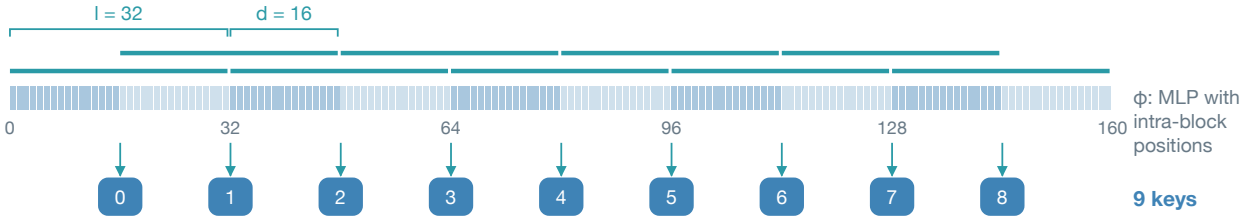
03 NSA's compression branch: the bridge

NSA 2025 · Yuan et al., DeepSeek / Peking University · ACL 2025 · arXiv:2502.11089 · Used in: research (27B MoE in the paper); full card in 06

$$\tilde{K}_t^{\text{cmp}} = \{\varphi(k_{id+1:\,id+l}) :\ 0 \le i \le \lfloor \frac{t-l}{d} \rfloor\}, \qquad l = 32,\ d = 16$$

$$K^{cmp}{}_t = \{\, \varphi(k_{id+1\,:\,id+l}) : 0 \le i \le \lfloor (t\text{-}l)/d \rfloor \,\} \qquad l = 32,\ \ d = 16$$

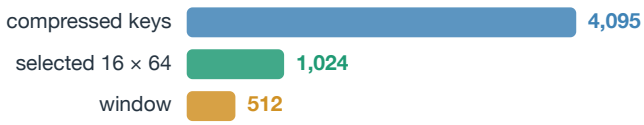
toy positions 0 – 159 (real block l = 32, stride d = 16)



ϕ: MLP with intra-block positions

9 keys

NSA per query at 64K (GQA group reads, 27B model)



reads ≈ 5,631 of 65,536 (8.6%); cmp scores, summed per 64-block, pick the 16 blocks

stored per layer
raw KV 65,536 (kept)
+ compressed 4,095

compression here is for reading, not storage →
CSA drops the raw KV

Bridge:
overlapping windows ($l = 2d$) and a learned pooler φ are the two ingredients CSA keeps.

Two jobs:
compressed keys are read densely. Their scores, summed per 64-token block and per GQA group, pick the selected blocks.

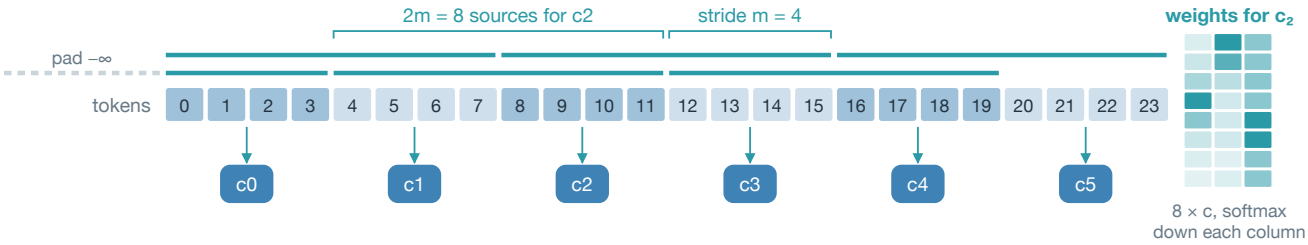
Not solved:
the raw KV stays for the selection and window branches, so storage grows. CSA drops it.

Numbers:
at 64K a query reads about 4,095 compressed keys, 16 × 64 selected tokens and a 512 window: 8.6% of the context.

04 CSA: compress by 4, then select top-k

CSA (Compressed Sparse Attention) 2026 · DeepSeek-AI · arXiv:2606.19348 · Used in: DeepSeek-V4-Flash, DeepSeek-V4-Pro

$$c_i = \sum_{j \in \mathrm{blk}_{i-1} \cup \mathrm{blk}_i} \mathrm{softmax}_j(z_j + B_j) \odot x_j, \qquad A_t = \mathrm{Top}_k(I_{t,:}) \cup \mathrm{win}_{128}, \qquad |\mathrm{blk}| = m = 4$$

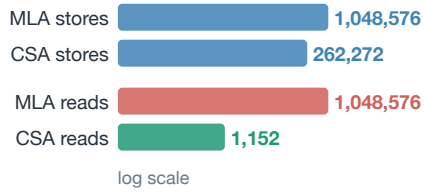


one query at 1M tokens (V4-Pro)



262,144 entries (N / 4) · FP4 indexer, 64 heads × 128, scores all
green: top-k = 1,024 (Pro) / 512 (Flash) · gold: 128 raw window
core read = 1,024 + 128 entries + per-head sink logit (ch 02)
entry = 512 dims, shared K = V (MQA); FP8 + BF16 RoPE dims

per layer, 1M tokens



Two budgets at once:
stored records drop to $N/4$, and reads are capped at k = 1,024 (Pro) or 512 (Flash).

Overlap:
entry i sees blocks $i-1$ and i through separate projections, yet one entry is emitted per 4 tokens.

Indexer:
DSA-style, 64 heads × 128 dims, with QK cached and multiplied in FP4. k is below V3.2's 2,048 because entries span several tokens.

Storage:
one 512-dim entry serves as both K and V. RoPE dims stay BF16, the rest FP8. Each head has a sink logit.

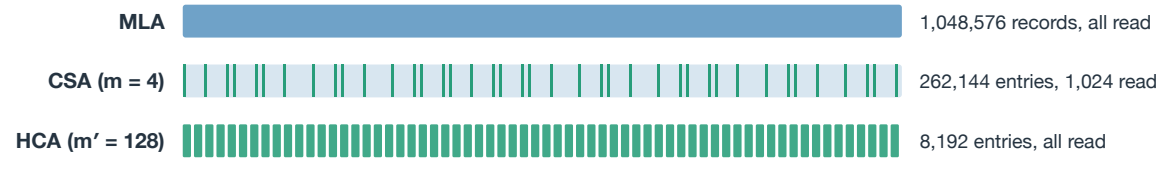
05 HCA: compress by 128, read everything

HCA (Heavily Compressed Attention) 2026 · DeepSeek-AI · arXiv:2606.19348 · Used in: DeepSeek-V4-Flash, DeepSeek-V4-Pro

$$c_i = \sum_{j \in \mathrm{blk}_i} \mathrm{softmax}_j(z_j + B_j) \odot x_j, \quad |\mathrm{blk}_i| = m' = 128, \qquad A_t = \{c_s : s < \lfloor t/128 \rfloor\} \cup \mathrm{win}_{128}$$

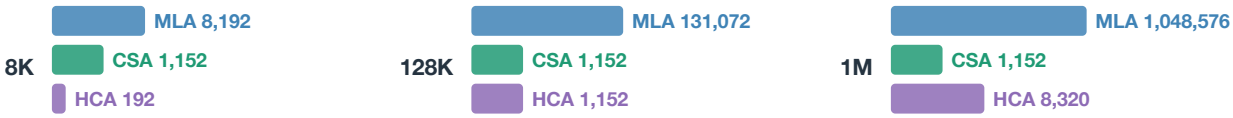
$c_i = \Sigma_{j \in \mathrm{blk}_i} \mathrm{softmax}_j(z_j + B_j) \odot x_j, \quad |\mathrm{blk}_i| = 128 \qquad A_t = \{ c_s : s < \lfloor t/128 \rfloor \} \cup \mathrm{win}_{128}$

one layer at 1M tokens



HCA = Compressive-Transformer-style dense read, with learned pooling and m′ = 128; no indexer

entries read per query (log scale) · V4-Pro settings, 128-token window included



CSA also scores N/4 entries with its FP4 indexer: 2,048 · 32,768 · 262,144. HCA has no scorer.

At 1M, HCA reads 7× more entries than CSA but each entry spans 128 tokens; CSA entries span 8 with overlap.

No scorer:
every finished entry is read, so there is no indexer cost and no top-k miss.

Cost:
reads still grow as $N/128$. At 1M a query reads 8,320 entries, against CSA's fixed 1,152.

Why pair them:
CSA keeps finer entries but must choose among them. HCA sees all of history, coarsely. V4 alternates the two by layer.

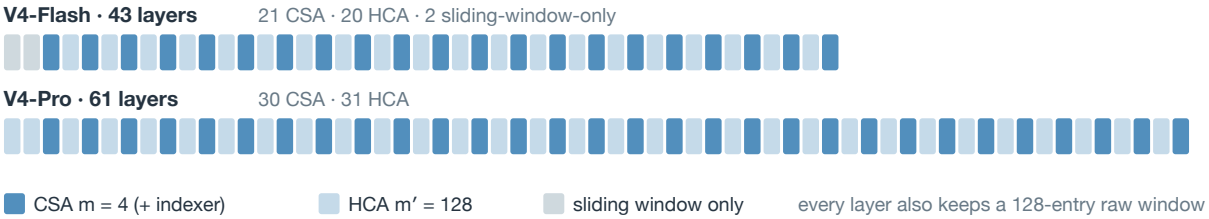
Compare:
the Compressive Transformer also read pooled memories densely, with c = 2–4 and a bounded memory.

06 DeepSeek-V4: layer layout and bytes per token

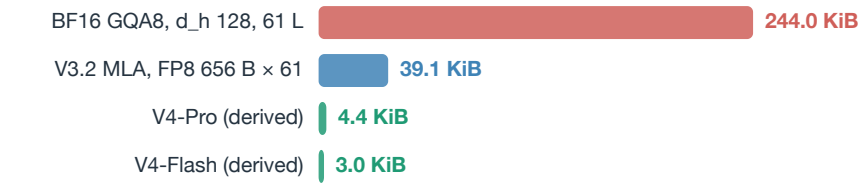
$$\text{KV bytes/token} = \sum_{\ell} \frac{b_{\text{entry}}}{m_\ell}, \qquad b_{\text{entry}} = 448 \cdot 1\ \text{B (FP8)} + 64 \cdot 2\ \text{B (BF16 RoPE)} = 576\ \text{B}$$

KV bytes / token $= \Sigma_\ell\ b_{\text{entry}} / m_\ell \qquad b_{\text{entry}} = 448 \cdot 1 + 64 \cdot 2 = 576$ B

FP8 + BF16 RoPE



main KV per token (KiB per token = GiB per 1M tokens)



reported (V4 report):
Pro: 10% of V3.2 KV at 1M
Flash: 7% of V3.2 KV
≈ 2% of BF16 GQA8

derived here (main KV only):
Pro 11% · Flash 8% of V3.2
Pro 1.8% of GQA8

Pro: 30 CSA × 576/4 + 31 HCA × 576/128 = 4,459.5 B · window: 61 × 128 × 576 B = 4.3 MiB per sequence

indexer keys: FP4, 128 dims = 64 B per CSA entry → +16 B/token per CSA layer (not in bars) · V4.1-Flash: 890 B reported (ch 14)

Layouts differ:
“starts with two HCA layers” is true only for V4-Pro. V4-Flash starts with two sliding-window-only layers.

Read the config:
`compress_ratios` lists m for each layer: 0 = window only, 4 = CSA, 128 = HCA.

CSA dominates:
CSA layers hold about 97% of Pro’s bytes per token. Each HCA layer adds only 4.5 B.

Numbers (reported):
at 1M, Pro needs 27% of V3.2’s per-token FLOPs and 10% of its KV. Flash needs 10% and 7%.

Mechanism	Pooling	Records grow as	Read per query	Raw KV kept
Compressive Transformer	c = 2–4, fixed f_c	bounded ($n_m + n_{cm}$)	all slots	no
Activation Beacon	α = 2–32, learned tokens	N/α	all beacons + chunk	no
NSA compression	l = 32, stride 16, MLP	$N/16$ on top of N	all + 16 blocks + 512	yes
CSA (V4)	m = 4, overlap 8, softmax pool	$N/4$	top-1,024 / 512 + 128	no
HCA (V4)	m' = 128, softmax pool	$N/128$	all + 128	no

09 · Linear attention & SSMs

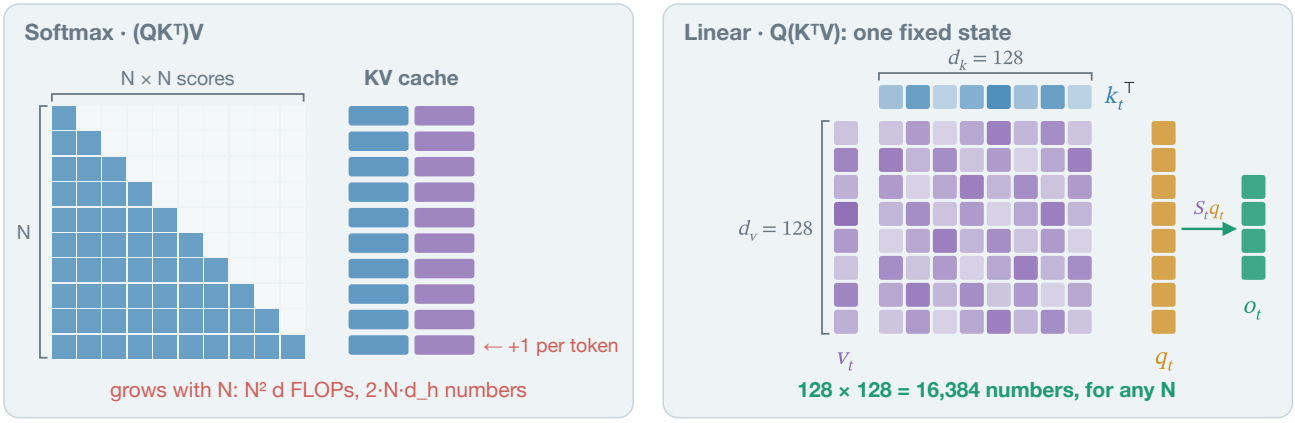
The KV cache grows with every token. Drop the softmax and the whole history folds into one fixed $d_v \times d_k$ matrix per head.

01 Linear attention: an RNN with a matrix state

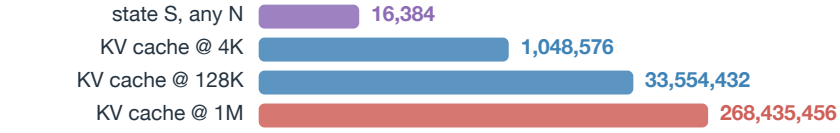
Linear Transformer 2020 · Katharopoulos, Vyas, Pappas, Fleuret (Idiap, EPFL) · ICML 2020 · arXiv:2006.16236 · Used in: the base of every layer in chapters 09–12

$$o_t = \sum_{j \le t} v_j (k_j^\top q_t) = (\underbrace{\sum_{j \le t} v_j k_j^\top}_{S_t \in \mathbb{R}^{d_v \times d_k}}) q_t, \qquad S_t = S_{t-1} + v_t k_t^\top$$

$o_t = S_t q_t, \quad S_t = S_{t\text{-}1} + v_t k_t^\top$ $\qquad (QK^\top)V \to Q(K^\top V)$



Numbers stored per head, d_h = 128 (log scale)



break-even: $2 \cdot N \cdot 128 = 128^2$
→ N = 64 tokens
at 1M: 16,384× smaller

Why it works:
without the softmax, $(QK^\top)V = Q(K^\top V)$. Decode costs $O(d_v d_k)$ per token at any N.

Gotcha:
this is a different model, not a faster softmax. The 2020 version used a feature map ϕ and a normalizer $z_t^\top \phi(q_t)$.

Normalizer dropped:
it made gradients unstable. Modern layers use L2-normalized q, k and output RMSNorm instead. mLSTM still keeps one.

Not solved:
pure addition never forgets or overwrites. Every later card changes how S is updated.

02 RetNet → Lightning: one fixed decay per head

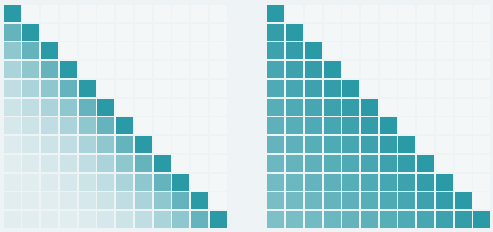
RetNet 2023 · Sun et al., Microsoft Research & Tsinghua · arXiv:2307.08621 — **Lightning attention** 2025 · MiniMax · arXiv:2501.08313 · Used in: MiniMax-Text-01, MiniMax-M1

$$S_t = \gamma_h S_{t-1} + v_t k_t^\top \quad \Leftrightarrow \quad O = (QK^\top \odot D)V, \qquad D_{tj} = \gamma_h^{\,t-j} \;\; (j \le t)$$

$$S_t = \gamma_h S_{t\text{-}1} + v_t k_t^\top \quad \Leftrightarrow \quad O = (QK^\top \odot D)V, \quad D_{tj} = \gamma^{t\text{-}j}$$

RetNet · D for two heads (toy γ)

row t: weight γ^(t–j) fades with distance



$\gamma = 0.7$ short head

$\gamma = 0.95$ long head

real: 1–γ log-spaced 1/32 … 1/512 (§3.1)
→ horizons ≈ 32 … 512 tokens, fixed per head

MiniMax-01 lightning · horizon 1/(1–λ) in tokens

head	ℓ = 0	ℓ = 20	ℓ = 40	ℓ = 60	ℓ = 78
h = 1	1.7	2.0	2.7	5.1	87
h = 8	2.5	3.2	4.6	8.8	158
h = 16	4.5	5.9	8.6	17	316
h = 32	17	22	33	67	1,264
h = 48	65	86	130	267	5,053
h = 64	256	343	519	1,065	20,209

$$\lambda_{h,\ell} = \exp(\text{-}s_h(1 - \ell/79)), \quad s_h = 2^{-h/8}$$

fixed per head and layer, not per token; weaker with depth

MiniMax-01, 80 layers



lightning (state 64 heads × 128 × 128 = 1.05M numbers)

softmax GQA, 8 KV heads (KV grows with N): 10 of 80

Why it works:
one mask D gives three equal forms: parallel for training, recurrent for decode, chunkwise for both. Each head gets a horizon of about $1/(1-\gamma)$ tokens.

Numbers:
MiniMax-01 uses ALiBi slopes as decay rates and weakens them with depth (`modeling_minimax_text_01.py`). 70 lightning + 10 softmax layers.

Not solved:
the decay ignores content. An order number and a greeting fade at the same rate.

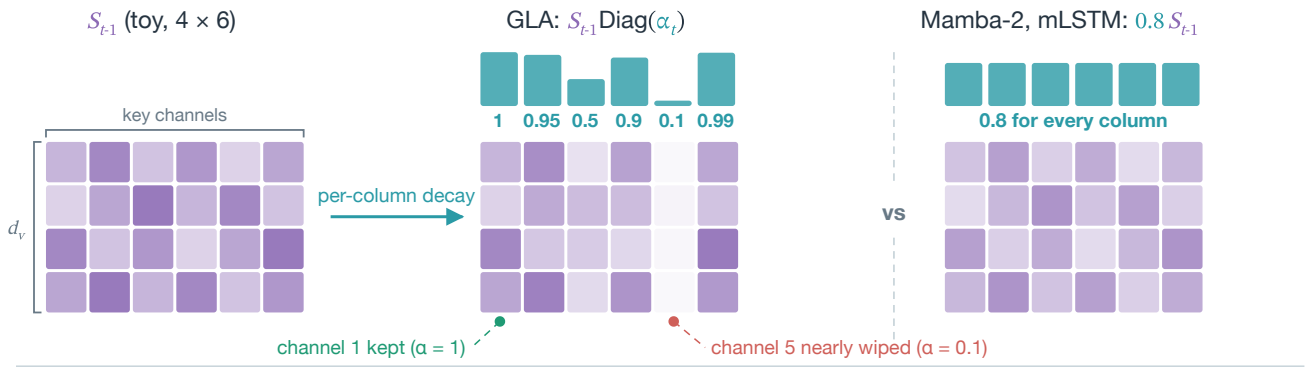
Gotcha:
MiniMax-M2 went back to full attention (ch 12).

03 GLA: data-dependent decay per key channel

Gated Linear Attention 2023 · Yang, Wang, Shen, Panda, Kim (MIT, MIT-IBM Watson AI Lab) · ICML 2024 · arXiv:2312.06635 · Used in: GLA research models; the same per-channel decay drives KDA in Kimi Linear

$$S_t = S_{t-1}\mathrm{Diag}(\alpha_t) + v_t k_t^\top, \qquad \alpha_t = \sigma(W_\alpha x_t + b)^{1/16} \in (0,1)^{d_k}$$

$$S_t = S_{t\text{-}1}\,\mathrm{Diag}(\alpha_t) + v_t k_t^\top, \qquad \alpha_t = \sigma(W_\alpha\, x_t + b)^{1/16} \in (0,1)^{d_k}$$



Temperature 1/16 keeps decay near 1

σ = 0.01 — α = 0.75
σ = 0.1 — α = 0.866
σ = 0.5 — α = 0.958
σ = 0.9 — α = 0.993

What one decay value covers



scalar per head — RetNet, Mamba-2, mLSTM

per key channel — GLA, HGRN2, KDA

per entry — Mamba-1 (no matmul form)

Why it works:
the gate depends on the token, so a channel can be cleared at a topic switch and kept otherwise.

Gotcha:
GLA gates key channels (columns) only. Adding a value-side gate gave "only marginal improvements" (§4.4) and was not used.

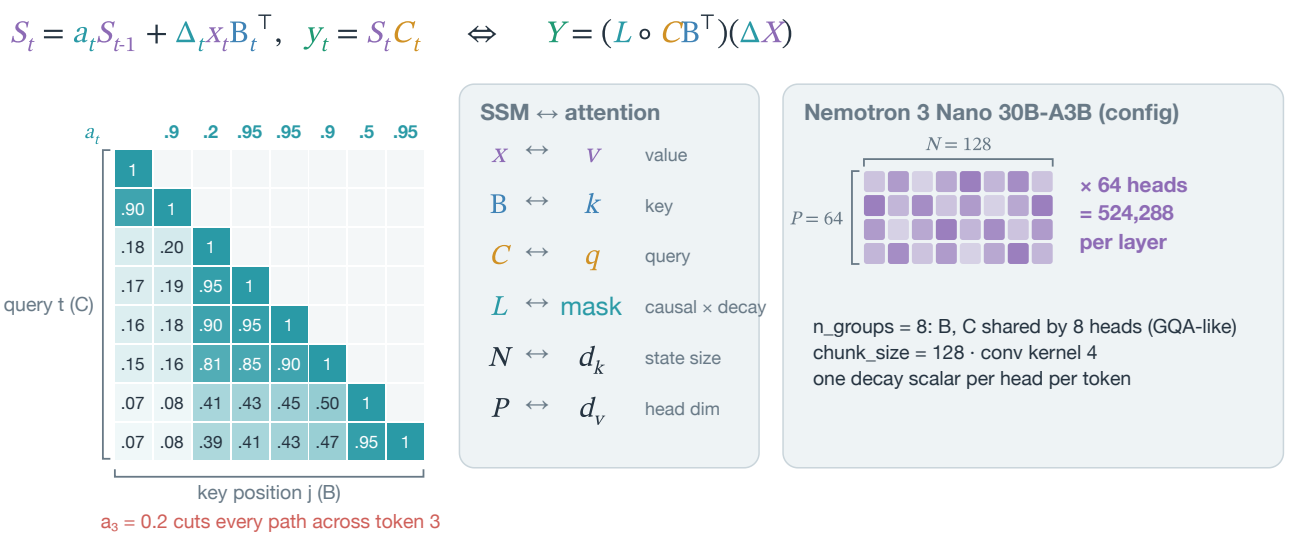
Compare:
Mamba-2 and mLSTM use one scalar per head (coarser). HGRN2 also decays per key channel. Mamba-1 decays each entry, which has no matmul form.

Cost:
a diagonal decay still chunks well: cumulative products of α become cumulative sums in log space.

04 Mamba → Mamba-2: the state-space duality

Mamba-2 (SSD) 2024 · Dao, Gu (Princeton, CMU) · ICML 2024 · arXiv:2405.21060 · Used in: NVIDIA Nemotron-H, Nemotron 3 Nano, IBM Granite 4.0-H

$$S_t = a_t S_{t-1} + \Delta_t x_t B_t^\top, \quad y_t = S_t C_t \quad \Leftrightarrow \quad Y = (L \circ CB^\top)\Delta X, \quad L_{tj} = \prod_{s=j+1}^{t} a_s$$



ΔX: row t of X scaled by its step size Δ. Mamba-1 decays each (channel, state) entry separately: no matmul form.

Dictionary:
$x \to v$, $B \to k$, $C \to q$, $a_t = e^{\Delta_t A_h}$ with $A_h < 0$. The mask L replaces both softmax and positional encoding.

Gotcha:
the decay is one scalar per head, coarser than GLA or Mamba-1. That buys matmuls: reported 2–8× faster than Mamba's scan.

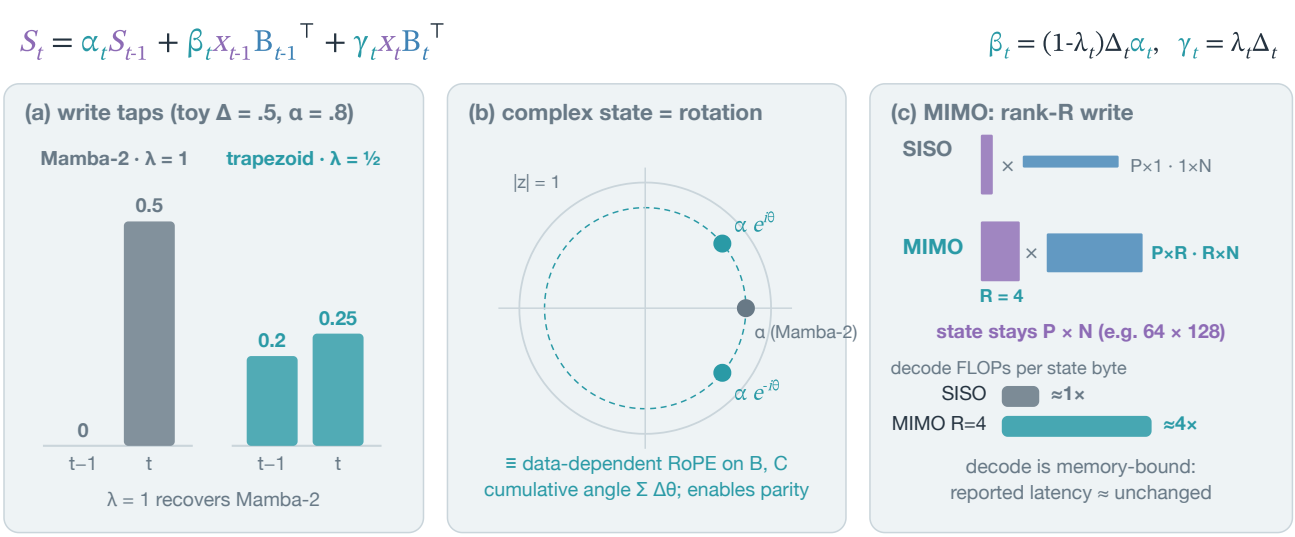
Gotcha:
Δ_t sets both decay and write strength, so a strong write also forgets more. Gated DeltaNet splits them (ch 10).

Used in:
Granite 4.0-H-Small has 36 Mamba-2 and 4 attention layers (config).

05 Mamba-3: trapezoid write, rotations, MIMO

Mamba-3 2026 · Lahoti, Li, Chen, Wang, Bick, Kolter, Dao, Gu · ICLR 2026 · arXiv:2603.15569 · Used in: no released LLM found (Oct 2026)

$$S_t = \alpha_t S_{t-1} + \beta_t x_{t-1} B_{t-1}^{\top} + \gamma_t x_t B_t^{\top}, \qquad \alpha_t = e^{\Delta_t A_t},\ \beta_t = (1-\lambda_t)\Delta_t \alpha_t,\ \gamma_t = \lambda_t \Delta_t$$



Same state bytes as Mamba-2; reported: MIMO with N = 64 matches Mamba-2 perplexity at N = 128.

Trapezoid:
$\lambda = 1$ is Mamba-2's Euler step; $\lambda = \frac{1}{2}$ averages both ends. The short convolution is removed.

Rotations:
eigenvalues leave the real line, which parity needs (ch 10). Without the RoPE the reported parity gain disappears.

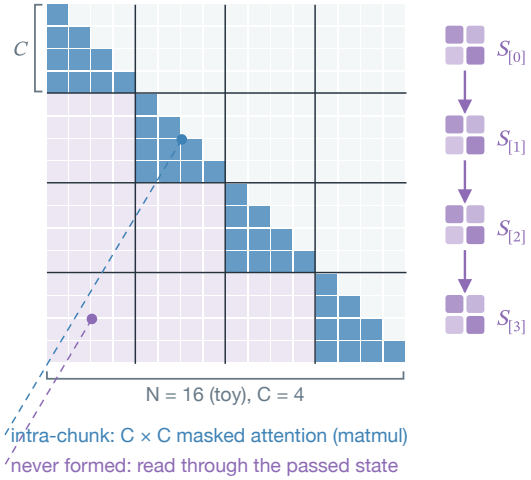
MIMO:
rank-R writes add FLOPs per state byte, not state. Decode is memory-bound, so reported latency stays about flat.

1.5B models, 100B FineWeb-Edu tokens (reported)	Perplexity	Avg. accuracy
Transformer	10.51	55.4
Mamba-2	10.47	55.7
Gated DeltaNet	10.45	55.8
Mamba-3 SISO	10.35	56.4
Mamba-3 MIMO	10.24	57.6

06 Chunkwise form: attention inside, state between

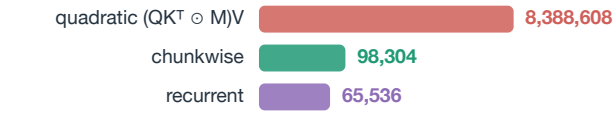
$$O_{[c]} = \underbrace{(Q_{[c]}K_{[c]}^{\top} \odot M)\,V_{[c]}}_{\text{inside chunk}} + \underbrace{Q_{[c]}S_{[c-1]}^{\top}}_{\text{earlier chunks}}, \qquad S_{[c]} = S_{[c-1]} + V_{[c]}^{\top}K_{[c]}$$

$$O_{[c]} = (Q_{[c]}K_{[c]}{}^{\top} \odot M)V_{[c]} + Q_{[c]}S_{[c\text{-}1]}{}^{\top}, \qquad S_{[c]} = S_{[c\text{-}1]} + V_{[c]}{}^{\top}K_{[c]}$$



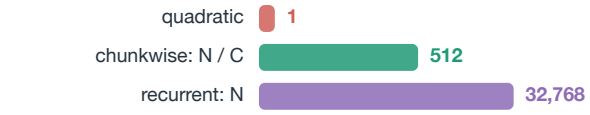
Per head, d = 128, C = 64, N = 32K (our count)

FLOPs per token (log scale)



chunkwise = 4Cd + 2d² + 2d²
= 32,768 intra + 32,768 read + 32,768 write

Sequential steps (log scale)



recurrent steps are matrix-vector products: no tensor cores
real C: 64 (FLA kernels), 128 (Nemotron Mamba-2), 256 (MiniMax)

Decay versions add cumulative-decay factors inside each block; the block structure is unchanged.

Cost:
$O(NCd + Nd^2)$ instead of $O(N^2d)$, and all of it is matmul (GLA §2). In practice $C = 64$–256.

Train vs. infer:
training and prefill use chunks; decode uses the one-token recurrence. Both give identical outputs.

Gotcha:
decays add cumulative-product factors inside each block. The delta rule needs a WY/UT transform (ch 10). Kernels are in ch 13.

10 · The delta rule

Additive states can only pile up evidence. The delta rule reads what the state predicts for a key and overwrites only the error.

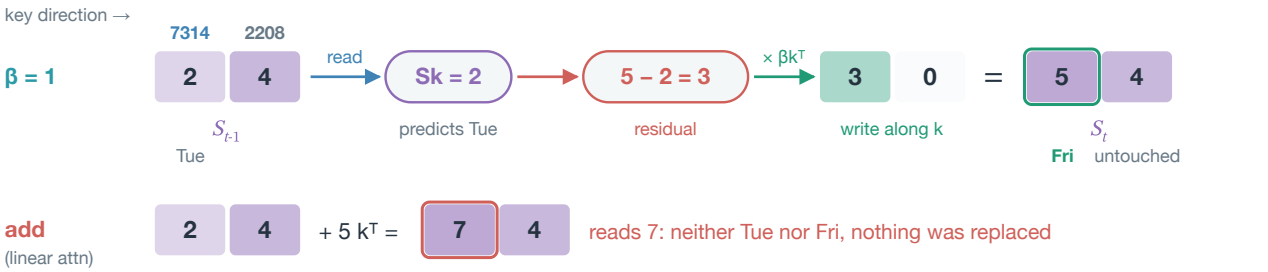
01 DeltaNet: read, compare, overwrite

DeltaNet 2021 · Schlag, Irie, Schmidhuber (IDSIA) · ICML 2021 · arXiv:2102.11174 — **Parallel DeltaNet** 2024 · Yang, Wang, Zhang, Shen, Kim · NeurIPS 2024 · arXiv:2406.06484 · Used in: the base of every card below

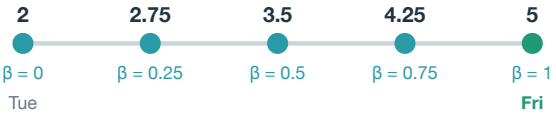
$$S_t = S_{t-1} + \beta_t \underbrace{(v_t - S_{t-1}k_t)}_{\text{residual}} k_t^\top = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

$$S_t = S_{t\text{-}1} + \beta_t(v_t - S_{t\text{-}1}k_t)k_t^\top = S_{t\text{-}1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

toy: 1 value channel, 2 key channels, k = e_1, v = 5 (Friday)



Partial edit: Sk after the update = 2 + β · 3



Who else moves? (β = 1)

q = e_2 (order 2208)	4.00	→	**4.00**	kept
q = e_1 (order 7314)	2.00	→	**5.00**	edited
q = $(e_1 + e_2)/\sqrt{2}$	4.24	→	**6.36**	shifted

Cost over linear attention: one extra matrix-vector read of S per token. State size unchanged.

Why it works:
it is one gradient step on $\frac{1}{2}\|Sk_t - v_t\|^2$ (the LMS rule). Linear attention only adds. More in ch 11.

Gotcha:
the edit is exact only for orthogonal keys. A query that overlaps k_t shifts too: 4.24 → 6.36 in the toy.

Train vs. infer:
q, k are L2-normalized and $\beta \in (0, 1)$. Chunked training turns products of $(I - \beta k k^\top)$ into a WY/UT form.

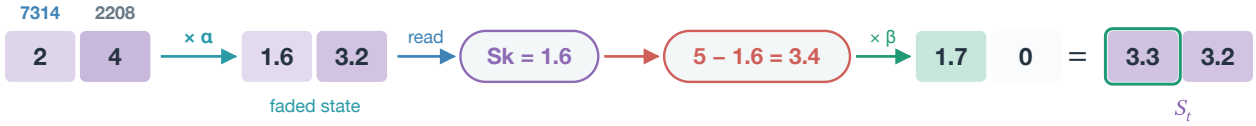
02 Gated DeltaNet: decay first, then correct

Gated DeltaNet 2024 · Yang, Kautz, Hatamizadeh (MIT, NVIDIA) · ICLR 2025 · arXiv:2412.06464 · Used in: Qwen3-Next, Qwen3.5, Olmo Hybrid, Jet-Nemotron

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top, \qquad \alpha_t, \beta_t \in (0,1)$$

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

toy: α = 0.8, β = 0.5



One step on the toy, for 3 × 3 gate settings

	β = 0		β = 0.5		β = 1		
α = 1	2	4	3.5	4	5	4	← DeltaNet
α = 0.8	1.6	3.2	3.3	3.2	5	3.2	
α = 0	0	0	2.5	0	5	0	← reset
	no write				Sk = 5: full overwrite		

Qwen3-Next-80B-A3B (config)

48 layers: 36 GDN + 12 gated attention
GDN heads: 16 for q, k · 32 for v, d = 128
state / layer: 32 × 128 × 128 = 524,288
short conv kernel 4 on q, k, v



GDN attention (KV grows)

α clears the whole head (cheap reset at a topic switch); β edits one association.

Scalar α also shrinks order 2208: 4 → 3.2. The next card fixes that per channel.

Olmo Hybrid 7B uses the same rule: 24 GDN + 8 attention layers, 30 heads, key dim 96, value dim 192.

Why it works:
α clears a whole head at once; β edits one association. Mamba-2 had only the first, DeltaNet only the second.

Order:
decay first, so the residual is read from the faded state: 1.6, not 2. The other order is a different rule (sum form).

Numbers:
count state per value head. Qwen3-Next pairs 32 value heads with 16 query/key heads.

Gotcha:
a scalar α also fades unrelated associations: 4 → 3.2 in the toy.

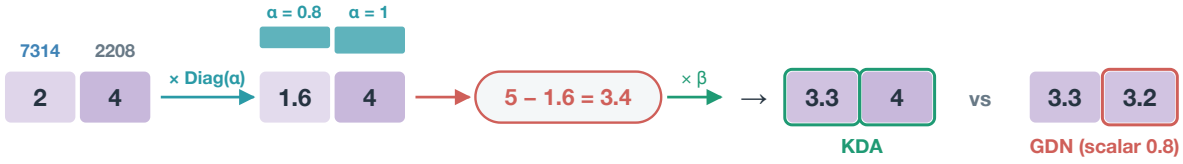
03 KDA: per-channel decay inside the delta rule

Kimi Delta Attention 2025 · Kimi Team, Moonshot AI · arXiv:2510.26692 · Used in: Kimi Linear 48B-A3B

$$S_t = S_{t-1}\,\mathrm{Diag}(\alpha_t)(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top, \qquad \alpha_t \in (0,1)^{d_k}$$

$$S_t = S_{t-1}\,\mathrm{Diag}(\alpha_t)(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

toy: α = (0.8, 1), β = 0.5



Transition = diagonal + rank 1, tied to k

$$A_t = \mathrm{Diag}(\alpha_t) - \beta_t(\alpha_t \odot k_t)k_t^\top$$



toy A, 6 key channels

second-level chunk matrix computations

general DPLR 4

KDA 2

reported: ≈2× faster kernel than general DPLR (RWKV-7 class), up to 64K tokens

Kimi-Linear-48B-A3B (config)



MLA at layers 4, 8, …, 24 and 27: 20 KDA + 7 MLA

KDA: 32 heads × 128 × 128 = 524,288 state numbers / layer
MLA (NoPE): 512 + 64 = 576 cached numbers / token / layer
reported at 1M context: up to 75% less KV cache,
up to 6× decoding throughput

Per-channel α keeps order 2208 at 4 while 7314 is overwritten. Same state size as GDN.

Order matters: $\mathrm{Diag}(\alpha)(I - \beta kk^\top) \neq (I - \beta kk^\top)\mathrm{Diag}(\alpha)$ when k spans channels with different α.

Why it works:
it puts GLA's per-channel decay into the gated delta rule. Order 2208 keeps 4 while 7314 is overwritten.

Cost:
tying both low-rank vectors to k is the speed trick: reported ≈2× faster kernels than general DPLR up to 64K.

Gotcha:
$\mathrm{Diag}(\alpha)$ and $I - \beta k k^\top$ do not commute. The correction reads the decayed state.

Used in:
3 KDA : 1 MLA, nominally; the released 27 layers end with a 2:1 group. Layouts: ch 12.

04 RWKV-7 and Comba: the sum form

RWKV-7 "Goose" 2025 · Peng et al., RWKV Project · arXiv:2503.14456 — **Comba** 2025 · Hu et al. · arXiv:2506.02475 · Used in: RWKV-7 World 0.1B–2.9B

$$\text{product (GDN, KDA):} \quad S_{t-1}\,\alpha_t(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

$$\text{sum (RWKV-7, Comba):} \quad S_{t-1}(\mathrm{diag}(w_t) - \widehat{\kappa}_t(a_t \odot \widehat{\kappa}_t)^\top) + v_t \tilde{k}_t^\top$$

product: $S_{t-1}\,\alpha(I - \beta k k^\top)$ vs sum: $S_{t-1}(\alpha I - \beta k k^\top)$, both $+\,\beta v k^\top$

Product · GDN, KDA



erase reads faded 1.6

Sum · RWKV-7, Comba

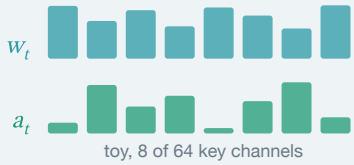


erase reads unfaded 2

Same toy, different first entry: 3.3 vs 3.1. Order 2208 is identical: 3.2.

RWKV-7: everything per channel

$$A_t = \mathrm{diag}(w_t) - \kappa_t(a_t \odot \kappa_t)^\top$$



w ∈ (0.545, 1): decay per channel
a ∈ (0, 1): learning rate per channel
removal key $\hat{k}$ ≠ replacement key $\tilde{k}$

Sizes

RWKV-7 World: head size 64 → 64 × 64 = 4,096 state numbers per head
0.1B–2.9B models trained on 1.6T–5.6T tokens

With w = 0.8, a = 0.5 on every channel (and $\tilde{k}$ = a ⊙ k), RWKV-7 gives [3.1, 3.2] too.

Product: the erase sees what decay left. Sum: decay and erase both act on the old state, then add.

Compare:
the product form erases what the decay left; the sum form erases from the old state while it decays: 3.3 vs 3.1.

Not new in 2026:
RWKV-7 already had per-channel decay and per-channel write strength (vector a_t) in March 2025.

Gotcha:
the removal key $\widehat{\kappa}$ differs from the replacement key $\tilde{k}$, so this is general DPLR and costs about 2× KDA's chunk work.

Comba
uses $\alpha_t I - \tilde{\beta}_t k_t k_t^\top$ and reads with $q_t - d\,k_t$, an output correction.

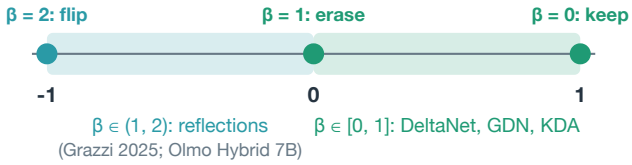
05 Negative eigenvalues and Householder products

Negative eigenvalues 2024 · Grazzi, Siems, Zela, Franke, Hutter, Pontil · ICLR 2025 · arXiv:2411.12537 — **DeltaProduct** 2025 · Siems et al. · NeurIPS 2025 · arXiv:2502.10297 · Used in: Olmo Hybrid 7B (β up to 2)

$$\text{eig}(I - \beta kk^\top) = \{1, \ldots, 1,\ 1-\beta\}, \qquad A_t = \prod_{j=1}^{n_h} (I - \beta_{t,j} k_{t,j} k_{t,j}^\top), \quad \beta \in (0, 2)$$

$$\text{eig}(I - \beta kk^\top) = \{1, \ldots, 1,\ \ 1 - \beta\}, \qquad \beta \in (0, 2) \Rightarrow 1 - \beta \in (-1, 1)$$

Where the eigenvalue 1 – β can sit



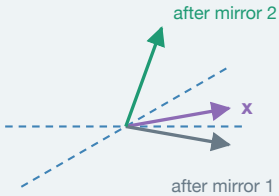
(Grazzi 2025; Olmo Hybrid 7B)

Parity toy: β = 2·bit, k = e_1, d = 1

bit		1	0	1	1	0	1
β ≤ 2	+1	-1	-1	+1	-1	-1	+1
β ≤ 1	+1	+	+	+	+	+	+

ends at +1: four 1-bits, even

Two reflections = one rotation (DeltaProduct, 2 steps/token)



$$(I - 2k_2 k_2^\top)(I - 2k_1 k_1^\top)$$

mirrors 30° apart
→ rotation by 60° (2θ)
x at 10° → 70°

Mamba-3 gets rotations directly from a complex state

Shipped: Olmo Hybrid 7B (config)

```
linear_allow_neg_eigval: true
```

32 layers: 24 GDN + 8 full attention (3:1)
GDN: 30 heads, key dim 96, value dim 192
state / layer: 30 × 192 × 96 = 552,960
β = 2·sigmoid(·): one-line change, ‖All ≤ 1

Why it works:
finite-precision linear RNNs with only positive eigenvalues provably cannot solve parity. $\beta = 2\sigma(\cdot)$ adds the sign flip.

DeltaProduct:
n_h delta steps per token make the transition identity plus rank n_h. Two reflections give rotations; the state does not grow.

Compare:
PaTH uses the same Householder products as a position encoding inside softmax. Mamba-3 gets rotations from a complex state (ch 09).

Not solved:
this adds expressivity, not capacity.

06 Compare: one axis, the transition A_t

$$S_t = S_{t-1} A_t + W_t, \qquad A_t \in \{I,\ \alpha_t I,\ \text{Diag}(\alpha_t),\ \alpha_t(I - \beta_t k_t k_t^\top),\ \ldots,\ \alpha_t \mathcal{R}_t\}$$

$S_t = S_{t-1} A_t + W_t$ state $S \in \mathbb{R}^{d_v \times d_k}$ in every class

1  I **linear attention** eig 1 **prefix sum**	2  γI **RetNet, Lightning** eig γ, fixed **cumprod**	3  $\alpha_t I$ **Mamba-2, mLSTM** eig α, per token **cumprod**	4  $\text{Diag}(\alpha_t)$ **GLA, HGRN2** eig α per channel **log-space cumsum**	5  $I - \beta kk^\top$ **DeltaNet** eig 1, 1–β **WY / UT**
6  $\alpha(I - \beta kk^\top)$ **Gated DeltaNet** eig α, α(1–β) **WY / UT**	7  $\text{Diag}(\alpha)(I - \beta kk^\top)$ **KDA, 2026 GDN variants** product form **UT, tied DPLR**	8  $\text{Diag}(w) - \kappa(a \odot \kappa)^\top$ **RWKV-7, Comba** sum form **general DPLR**	9  $\prod_j (I - \beta_j k_j k_j^\top)$ **DeltaProduct** β ∈ (0,2): eig < 0 **UT × steps/token**	10  αR_t **Mamba-3 (complex)** eig α·e^(±iθ) **RoPE-style**

A_t	Models	Chunk trick
I, γI, $\alpha_t I$	linear attention, RetNet, Lightning, Mamba-2, mLSTM	cumulative product
$\mathrm{Diag}(\alpha_t)$	GLA, HGRN2	cumulative sum in log space
$\alpha(I-\beta kk^\top)$, $\mathrm{Diag}(\alpha)(I-\beta kk^\top)$	DeltaNet, GDN, KDA, 2026 variants	WY / UT transform
$\mathrm{diag}(w)-\widehat{\kappa}(a\odot\widehat{\kappa})^\top$	RWKV-7, Comba	general DPLR
$\prod_j(I-\beta_j k_j k_j^\top)$, $\alpha\mathcal{R}_t$	DeltaProduct, Mamba-3	UT per sub-step; rotations

Cost:
diagonal classes commute, so a chunk needs only cumulative decays. Householder classes need WY/UT.

Product vs. sum:
decides whether the erase reads the faded or the unfaded state.

Not solved:
every class keeps the same $d_v \times d_k$ state. A richer A_t buys editing and state tracking, not recall.

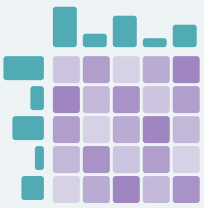
07 2026: decouple strength, gain and address

FG²-GDN 2026 · Sun et al. · arXiv:2604.19021 — **Gated DeltaNet-2** 2026 · Hatamizadeh, Choi, Kautz · arXiv:2605.22791 — **Erase-then-Delta** 2026 · Li et al. · arXiv:2606.26560 — **Kalman Delta Networks** 2026 · Bui, Huang, Ying · arXiv:2609.07816 · Used in: research models only (Oct 2026)

$$S_t = S_{t-1}\mathrm{Diag}(\alpha_t)(I-(b_t\odot k_t)k_t^\top)+(w_t\odot v_t)k_t^\top \qquad \text{(GDN-2; } b=w=\beta\mathbf{1} \text{ gives KDA)}$$

$$S_t = S_{t\text{-}1}\,\mathrm{Diag}(\alpha_t)(I\text{-}(b_t\odot k_t)k_t^\top)+(w_t\odot v_t)k_t^\top$$ GDN-2 form

FG²-GDN (+)
Apr 2026



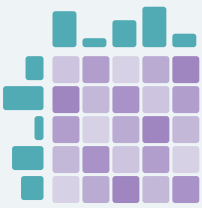
β per channel (key = value)

$k\to\sqrt{\beta}\odot k,\quad v\to\sqrt{\beta}\odot v$

strength: vector β
tested: 340M/15B, 1.3B/100B

Gated DeltaNet-2
May 2026

b: erase, per key



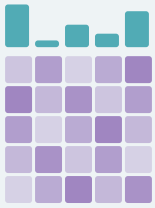
w: write, per value

$b\in\mathbb{R}^{d_k},\quad w\in\mathbb{R}^{d_v}$

erase b ≠ write w
tested: 1.3B/100B

Kalman Delta Networks
Sep 2026

p: uncertainty per key

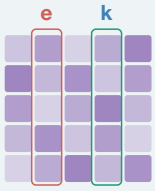


$\kappa = p\odot k\,/\,(r+\Sigma_i\, p_i k_i^2)$

step from uncertainty p
tested: 750M/50B, 1.3B/100B

Erase-then-Delta
Jun 2026

erase stale at e, write at k



$(I\text{-}\gamma ee^\top)(I\text{-}\beta kk^\top)$

address: e ≠ k
tested: 2.5B dense, 25B-A2.8B MoE

RWKV-7 Mar 2025 · FG²-GDN Apr 2026 · GDN-2 May 2026 · EDA Jun 2026 · KDN Sep 2026

per-channel write strength first shipped in RWKV-7 (vector a), a year before FG²-GDN

Correction:
per-channel write strength is not new. RWKV-7's vector a_t had it; FG²-GDN brings it to GDN/KDA kernels.

Strength vs. address:
FG²-GDN+ and GDN-2 gate how much is erased and written at k. EDA erases along a separate direction e_t.

Gain:
KDN's isotropic variant is KDA with β_t set by a scalar covariance; the diagonal variant writes along a gain vector.

Not solved:
only EDA was trained beyond 1.3B. No released model uses any of them.

11 · Memory as test-time learning

A recurrent state is a small model trained during inference. Changing its loss, optimizer or batch size gives the newest memory layers.

01 Every update is one optimizer step

$$S_t = \alpha_t S_{t-1} - \beta_t \nabla_S \ell_t(\alpha_t S_{t-1}), \qquad \ell_t^{\mathrm{LA}}(S) = -\langle Sk_t, v_t\rangle, \qquad \ell_t^{\Delta}(S) = \tfrac{1}{2}\left\|Sk_t - v_t\right\|^2$$

$$S_t = \alpha_t S_{t-1} - \beta_t \nabla \ell_t(\alpha_t S_{t-1}) \qquad \text{read} \qquad o_t = S_t q_t$$

inputs (toy)

S_{t-1} (3×3)

1	0	0
0	1	0
0	0	0

stores $e_1 \to e_1$, $e_2 \to e_2$

k_t	v_t
1	0
0	0
0	1

rewrite key $e_1 \to e_3$

Linear attention $\ell = -\langle Sk, v\rangle$

$\nabla \ell = -vk^\top$

0	0	0
0	0	0
−1	0	0

$-\beta\cdot$ →

S_t

1	0	0
0	1	0
1	0	0

k_t →

$S_t k_t$

1
0
1

vs

v_t

0
0
1

error 1.0
old value survives

DeltaNet $\ell = \tfrac{1}{2}\|Sk - v\|^2$

$\nabla \ell = (Sk - v)k^\top$

1	0	0
0	0	0
−1	0	0

$-\beta\cdot$ →

S_t

0	0	0
0	1	0
1	0	0

k_t →

$S_t k_t$

0
0
1

vs

v_t

0
0
1

error 0
old value replaced

Decay α < 1 before the step = weight decay (Mamba-2, GDN).

Other optimizers → TTT, Titans, ATLAS, Miras

Why it works:
S is a regressor's weights and the read $S_t q_t$ is its prediction. The outer loop trains the projections and gates.

Compare:
linear attention and DeltaNet differ only in the loss. Details: 10 · The delta rule.

Gotcha:
this is a reading of fixed algebra. At inference, the inner learner has no free choices.

Framework:
GDN Table 1 lists these objectives. Miras splits any layer into memory, loss, retention, optimizer.

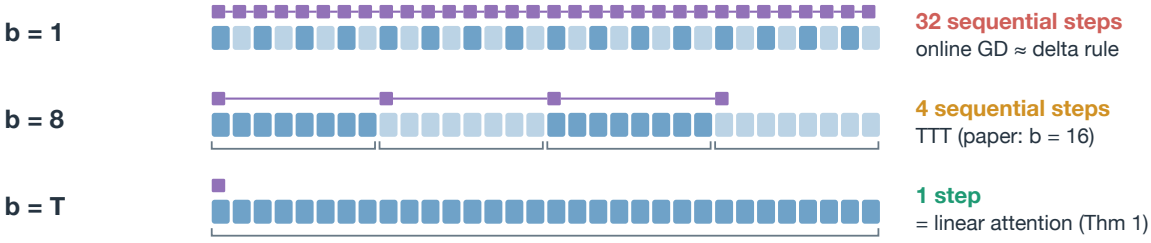
Rule	Loss $\ell_t(S)$	Gradient	Decay α_t
Linear attention	$-\langle Sk_t, v_t\rangle$	$-v_t k_t^\top$	1
Mamba-2, RetNet	$-\langle Sk_t, v_t\rangle$	$-v_t k_t^\top$	scalar < 1
DeltaNet	$\frac{1}{2}\left\|Sk_t - v_t\right\|^2$	$(Sk_t - v_t)k_t^\top$	1
Gated DeltaNet	$\frac{1}{2}\left\|Sk_t - v_t\right\|^2$	$(Sk_t - v_t)k_t^\top$	scalar < 1

02 TTT layers: the state is a model, b is the knob

TTT-Linear / TTT-MLP 2024 · Sun, Li, Dalal et al., Stanford · UC San Diego · UC Berkeley · ICML 2025 · arXiv:2407.04620 · Used in: research models, 125M–1.3B

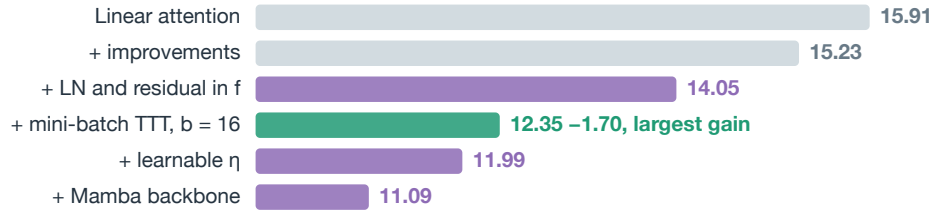
$$W_t = W_{t-1} - \eta_t \nabla_W \| f(\theta_K x_t; W_{t'}) - \theta_V x_t \|^2, \quad t' = t - (t \bmod b); \qquad z_t = f(\theta_Q x_t; W_t)$$

$$W_t = W_{t\text{-}1} - \eta_t \nabla \ell(W_{t'}; x_t), \qquad t' = t - (t \bmod b)$$



Reported ablation, 125M on the Pile (T = 2048), perplexity ↓

TTT-Linear, 1.3B: 32 heads × 64 dims



Why it works:
a batch shares $W_{t'}$, so b gradients run as one matmul. The dual form trains >5× faster than the primal.

Compare:
$b = 1$ is online GD (delta-like). $b = T$ with linear f, $\eta = \frac{1}{2}$, $W_0 = 0$ is linear attention (Thm 1).

Gotcha:
$f(x) = x + \mathrm{LN}(f_{\mathrm{res}}(x))$, so TTT-Linear is not DeltaNet. TTT-MLP: 4× hidden, GELU.

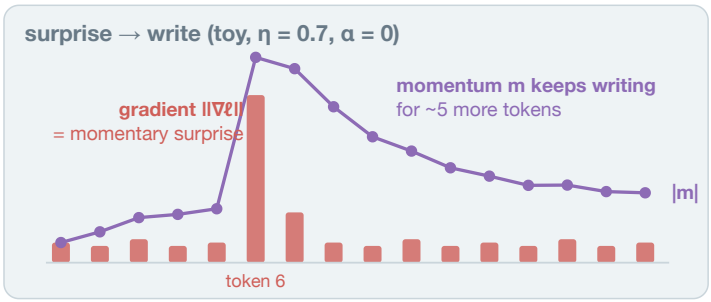
Numbers:
$b = 16$ throughout; $\eta = \eta_{\mathrm{base}}\, \sigma(\theta_{lr} x)$, η_{base} = 1 (Linear) or 0.1 (MLP).

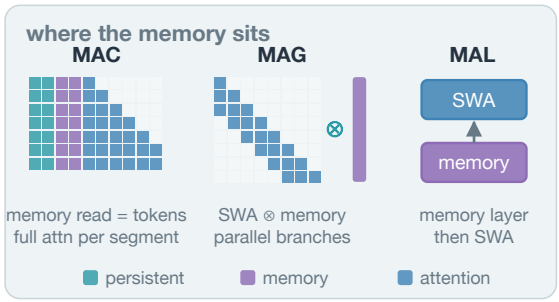
03 Titans: surprise, momentum, forgetting

Titans 2024 · Behrouz, Zhong, Mirrokni, Google Research · NeurIPS 2025 · arXiv:2501.00663 · Used in: research models, 170M–760M

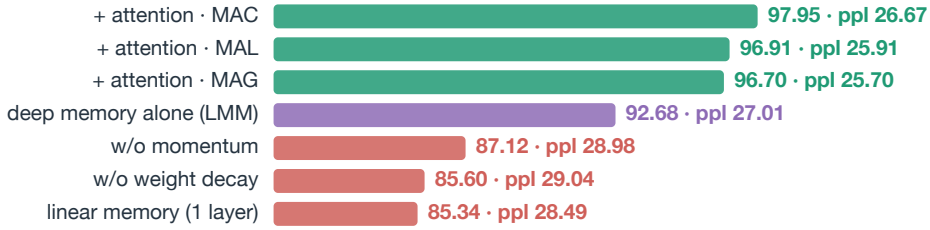
$$\mathcal{M}_t = (1 - \alpha_t)\mathcal{M}_{t-1} + m_t, \qquad m_t = \eta_t m_{t-1} - \theta_t \nabla \| \mathcal{M}_{t-1}(k_t) - v_t \|^2, \qquad y_t = \mathcal{M}_t(q_t)$$

$$M_t = (1 - \alpha_t) M_{t\text{-}1} + m_t, \qquad m_t = \eta_t\, m_{t\text{-}1} - \theta_t \nabla \| M_{t\text{-}1}(k_t) - v_t \|^2$$





Reported ablation (Table 5): long-context accuracy ↑ · perplexity ↓



Why it works:
gradient = surprise. Momentum m_t (the paper's S_i) keeps writing after a surprising token; α_t is weight decay.

Compare:
the gradient uses the undecayed $\mathcal{M}_{t-1}$, so linear Titans is the sum form, like RWKV-7, not GDN's product.

Cost:
$\mathcal{M}$ is an MLP. Deep memory loses closed-form chunk algebra and trains slower than Mamba2/GDN (§5.8).

Numbers:
removing weight decay costs the most: perplexity 27.01 → 29.04 (Table 5).

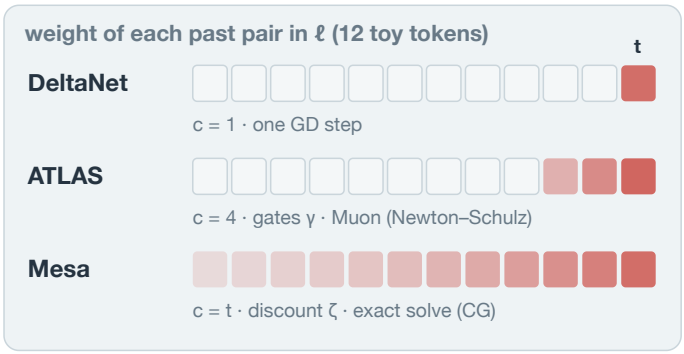
04 ATLAS and MesaNet: fit the window, or solve it

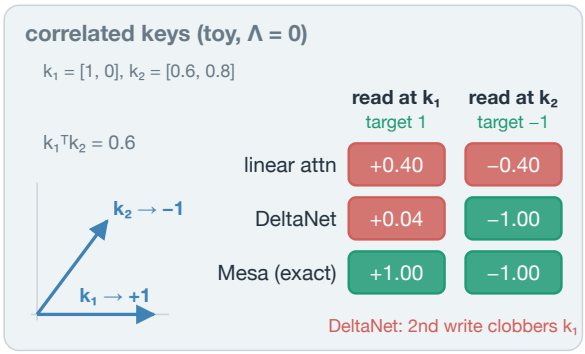
ATLAS 2025 · Behrouz et al., Google Research · arXiv:2505.23735 — **MesaNet** 2025 · von Oswald, Scherrer et al., Google Paradigms of Intelligence · ICLR 2026 · arXiv:2506.05233 · Used in: research models, ≤1.3B

ATLAS (Omega rule): $\min_{\mathcal{M}} \sum_{i=t-c+1}^{t} \gamma_i^{(t)} \left\| \mathcal{M}(\phi(k_i)) - v_i \right\|^2$, Muon steps

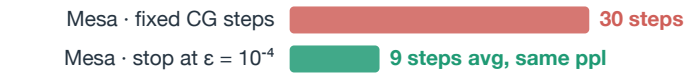
Mesa: $o_t = G_t (H_t + \Lambda)^{-1} q_t, \quad G_t = \gamma_t G_{t-1} + \beta_t v_t k_t^\top, \quad H_t = \gamma_t H_{t-1} + \beta_t k_t k_t^\top$

$$\ell_t(M) = \Sigma_{i=t-c+1}^{\ t} \gamma_i \ \| M(\varphi(k_i)) - v_i \|^2 \qquad c = 1\text{: delta} \cdot c = t\text{: least squares}$$





Cost readouts (reported)



k CG steps ≈ k × GLA FLOPs; k = 0 is GLA.
extra state $H \in \mathbb{R}^{d_k \times d_k}$: < 1 % of footprint

ATLAS capacity bound (Prop. 1–2):
$O(d_k)$ pairs → $O(d_k^{\,p})$ with degree-p φ
Newton–Schulz steps: test-time compute knob.
BABILong at 10M tokens: > 80 % acc; Titans drops.

Compare:
$c = 1$ is the delta rule. $c = t$ with linear $\mathcal{M}$ is least squares: DeltaNet takes one step, Mesa solves it.

Cost:
Mesa ≈ $k\times$ GLA FLOPs for k CG steps. Stopping at $\varepsilon = 10^{-4}$ matches 30 steps with ≈9 (reported).

Gotcha:
Mesa's writes stay additive and chunk-friendly. The editing happens at read time, inside $(H_t + \Lambda)^{-1}$.

Numbers:
ATLAS capacity bound $O(d_k) \to O(d_k^p)$ with degree-p features; BABILong above 80% at 10M (reported).

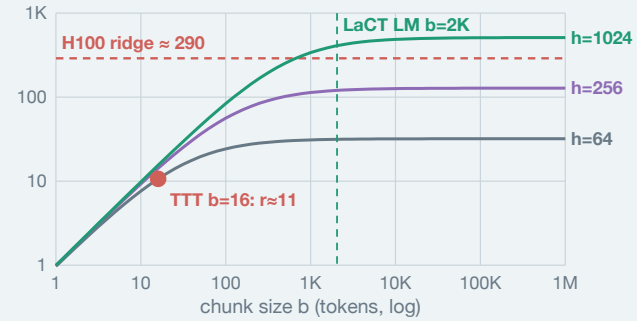
05 LaCT and TTT-E2E: huge chunks make it fast

LaCT 2025 · Zhang, Bi, Hong et al., MIT · Adobe Research · ICLR 2026 · arXiv:2505.23884 — **TTT-E2E** 2025 · Tandon, Dalal, Li et al., Astera · Stanford · NVIDIA et al. · arXiv:2512.23675 · Used in: research; LaCT also in a 14B autoregressive video model fine-tuned from Wan 2.1

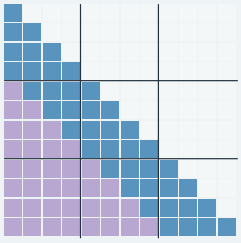
$$W \leftarrow \mathrm{L2Norm}\left(W - \mathrm{Muon}\left(\nabla_W \sum_{i=1}^{b} -\eta_i f_W(k_i)^\top v_i\right)\right), \qquad f_W(x) = W_2[\mathrm{SiLU}(W_1 x) \odot W_3 x]$$

$r = 2h^2 b \,/\, (2h^2 + 4hb) \le \min(h/2,\ b)$ FLOP/B one update per $b = 2\text{K}–1\text{M}$ tokens

intensity of one fast-weight matmul (Eq. 3)



LaCT language model: who reads what

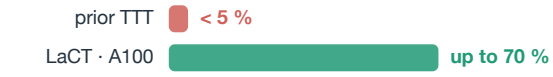


toy: 12 tokens, chunk 4

- via W: earlier chunks (apply, then update)
- SWA, window = chunk

current chunk is invisible to W → local attention is mandatory

FLOPs utilization (reported)



3B, 32K, A100: train tok/s 4.3K (LaCT-Muon) vs 4.1K full attention

Fast-weight state per block (3B)



largest LaCT state: $12d^2$ per block ≈ 40 % of weights

Why it works:
one update per 2K–1M tokens makes TTT matmul-bound and lets the state grow to ~40% of weights.

Gotcha:
a chunk never sees its own update, so every LaCT layer pairs with window attention.

TTT-E2E:
SWA-8K Transformer; trains last-¼ MLPs by next-token loss, 1K mini-batches. Prefill 2.7× faster than full attention at 128K (3B).

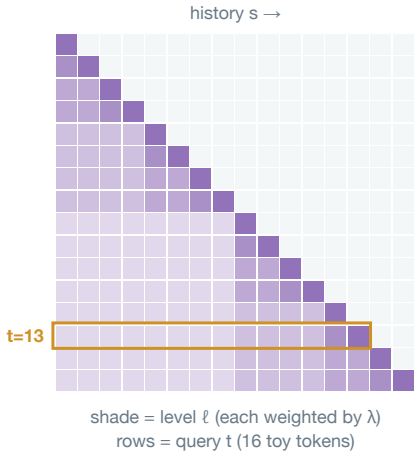
Cost:
gradients of gradients in training: 3.4× slower than full attention at 8K, 1.2× faster at 128K.

06 Log-linear attention: $O(\log T)$ states

Log-Linear Attention 2025 · Guo, Yang, Goel, Xing, Dao, Kim, MIT · CMU · Princeton · ICLR 2026 · arXiv:2506.04761 · Used in: research models (log-linear Mamba-2 and Gated DeltaNet, ~800M)

$$o_t = \sum_{\ell=0}^{L-1} \lambda_t^{(\ell)} S_t^{(\ell)} q_t, \qquad S_t^{(\ell)} = \sum_{s \in \mathcal{B}_t^{(\ell)}} v_s k_s^\top, \qquad L \le \lceil \log_2 t + 1 \rceil + 1$$

$o_t = \Sigma_\ell\ \lambda_t^{(\ell)}\ S_t^{(\ell)}\ q_t, \qquad S_t^{(\ell)} = \Sigma_{s \in B_t^{(\ell)}}\ v_s\ k_s^\top$

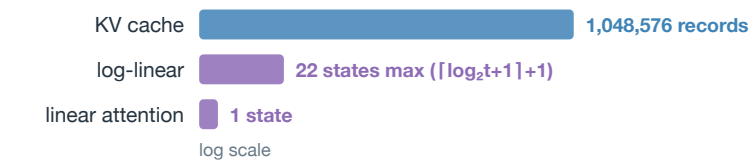


query t = 13 reads 4 states



$13 = 1101_2$: one bucket per set bit, plus the token itself.
Each new token: merge buckets up to lssb(t), promote one level.

States held at $t = 2^{20} \approx 1\text{M}$ (per head, per layer)



Reported, 21 layers, 50B tokens, 16K: Wiki ppl Mamba-2 22.44 → 22.11, GDN 21.73 → 21.45; λ adds < 3 % / < 0.4 % parameters.

Compare:
one bucket per token is the KV cache; equal λ across levels collapses to linear attention (§3.1).

Cost:
decode keeps $O(\log T)$ states; training is a chunkwise scan, $O(T \log T)$.

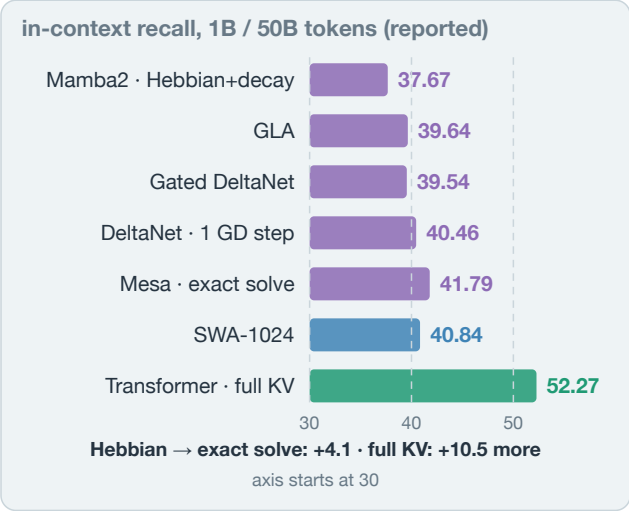
Used in:
wraps any linear cell. λ adds <3% parameters to Mamba-2 and <0.4% to GDN.

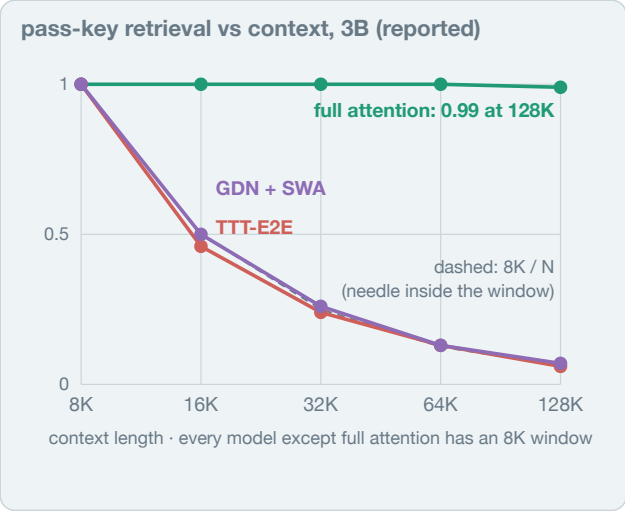
Not solved:
the authors still report a significant gap to Transformers on every benchmark.

07 Limits: learning is not recall

$\#\{\text{pairs stored exactly}\} \le O(d_k)$ per $d_v \times d_k$ state (ATLAS Prop. 1) vs. N for a KV cache

exactly recallable pairs $\le O(d_k)$ per $d_v \times d_k$ state vs. N for a KV cache





Not solved:
Hebbian → exact solve adds ~4 recall points; the full cache adds 10.5 more (MesaNet Table 3).

Numbers:
TTT-E2E pass-key at 128K: 0.06 vs 0.99 for full attention. Its authors: compression "leaves out seemingly irrelevant details".

Gotcha:
ATLAS and Titans long-context wins are fine-tuned BABILong or ≤16K needles; check the setup.

Fix in practice:
keep a few full-attention layers: 12 · Hybrid stacks.

Layer	Loss	Optimizer	State
TTT-Linear / MLP	$\left\|f(\theta_K x) - \theta_V x\right\|^2$	GD, mini-batch $b = 16$	W or 2-layer MLP per head
Titans	$\left\|\mathcal{M}(k) - v\right\|^2$	GD + momentum + decay	MLP, depth $L_{\mathcal{M}} \ge 1$
ATLAS	window of c pairs, $\phi(k)$	Muon	memory $\mathcal{M}$ on $\phi(k)$
Mesa	all pairs, discounted, ridge	exact (CG)	G and H
LaCT	$-f_W(k)^\top v$	Muon per 2K–1M chunk	SwiGLU, up to $12d^2$

Layer	Loss	Optimizer	State
TTT-E2E	next-token loss	GD, 1K mini-batch	last ¼ of the MLPs

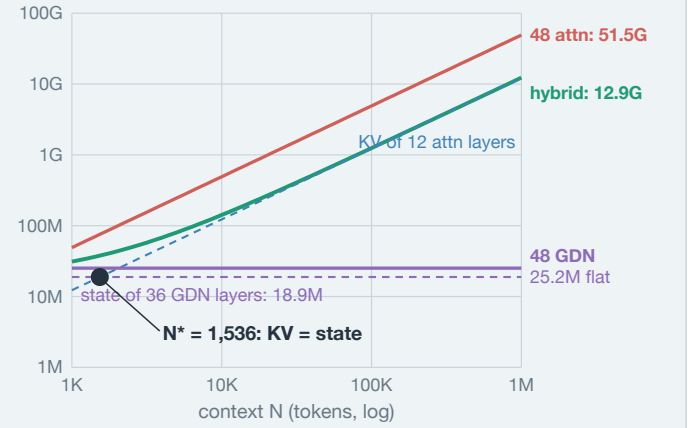
12 · Hybrid stacks

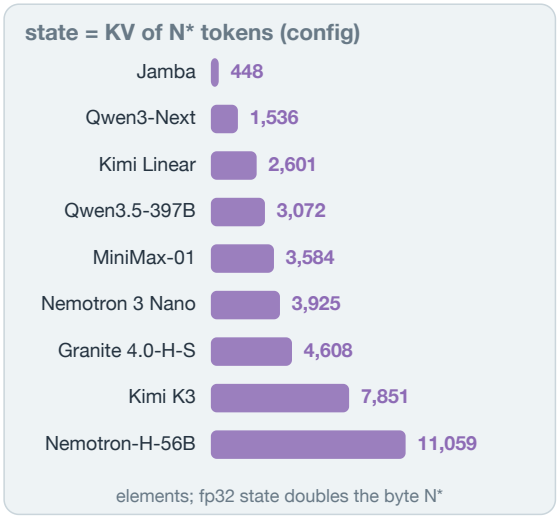
A fixed state compresses but cannot look things up. Production models keep a few full-attention layers and make the rest cheap.

01 Why hybrid: state is cheap, retrieval is not

$$M(N) = \sum_{\ell \in \mathcal{R}} |S_\ell| + N \sum_{\ell \in \mathcal{G}} c_\ell, \qquad N^* = \frac{\sum_{\ell \in \mathcal{R}} |S_\ell|}{\sum_{\ell \in \mathcal{G}} c_\ell}, \qquad c_\ell = 2H_{kv} d_h \ \text{(MLA: } d_c + d_r\text{)}$$

$$M(N) = \Sigma_{\ell \in R} |S_\ell| + N \cdot \Sigma_{\ell \in G} c_\ell \qquad N^* = \Sigma |S_\ell| \ / \ \Sigma c_\ell$$





Past N*, the few attention layers dominate memory. The hybrid cuts the KV slope 4×; it does not stop it growing.

Why it works:
recurrent layers carry a summary; a few attention layers keep every token for exact lookup (11 · Limits).

Numbers:
Qwen3-Next's whole state equals the KV of 1,536 tokens. Past that, its 12 attention layers dominate.

Gotcha:
Qwen3.5 and Nemotron 3 configs keep state in fp32 while KV is BF16, which doubles the byte crossover.

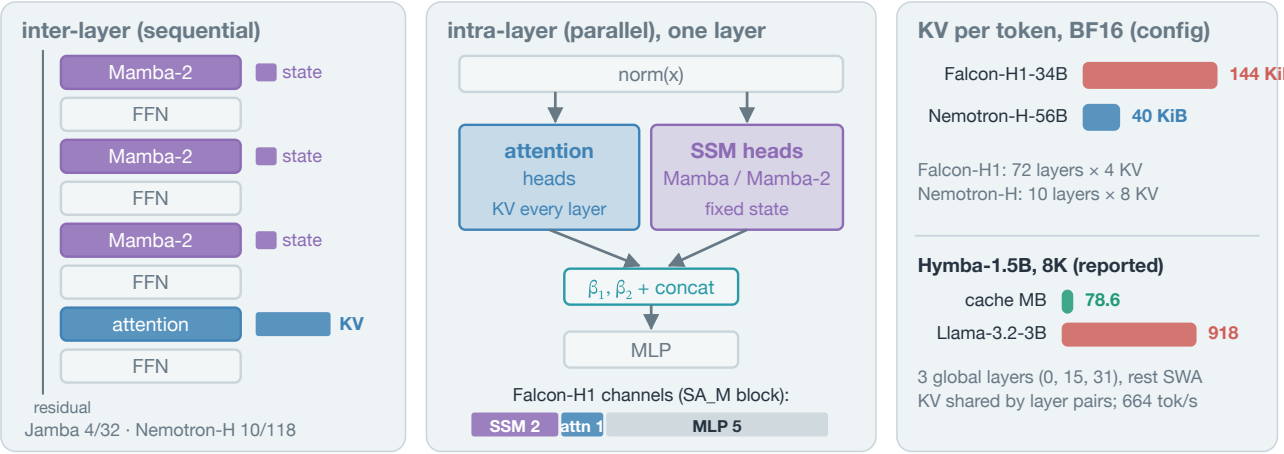
Not solved:
global-layer KV still grows with N, and their prefill stays quadratic.

02 Interleaved layers vs parallel heads

Jamba 2024 · AI21 Labs · arXiv:2403.19887 — **Hymba** 2024 · Dong, Fu et al., NVIDIA · arXiv:2411.13676 — **Falcon-H1** 2025 · TII · arXiv:2507.22448 · Used in: Jamba, Samba, Nemotron-H (sequential); Hymba-1.5B, Falcon-H1 (parallel)

sequential: $x_{\ell+1} = x_\ell + F_\ell(x_\ell),\ F_\ell \in \{\text{SSM}, \text{attn}\}$ parallel (Hymba): $Y = W_{\text{out}}(\beta_1 \odot \text{norm}(M_{\text{attn}}\tilde{X}) + \beta_2 \odot \text{n}$

$$\text{Hymba: } Y = W_{out}(\beta_1 \odot \text{norm}(M_{attn}X) + \beta_2 \odot \text{norm}(M_{ssm}X))$$



Parallel ≠ fewer KV layers: Falcon-H1 keeps KV in all 72 layers and saves through few KV heads.
Sequential hybrids put KV in few layers. Hymba gets both: parallel heads + SWA + cross-layer KV sharing.
Samba (Microsoft): Mamba-MLP-SWA(2048)-MLP repeated, no global layer at all.

Why it works:
in Hymba, attention heads give high-resolution recall and SSM heads summarize. Norms rescale before fusion; SSM outputs run larger.

Gotcha:
parallel ≠ fewer KV layers. Falcon-H1-34B keeps KV in all 72 layers (144 KiB/token, computed).

Numbers:
Hymba-1.5B at 8K: 11.67× smaller cache and 3.49× faster than Llama-3.2-3B (reported).

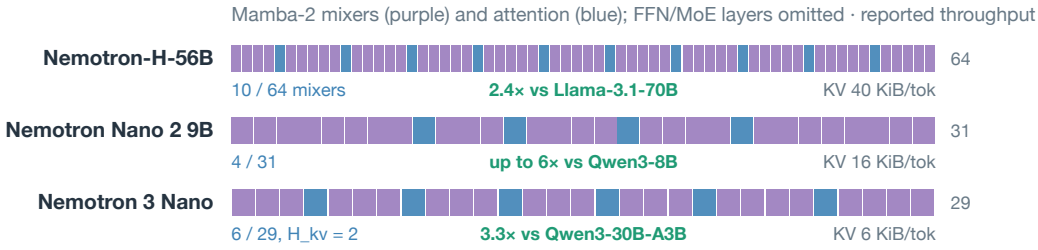
Compare:
Samba (ICLR 2025) interleaves Mamba with 2048-token SWA and has no global layer.

03 NVIDIA: Mamba-2 stacks and PostNAS retrofits

Nemotron-H 2025 · NVIDIA · arXiv:2504.03624 — **Jet-Nemotron** 2025 · Gu, Hu et al., NVIDIA · NeurIPS 2025 · arXiv:2508.15884 · Used in: Nemotron-H 8B/56B, Nemotron Nano 2, Nemotron 3 Nano, Jet-Nemotron 2B/4B

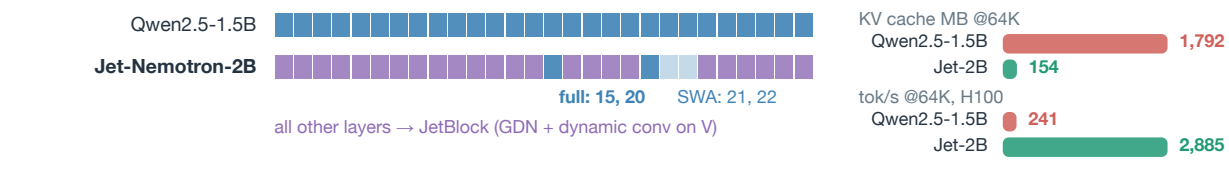
$$\text{KV/token} = L_{\text{attn}} \cdot 2H_{kv}d_h\, s, \qquad |S| = L_{\text{ssm}} \cdot H \cdot P \cdot N_{\text{state}}$$

$$\text{KV/tok} = L_{attn} \cdot 2H_{kv}\, d_h\, s \qquad \text{state} = L_{ssm} \cdot H \cdot P \cdot N_{\text{state}}$$



No position embeddings; first layer Mamba-2; attention ≈ 8 % of layers, evenly spread (Nemotron-H).

Jet-Nemotron-2B · PostNAS retrofit of Qwen2.5-1.5B (MLPs frozen, search where attention stays)



Reported: KV cache size, not parameter count, sets long-context generation throughput (Key Finding 4).
Attention importance is task-specific: retrieval and MMLU depend on different 2–3 layers.

Placement rule:
≈8% attention, evenly spread; first layer Mamba-2; no position embeddings (Nemotron-H §2).

Numbers:
Nemotron-H-56B: 2.4× output tokens/s of Llama-3.1-70B at 64K in, 1K out (reported, H100).

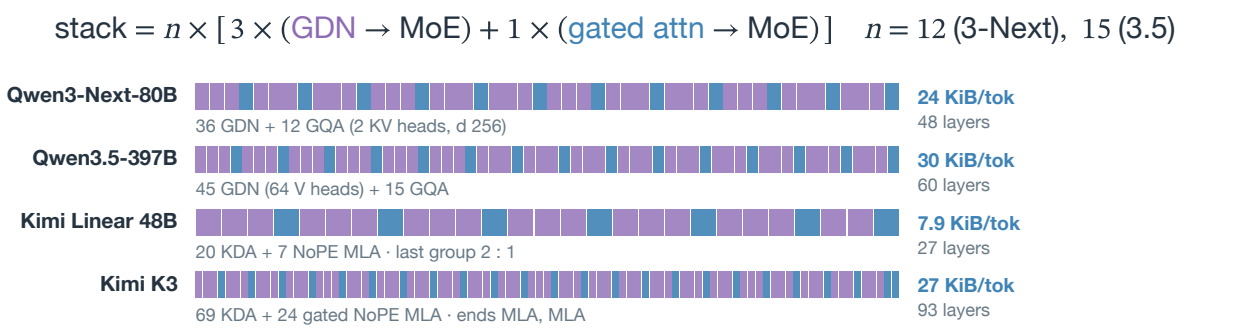
Converts from:
PostNAS freezes the MLPs, trains a once-for-all supernet and beam-searches where full attention stays.

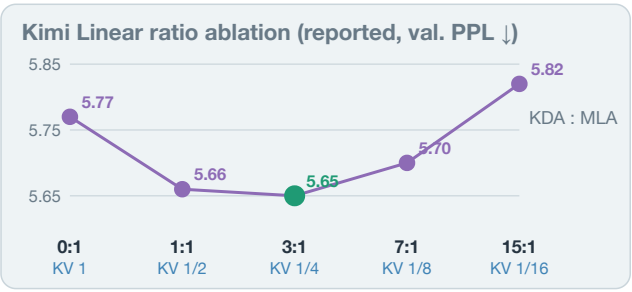
Gotcha:
important layers are task-specific: MMLU and retrieval each lean on different 2–3 layers (Jet-Nemotron).

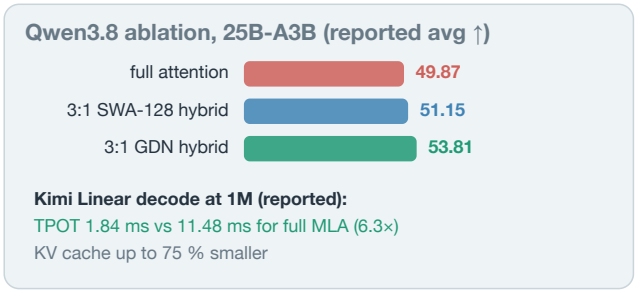
04 Qwen3-Next, Qwen3.5, Kimi: delta rule 3:1

Qwen3-Next / Qwen3.5 2025–26 · Qwen team, Alibaba · model card — **Kimi Linear** 2025 · Kimi Team, Moonshot AI · arXiv:2510.26692 · Used in: Qwen3-Next-80B-A3B, Qwen3.5-397B-A17B, Qwen3.8-Flash-Next, Kimi Linear 48B-A3B, Kimi K3

$$\text{stack} = n \times [\,3 \times (\text{GDN} \rightarrow \text{MoE}) + 1 \times (\text{gated attention} \rightarrow \text{MoE})\,], \qquad n = 12\ (\text{Qwen3-Next}),\ 15\ (\text{Qwen3.5})$$







Gotcha:
3:1 is nominal. Kimi Linear has 27 layers (last group 2:1); Kimi K3 ends with two MLA layers.

Compare:
Kimi's MLA uses NoPE, letting KDA carry position. Qwen keeps partial RoPE: its NoPE variant generated endlessly more often (01).

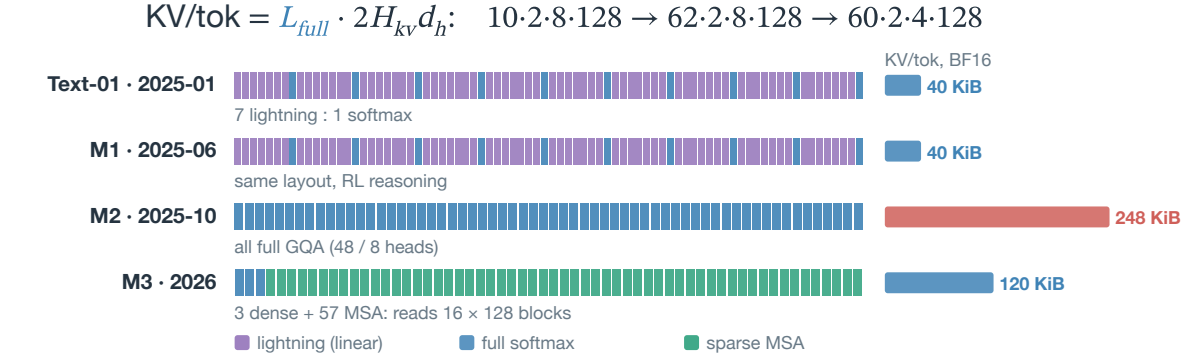
Numbers:
Kimi Linear at 1M: 1.84 vs 11.48 ms per output token for full MLA (reported).

Trend:
Qwen3.8 makes the global layer sparse (QSA, 06). Output gates: 02.

05 MiniMax: hybrid, then full, then sparse

MiniMax-01 / M1 2025 · MiniMax · arXiv:2501.08313, arXiv:2506.13585 — **M2** blog 2025-10-30 · Hugging Face — **M3 / MSA** 2026 · arXiv:2606.13392 · Used in: MiniMax-Text-01, M1, M2, M3

KV/token (elements) $= L_{\text{full}} \cdot 2H_{kv}d_h$: $10 \cdot 2 \cdot 8 \cdot 128 = 20,480 \rightarrow 62 \cdot 2 \cdot 8 \cdot 128 = 126,976 \rightarrow 60 \cdot 2 \cdot 4 \cdot 128 =$



Why M2 went back to full attention (MiniMax blog, 2025-10-30)

1	**Hidden cost**	Multi-hop reasoning deficits showed only at scale; old benchmarks missed them.
2	**Evidence is expensive**	Hard tasks need huge runs for a signal; precision bugs surfaced in M1 RL.
3	**Infrastructure**	Memory-bound kernels; low-precision state; prefix caching; speculative decoding.
4	**SWA retrofit failed**	Continued pretraining into an SWA hybrid degraded as context grew.

Same blog: the theoretical compute/memory crossover lies at a few thousand tokens.

Not solved:
MiniMax-01's own study found pure linear attention unable to retrieve, hence softmax every 8th layer.

Numbers:
M1 used ~25% of DeepSeek-R1's FLOPs at 100K generated tokens (reported).

Gotcha:
M2's reasons are evaluation and infrastructure, not a proof against hybrids; the blog expects them to be solvable.

Compare:
M3 keeps all KV and bounds reads; reported 9× prefill and 15× decode vs M2 at 1M (06).

06 Frontier layouts 2024–2026

$$\text{KV/token} = \sum_{\ell \in \mathcal{G}} c_\ell \quad \text{(global layers only; SWA bounded by } W\text{, recurrent by } |S_\ell|\text{)}$$



Read it off config:
`layer_types`, `hybrid_override_pattern`, `full_attn_layers`, `attn_type_list`, `attn_layer_period`.

Numbers:
global share clusters at 1/2 (gpt-oss), 1/4 (Qwen, Kimi), 1/6 (Gemma 3), 1/8–1/10 (Jamba, MiniMax-01, Granite).

Gotcha:
"hybrid" can mean SWA, SSM, delta-rule or parallel heads beside global attention. KV savings differ by family.

Not solved:
MiniMax went the other way: M2 is all full attention, and M3 still stores every token.

Family	Examples	Global layers	Bounded part
Local / global	Gemma 3, gpt-oss	1/6, 1/2	SWA cache (W = 1024, 128)
SSM + attention	Jamba, Nemotron-H, Nemotron 3 Nano, Granite 4.0-H	4–10 layers	Mamba state
Delta rule + attention	Qwen3-Next, Qwen3.5, Kimi Linear, Kimi K3	~1/4	GDN / KDA state
Lightning + softmax	MiniMax-Text-01, M1	1/8	lightning state
Parallel heads	Hymba, Falcon-H1	every layer	SSM state
Full / sparse	MiniMax-M2, M3	all	M3: reads only

13 · Kernels

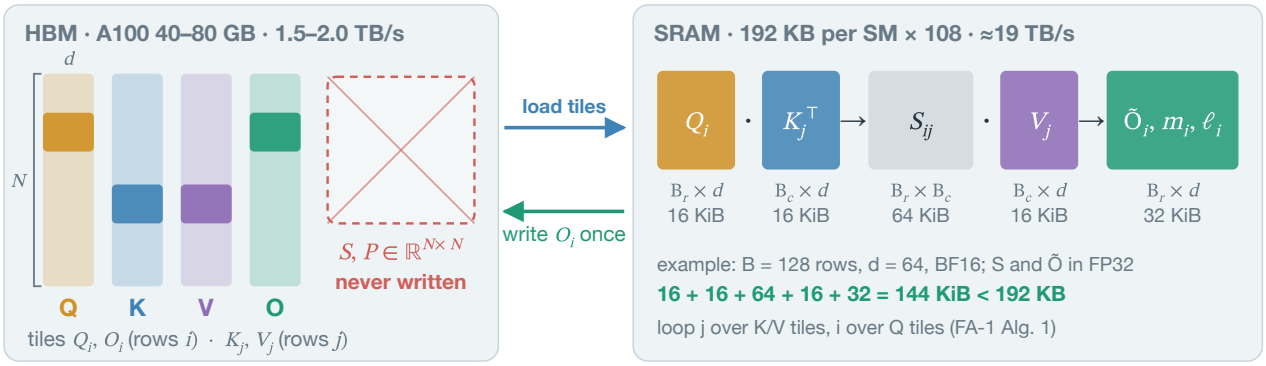
Same exact attention, different schedule: kernels decide how many bytes cross HBM and how many SMs stay busy.

01 FlashAttention: tiles in SRAM, no $N \times N$ in HBM

FlashAttention 2022 · Dao, Fu, Ermon, Rudra, Ré · NeurIPS 2022 · arXiv:2205.14135 · Used in: PyTorch SDPA, vLLM, Megatron-LM

$$m' = \max(m, m_j), \qquad \ell' = e^{m-m'}\ell + e^{m_j-m'}\ell_j, \qquad \tilde{O}' = e^{m-m'}\tilde{O} + e^{s_j-m'}V_j, \qquad O = \tilde{O}/\ell$$

HBM accesses: standard $\Theta(Nd + N^2)$ → FlashAttention $\Theta(N^2 d^2 / M)$, M = SRAM size



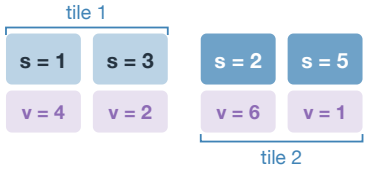
Reported (FA-1 Fig. 2): GPT-2 medium attention fwd + bwd, N = 1024, d = 64, 16 heads, batch 64, A100



Fewer bytes beats fewer FLOPs: 9× less traffic, 5.7× faster, 13% more arithmetic.

$$m' = \max(m, m_j) \qquad \ell' = e^{m-m'}\ell + e^{m_j-m'}\ell_j \qquad \tilde{O}' = e^{m-m'}\tilde{O} + \Sigma_j\, e^{s_j-m'}\, v_j$$

toy · one query row, 4 keys, 2 tiles



	m	ℓ	$\tilde{O}$
after tile 1	**3**	$e^{-2} + 1 = 1.135$	$4e^{-2} + 2 = 2.541$
tile 2 alone	**5**	$e^{-3} + 1 = 1.050$	$6e^{-3} + 1 = 1.299$
rescale the old (m, ℓ, $\tilde{O}$) by		$e^{3-5} = 0.135$	
merged	**5**	0.135·1.135 + 1.050 **= 1.203**	0.135·2.541 + 1.299 **= 1.643**

$$o = \tilde{O} / \ell = 1.643 / 1.203 = 1.365$$

check, full softmax: $\Sigma\, e^{s_i-5} = 1.203$

$(4e^{-4} + 2e^{-2} + 6e^{-3} + 1) / 1.203 = 1.365$

On chip between tiles: only $(m, \ell, \tilde{O})$ per query row, i.e. $B_r(d+2)$ numbers, never the score row.

Why it works:
softmax needs only a row max m and row sum ℓ, and both merge across tiles exactly.

Cost:
HBM accesses fall from $\Theta(Nd + N^2)$ to $\Theta(N^2d^2/M)$ (Thm. 2). FLOPs stay $O(N^2d)$; the backward pass recomputes S.

Numbers:
A100 SRAM ≈19 TB/s vs HBM 1.5–2.0 TB/s. Reported: 3× GPT-2 training, 15% faster BERT-large.

02 FA-2 → FA-3 → FA-4: re-tiled for each GPU

FlashAttention-2 2023 · Dao · ICLR 2024 · arXiv:2307.08691 — **FlashAttention-3** 2024 · Shah, …, Dao · NeurIPS 2024 · arXiv:2407.08608 — **FlashAttention-4** 2026 · Zadouri, …, Dao · arXiv:2603.05451

$O = \mathrm{diag}(\ell)^{-1}\tilde{O}$ once, after the last tile (FA-2); $m' = m$ unless $m_j - m > \tau$, $2^\tau = 256$ (FA-4)

matmul : non-matmul throughput 16× (A100) → 256× (H100) → 512× (B200)

FA-2 · 2023 A100

matmul BF16 312 TF

FP32 non-matmul 19.5 TF **16×**

- divide by ℓ once, at the end
- parallelize over N (Q blocks), not only batch × heads
- split Q (not K) over 4 warps: no shared-memory reduction

FA-3 · 2024 H100

matmul FP16 989 TF

exp (MUFU) 3.9 TF **256×**

- TMA producer warps feed WGMMA consumer warps
- ping-pong: softmax under the other warpgroup's GEMM
- FP8: block scales + Hadamard

ping-pong: 570 → 620–640 TFLOP/s

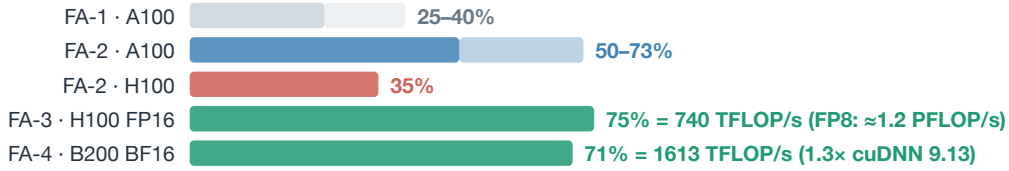
FA-4 · 2026 B200

tensor ops/clk/SM 8192

MUFU exp ops/clk/SM 16 **512×**

- exp2 as a polynomial on FMA units for 10–25% of entries
- rescale only if the max grows by > 8 ($\log_2 256$)
- tensor memory + 2-CTA MMA

Reported utilization of peak (bar = % of peak)



Why it works: every non-matmul FLOP (exp, rescale) costs 16–512 matmul FLOPs, so each version hides or removes them.

Numbers: reported FA-4 on B200 BF16: 1.3× cuDNN 9.13, 2.7× Triton; CuTe-DSL compiles 20–30× faster than C++ templates.

Gotcha: speed is tied to hardware. FA-2 reached only 35% on H100; FA-3 needs Hopper; FA-4 targets Blackwell.

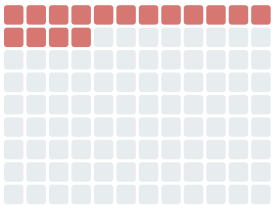
03 Split-KV decoding (Flash-Decoding)

Flash-Decoding 2023 · Dao, Haziza, Massa, Sizov (Stanford CRFM, Meta) · blog · Used in: FlashInfer (split-KV scheduler), FlashMLA (`num_splits`)

$$o = \sum_{s=1}^{S} e^{\mathrm{LSE}_s - \mathrm{LSE}}\, o_s, \qquad \mathrm{LSE}_s = m_s + \log \ell_s, \qquad \mathrm{LSE} = \log \sum_s e^{\mathrm{LSE}_s}$$

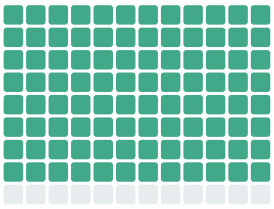
$o = \Sigma_s\, w_s\, o_s$ $\quad w_s = e^{\mathrm{LSE}_s - \mathrm{LSE}}$ $\quad \mathrm{LSE}_s = m_s + \log \ell_s$ $\quad \mathrm{LSE} = \log \Sigma_s\, e^{\mathrm{LSE}_s}$

one block per head



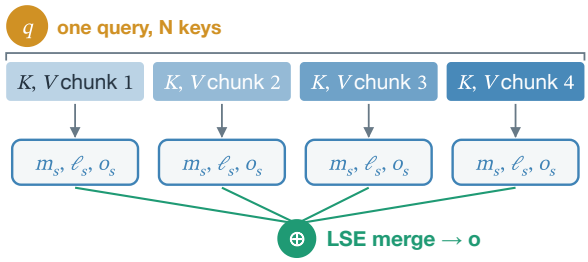
16 / 108 SMs = 15%

6 KV splits per head



96 / 108 SMs = 89%

toy: batch 1, 16 heads, A100 (108 SMs)



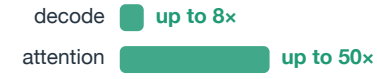
same bytes read, spread over more SMs

toy merge, 4 keys in 2 chunks

chunk	scores	values	m	ℓ	o	LSE	w
A	1, 2	10, 20	2	1.368	17.31	2.313	0.324
B	3, 0	30, 0	3	1.050	28.58	3.049	0.676

$o = 0.324 \cdot 17.31 + 0.676 \cdot 28.58 = 24.93$ = softmax over all 4 keys

Reported · CodeLlama-34B · A100



vs FlashAttention, 512 to 64K tokens

batch 1: FlashAttention used < 1% of GPU

Why it works: decode has one query row, so blocks = batch × heads. Splitting N adds blocks, and every SM pulls HBM bandwidth.

Cost: bytes read are unchanged; it adds S partial outputs and one small reduction.

Numbers: reported, A100: attention up to 50× faster than FlashAttention; at batch 1 FlashAttention used <1% of the GPU.

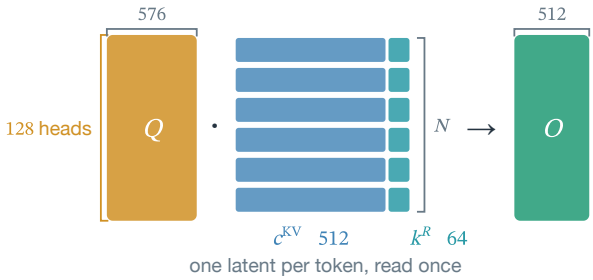
04 FlashMLA and FlashInfer: decode kernels for real caches

FlashMLA 2025 · DeepSeek · GitHub · Used in: DeepSeek-V3/V3.2/V4.1 serving — **FlashInfer** 2025 · Ye et al. · MLSys 2025 · arXiv:2501.01005 · Used in: SGLang, vLLM, MLC-Engine

$$I = \frac{H_q \cdot 2\left((d_c + d_r) + d_c\right)}{(d_c + d_r)\, s} = \frac{128 \cdot 2 \cdot (576 + 512)}{576 \cdot 2} \approx 242 \text{ FLOP/B}$$

$$I = H_q \cdot 2(576 + 512) \text{ FLOP} \;/\; \text{bytes per token} = 278{,}528 \,/\, 1152 \text{ B} \approx 242$$

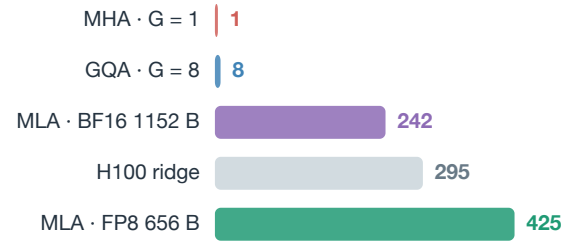
decode, one layer, absorbed weights (MQA mode)



one latent per token, read once

MMA rows = $s_q\, H_q \,/\, H_{kv} = 128 \rightarrow$ full tensor-core tiles

FLOP per KV byte at decode (computed)



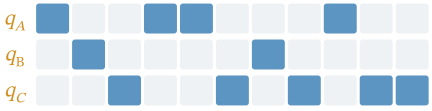
absorbed MLA sits near the ridge; MHA/GQA far below it

FlashMLA · reported, H800 SXM5



paged KV, block 64 tokens · FP8 cache since Sep 2025

FlashInfer · ragged paged batch = one block-sparse matrix



rows: query tiles (B_r) · columns: KV pages (B_c = page)

Why it works:
after weight absorption all heads share one latent, so $G = 128$ (intensity = G).

Gotcha:
FP8 KV (656 B) lifts intensity to ≈425 FLOP/B, past the ridge: absorbed MLA decode becomes compute-bound.

FlashInfer:
page tables and shared prefixes become BSR blocks; JIT variants. Reported 29–69% lower inter-token latency vs compiler backends.

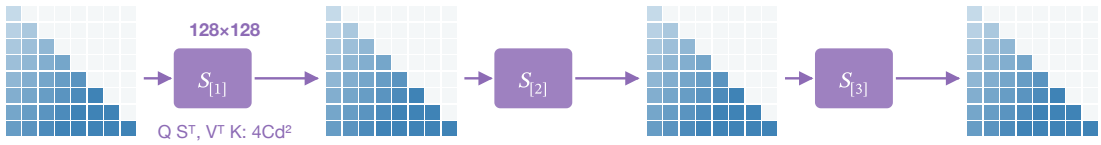
05 Chunkwise kernels for linear and delta layers (FLA)

FlashLinearAttention (GLA) 2023 · Yang, Wang, Shen, Panda, Kim · ICML 2024 · arXiv:2312.06635 — **DeltaNet chunkwise** 2024 · Yang et al. · NeurIPS 2024 · arXiv:2406.06484 · Used in: flash-linear-attention

$$O_{[c]} = (Q_{[c]} K_{[c]}^\top \odot M)\, V_{[c]} + Q_{[c]} S_{[c-1]}^\top, \qquad S_{[c]} = S_{[c-1]} + V_{[c]}^\top K_{[c]}$$

$$O_{[c]} = (Q_{[c]}\, K_{[c]}{}^\top \odot M)\, V_{[c]} + Q_{[c]}\, S_{[c-1]}{}^\top \qquad S_{[c]} = S_{[c-1]} + V_{[c]}{}^\top\, K_{[c]}$$

N tokens in chunks of C = 64 (GLA benchmark setting; C a multiple of 16 for tensor cores)



chunk 1
$(QK^\top \odot M)\, V$
4C²d FLOP, matmul

chunk 2
$(QK^\top \odot M)\, V$
4C²d FLOP, matmul

chunk 3
$(QK^\top \odot M)\, V$
4C²d FLOP, matmul

chunk 4
$(QK^\top \odot M)\, V$
4C²d FLOP, matmul

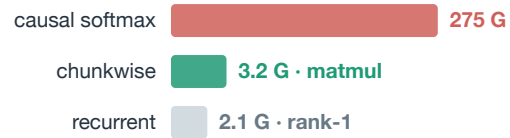
two schedules (GLA §3.3)

non-materialized: S stays in SRAM
64 KiB FP32 · chunks in sequence · parallel over B, H

materialized: all N/C states → HBM
512 × 64 KiB = 32 MiB/head at 32K · chunks in parallel

materialized: reported ≈10–20% more memory

FLOPs per head, N = 32K, d = 128 (computed, log axis)



reported: FLA faster than FlashAttention-2 even at 1K tokens (GLA, ICML 2024)

Math:
derivation and per-token costs are in 09. This card shows where S lives.

Gotcha:
decays need log-space factors inside a chunk; GLA adds a second sub-chunk level to keep half-precision matmuls.

Delta rule:
the chunk's product of Householder-like updates is rewritten in WY form, so it stays matmul-shaped (10).

Train vs. infer:
prefill uses chunks; decode is the one-token recurrence on the cached S.

06 Low-bit attention: SageAttention 1 → 3

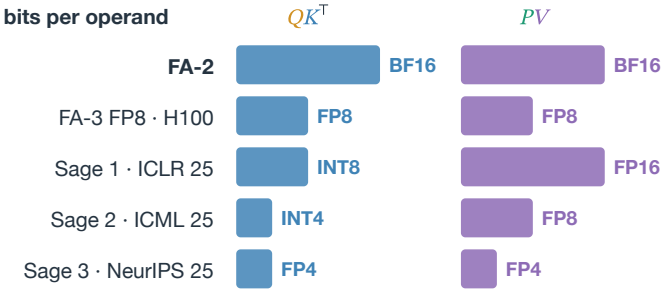
SageAttention 2024 · Zhang et al., Tsinghua · ICLR 2025 · arXiv:2410.02367 — **SageAttention2** · ICML 2025 · arXiv:2411.10958 — **SageAttention3** · NeurIPS 2025 · arXiv:2505.11594 · Used in: video diffusion inference (HunyuanVideo, CogVideoX, reported)

$$\mathrm{softmax}_j(q^\top(k_j - \mu_K)) = \mathrm{softmax}_j(q^\top k_j), \qquad QK^\top \approx \delta_Q\,\delta_K\,(Q_{\mathrm{int8}}K_{\mathrm{int8}}^\top)$$

$$\mathrm{softmax}_j(q^\top(k_j - \mu_K)) = \mathrm{softmax}_j(q^\top\ k_j)$$

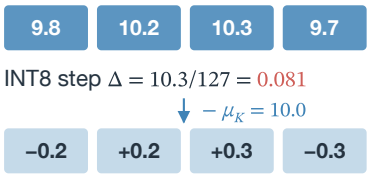
$$QK^\top \approx \delta_Q\ \delta_K \cdot Q_8\ K_8^{\ \top}$$

bits per operand



Sage 1: FP16 accumulator in PV (2× faster on RTX 4090)
Sage 3: NVFP4 = E2M1 values + E4M3 scale per 16

toy · one key channel, 4 tokens



INT8 step $\Delta = 10.3/127 = 0.081$

$\Delta = 0.3/127 = 0.0024$ (34× finer)

$q^\top \mu_K$ is the same for every key j
so the row softmax cancels it exactly

Reported kernel throughput vs FlashAttention-2 on the same GPU



Sage 3: 1038 TOPS on RTX 5090

FA-3 FP8: ≈1.2 PFLOP/s on H100, 2.6× lower error than baseline FP8 (RMSE 9.1e-3 vs 2.4e-2)

Sage 2 matches FA-3 FP8 speed on Hopper

Why it works:
keys share a large per-channel mean; $q^\top \mu_K$ is constant across j, so removing it is exact.

Compare:
this quantizes the *arithmetic*. KV-cache quantization quantizes *storage* and dequantizes before the matmul.

Not solved:
8-bit attention is lossless in fine-tuning but converges slower in pretraining (SageAttention3).

14 · Serving the cache

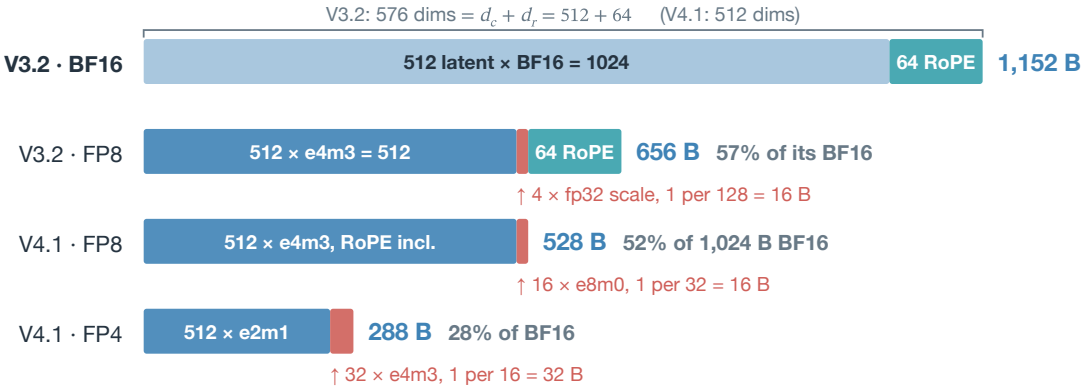
A server holds thousands of caches. Shrink each record, page them, reuse shared prefixes, ship them between GPUs, spill them to cheaper tiers.

01 FP8 / FP4 KV cache: the production default

FP8 / FP4 KV formats 2025–26 · DeepSeek · FlashMLA (V3.2 656 B, V4.1 528 / 288 B) · vLLM V3.2 blog · NVIDIA NVFP4 KV · Used in: DeepSeek-V3.2, DeepSeek-V4.1, vLLM

$$\text{bytes per token per layer} = \underbrace{n\,b/8}_{\text{quantized}} + \underbrace{(n/g)\,s_{\text{scale}}}_{\text{scales}} + \underbrace{2\,n_{\text{bf16}}}_{\text{kept}}, \qquad \text{V3.2: } 512 + 4 \cdot 4 + 64 \cdot 2 = 656$$

bytes / token / layer = $n \cdot b / 8$ + scales + unquantized (RoPE)



V3.2, all 61 layers (computed)

BF16 70,272 B/token → 8.58 GiB at 128K

FP8 40,016 B/token → 4.88 GiB at 128K

NVFP4 KV (NVIDIA, reported, Blackwell)

≈50% of FP8 · <1% accuracy loss

dequantized to FP8 before the attention matmuls

Gotcha:
scales and the BF16 RoPE lane make V3.2's FP8 cache 57% of BF16, not 50%. V4.1 quantizes RoPE too.

Cost:
saves memory and bandwidth, not FLOPs. NVFP4 KV is dequantized to FP8 before the matmuls (NVIDIA).

Numbers:
NVIDIA reports NVFP4 KV at ≈50% of FP8 memory with <1% accuracy loss on Blackwell.

Compare:
low-bit *arithmetic* inside the kernel is SageAttention.

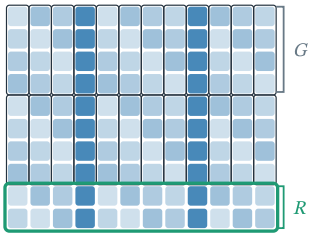
02 KIVI and KVQuant: quantize along the outliers

KIVI 2024 · Liu et al. · ICML 2024 · arXiv:2402.02750 — **KVQuant** 2024 · Hooper et al., UC Berkeley · NeurIPS 2024 · arXiv:2401.18079

$\hat{x} = z + \Delta\text{round}(\frac{x-z}{\Delta}), \quad \Delta = \frac{\max - \min}{2^b - 1}, \; z = \min; \qquad K$: group per channel, V: group per token

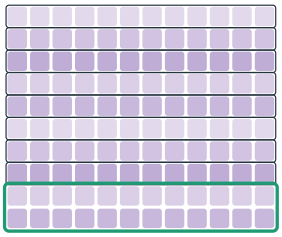
$$Q_b(x) = z + \Delta \cdot \text{round}((x - z)/\Delta) \qquad \Delta = (\max - \min)/(2^b - 1) \qquad z = \min \text{ of the group}$$

K: per-channel groups



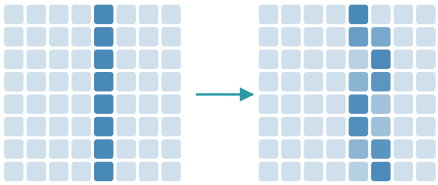
outlier channels stay inside their own group

V: per-token



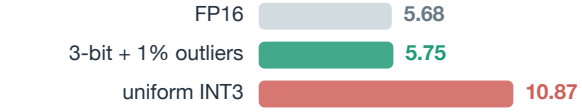
green: last R tokens in FP16
toy: 10 tokens × 12 channels

KVQuant: quantize K before RoPE



RoPE mixes channel pairs (4, 5) by position:
a clean outlier channel becomes a moving one

KVQuant · LLaMA-7B Wikitext-2 perplexity (reported)



non-uniform levels (per-layer k-means) · first token kept FP16

KIVI · 2-bit, tuning-free (reported)

2.6× lower peak memory (incl. weights)
up to 4× larger batch
2.35–3.47× throughput

KVQuant: 1M context on one A100-80GB

Why it works:
key outliers sit in a few fixed channels; values show no channel pattern (KIVI).

Gotcha:
RoPE smears those channels across positions. KVQuant dequantizes pre-RoPE keys, then applies RoPE.

Numbers:
KIVI 2-bit: 2.6× lower peak memory, up to 4× batch, 2.35–3.47× throughput. KVQuant: 1M tokens on one A100-80GB.

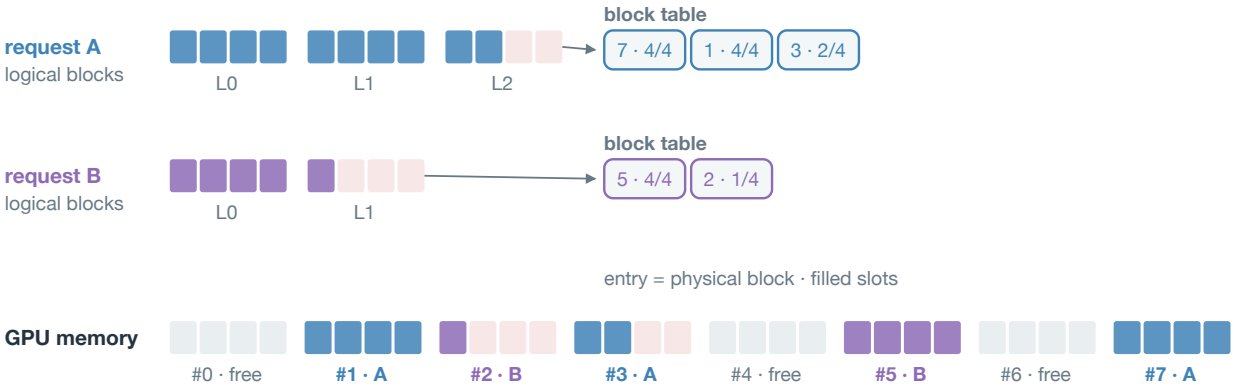
Converts from:
both are training-free; KVQuant calibrates its non-uniform levels offline.

03 PagedAttention: KV in fixed blocks

PagedAttention / vLLM 2023 · Kwon et al., UC Berkeley · SOSP 2023 · arXiv:2309.06180 · Used in: vLLM, SGLang (page = 1 token), FlashMLA (block 64)

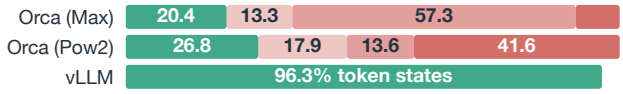
$\text{slot}(t) = \text{table}[\lfloor t/b \rfloor] \cdot b + (t \bmod b), \qquad \text{waste} \le b - 1 \text{ tokens per sequence}$

$$\text{slot}(t) = \text{table}[\lfloor t \,/\, b \rfloor] \cdot b + (t \bmod b) \qquad \text{waste} \le b - 1 \text{ tokens per sequence}$$



toy: b = 4 · red = unused slot in a last block

KV memory use, reported (vLLM Fig. 2)



legend: green token states · red: reservation, internal, external fragmentation (light → dark)

OPT-13B, 800 KB/token: reserve 2,048 tok = 1.6 GB
paged, b = 16: waste ≤ 15 tok = 12 MB (computed)

Why it works:
blocks are allocated on demand and all have one size: no reservation, no external fragmentation.

Numbers:
reported 2–4× throughput vs FasterTransformer and Orca; default $b = 16$.

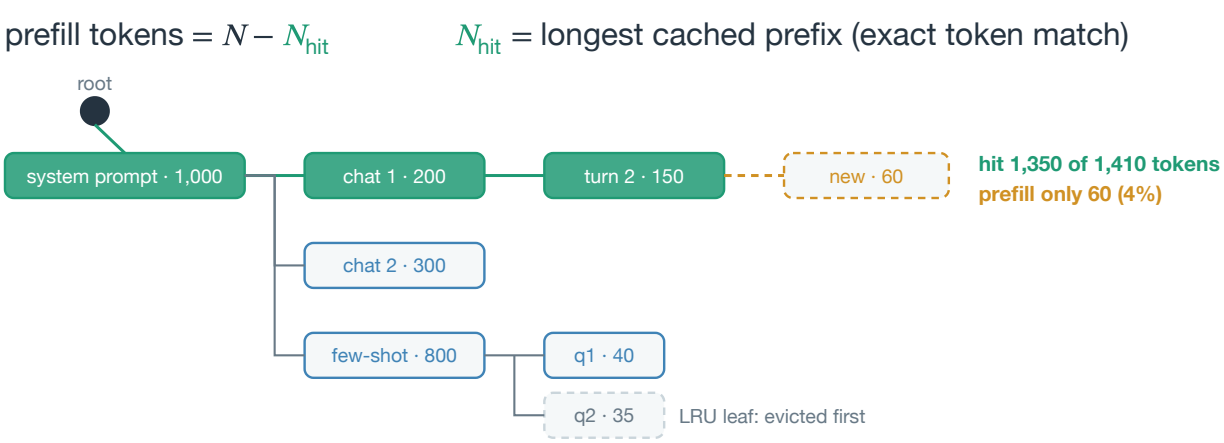
Sharing:
reference-counted blocks with copy-on-write save 6.1–9.8% (parallel sampling) and 37.6–55.2% (beam search).

Gotcha:
kernels must gather scattered blocks; FlashMLA and FlashInfer read the block table directly (13).

04 Prefix caching: RadixAttention

SGLang / RadixAttention 2023 · Zheng et al., Stanford / UC Berkeley · NeurIPS 2024 · arXiv:2312.07104 · Used in: SGLang

$$\text{prefill tokens} = N - N_{\text{hit}}, \qquad N_{\text{hit}} = \max\{\, n : x_{1:n} \text{ is a path in the radix tree} \,\}$$



Prefix-cache hit rates, reported



SGLang: up to 6.4× throughput
hit needs a byte-identical prefix:
context truncation can halve it (LMCache)

Why it works:
tree nodes own KV pages; LRU evicts leaves first, so shared prefixes survive longest.

Gotcha:
a hit needs the exact token prefix. Context truncation can halve the hit ratio (LMCache).

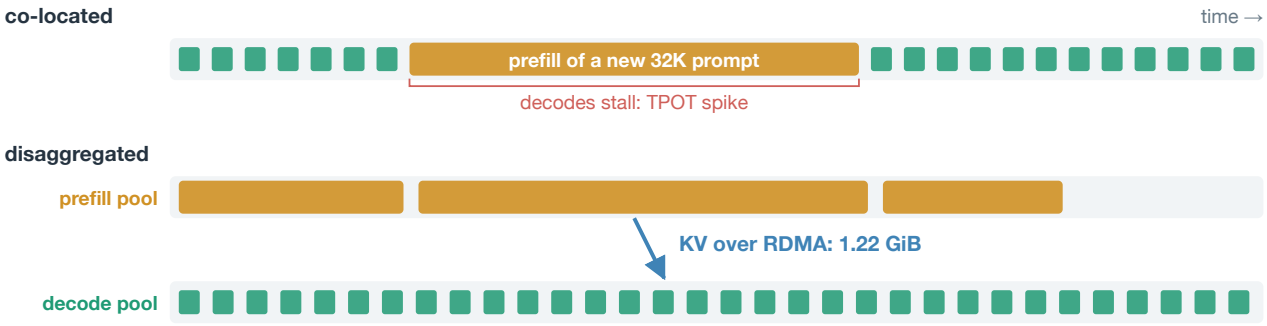
Numbers:
SGLang reports up to 6.4× throughput. DeepSeek: 342B of 608B input tokens hit its disk cache in 24 h.

05 Prefill–decode disaggregation

DistServe 2024 · Zhong et al. · OSDI 2024 · arXiv:2401.09670 — **Splitwise** 2024 · Patel et al., UW / Microsoft · ISCA 2024 · arXiv:2311.18677 — **Mooncake** 2025 · Qin et al., Moonshot AI / Tsinghua · FAST 2025 · arXiv:2407.00079 · Used in: Kimi, DeepSeek-V3/R1 serving

$$T_{\text{xfer}} = \frac{N_{\text{prompt}} \sum_{\ell} c_{\ell}}{\text{BW}}, \qquad \text{V3.2 FP8, 32K: } \frac{32,768 \cdot 61 \cdot 656\ \text{B}}{50\ \text{GB/s}} \approx 26\ \text{ms}$$

$T_{\text{xfer}} = N_{\text{prompt}} \cdot \Sigma_{\ell}\, c_{\ell}\ /\ \text{BW}$ — c_{ℓ} = KV bytes per token in layer ℓ



32K-token prompt, KV to ship (computed)

V3.2 · FP8 · 40,016 B/tok	1.22 GiB → 26 ms
Llama-3-70B · BF16 · 320 KiB/tok	10 GiB → 215 ms

50 GB/s = one 400 Gb/s NIC · Mooncake transfer engine, reported 190 GB/s on 8 × 400 Gb/s → 6.9 ms for V3.2

Reported

DistServe (OSDI 24)	7.4× requests or 12.6× tighter SLO
Splitwise (ISCA 24)	1.4× throughput at 20% lower cost
Mooncake (FAST 25)	+59–498% effective capacity
DeepSeek, Feb 2025	prefill 4 nodes · decode 18 nodes

Why it works:
prefill is compute-bound (TTFT), decode bandwidth-bound (TPOT). Separate pools stop interference and size each phase.

Cost:
transfer scales with KV per token: V3.2 FP8 ships 8× fewer bytes than Llama-3-70B BF16.

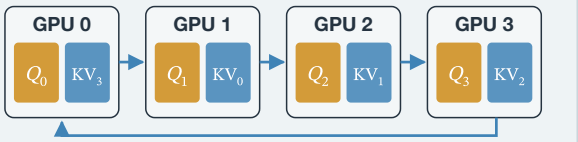
Numbers:
DeepSeek runs prefill on 4-node units and decode on 18-node units: ≈73.7k input and 14.8k output tok/s per H800 node.

06 Context parallelism: Ring vs Ulysses

Ring Attention 2023 · Liu, Zaharia, Abbeel, UC Berkeley · ICLR 2024 · arXiv:2310.01889 — **DeepSpeed-Ulysses** 2023 · Jacobs et al., Microsoft · arXiv:2309.14509 · Used in: DeepSpeed; Llama 3 (all-gather variant)

$$\text{Ring: comm hidden if } c \geq \frac{F}{B}; \qquad \text{Ulysses: } \frac{4Nh}{P} \text{ per link, vs } 4Nh \text{ (Megatron-SP)}, \qquad P \leq H$$

Ring: hidden if block $c \geq F / \text{B}$ Ulysses: $4Nh/P$ per link, $P \leq H$

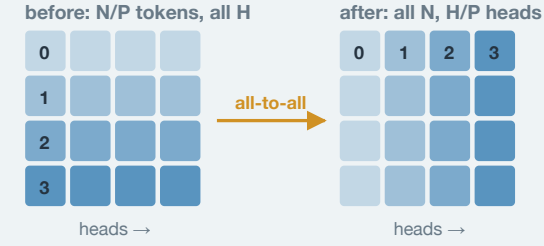


step t: GPU i uses block (i – t) mod P, then sends it on

min block c = F/B (reported, Table 2):

A100 NVLink **1.0K** A100 IB **24.5K** TPU v4 **1.0K**

causal mask unbalances work: stripe or use 2×P chunks



each GPU holds H/P heads → $P \leq H$ (K/V: $P \leq H_{kv}$)

Bytes received per GPU per layer: $N = 128$K, $P = 8$, $d_h = 128$, BF16 (computed)

MHA 64/64 · ring	4 GiB
MHA 64/64 · Ulysses	1 GiB
GQA 64/8 · ring	512 MiB
GQA 64/8 · Ulysses	576 MiB

$\text{ring} \approx 2N\, H_{kv}\, d_h\, s$ (flat in P), $\quad \text{Ulysses} \approx N(2H_q + 2H_{kv})\, d_h\, s / P$

Ulysses: 2.5× faster at 4× longer sequences (reported)

Llama 3: all-gather K/V instead; GQA K/V ≪ Q · CP = 16 at 131K

Compare:
ring traffic $\approx 2NH_{kv}d_h s$ per GPU whatever P; Ulysses falls as $1/P$ but caps at $P = H$ (computed).

Gotcha:
under GQA, Ulysses splits the H_{kv} heads too; beyond $P = H_{kv}$ K/V must be replicated.

Used in:
Llama 3 all-gathers K/V (small under GQA) over 2×CP load-balanced chunks; CP = 16 at 131K.

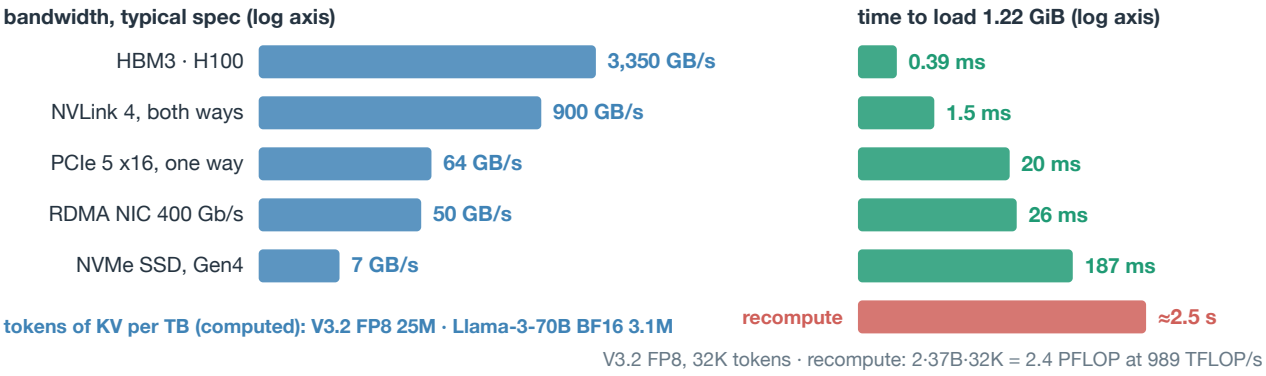
Not solved:
FLOPs stay $O(N^2)$; context parallelism only spreads them.

07 Tiered storage: HBM → DRAM → SSD

Mooncake Store 2025 · Moonshot AI · GitHub — **LMCache** 2025 · Liu et al. · arXiv:2510.09665 — **3FS** 2025 · DeepSeek · GitHub · Used in: Kimi, DeepSeek API, vLLM and SGLang (LMCache)

$$\text{reload if } \frac{N\, b_{\text{tok}}}{\text{BW}_{\text{tier}}} < T_{\text{prefill}}(N) \approx \frac{2\, n_{\text{active params}}\, N}{F_{\text{GPU}}}$$

reload wins if $N \cdot b_{\text{tok}}$ / BW_{tier} < $T_{\text{prefill}}(N)$



tokens of KV per TB (computed): V3.2 FP8 25M · Llama-3-70B BF16 3.1M

V3.2 FP8, 32K tokens · recompute: 2·37B·32K = 2.4 PFLOP at 989 TFLOP/s

Reported

DeepSeek, 24 h (Feb 2025)	342B of 608B input tokens (56.3%) hit the on-disk KV cache
3FS (DeepSeek)	≈6.6 TiB/s aggregate reads, 180 storage nodes
LMCache + vLLM	up to 15× throughput (multi-round QA, document analysis)
Mooncake store	KV pool over DRAM and SSD/NVMe of the GPU cluster

Why it works:
thin records make even SSD reloads beat prefill: one TB holds 25M tokens of V3.2 FP8 KV (computed).

Numbers:
3FS reports ≈6.6 TiB/s aggregate reads on 180 nodes; LMCache up to 15× throughput with vLLM.

Gotcha:
cached KV is tied to one model, one precision and an exact prefix. It is not semantic memory.

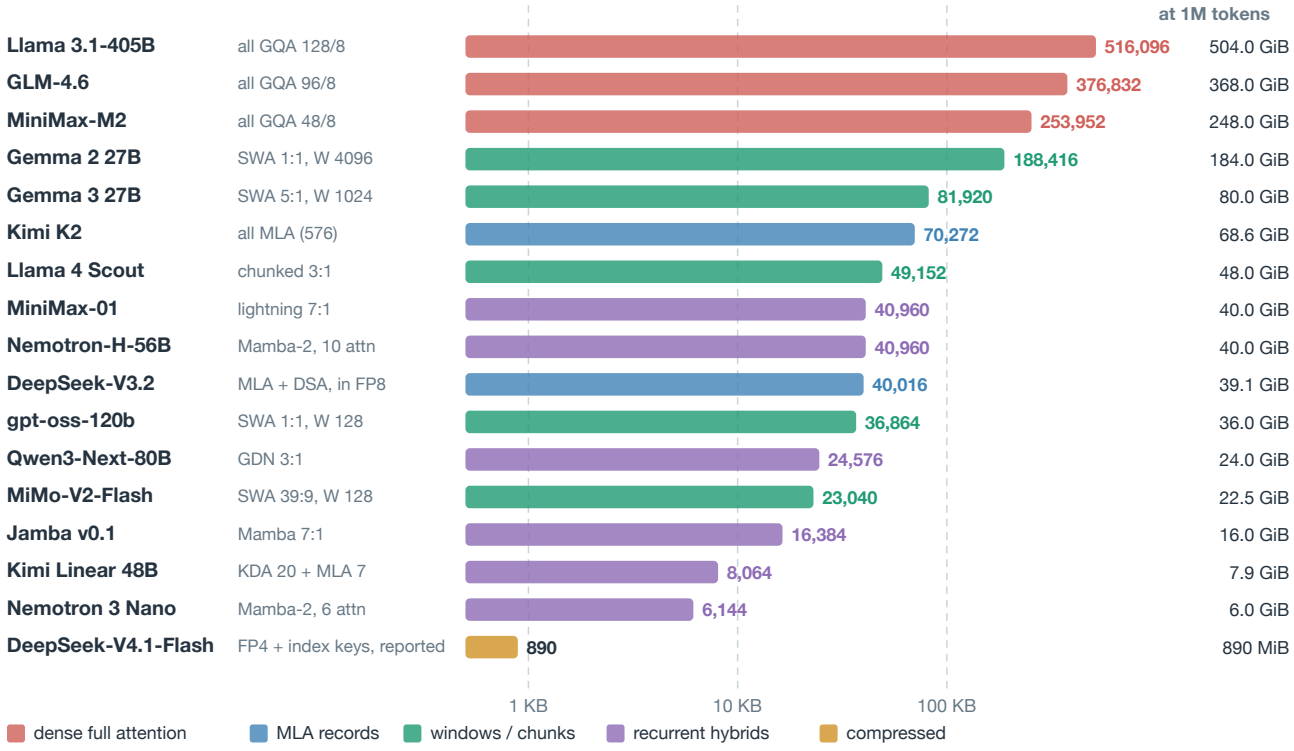
15 · Compare and choose

Every lever trades recall for bytes in a different place. Compare the real stacks, check what each guarantees by construction, then measure.

01 The bill across 17 real stacks

$$\text{growing KV per token} = \sum_{\ell \in \text{full layers}} \text{record}_\ell \cdot s \qquad \text{at } 10^6 \text{ tokens: } \times 2^{20}$$

growing KV / token = $\Sigma_{\ell \in \text{full layers}}$ record$_\ell \cdot s$ (bounded windows and states excluded)



A 580× spread.
Dense 405B → Nemotron 3 Nano is 84×. V4.1-Flash's reported 890 B (main KV + indexer keys, FP4) adds the rest.

Excluded on purpose:
bounded parts. Qwen3-Next's recurrent state is 72 MiB per sequence and gpt-oss's windows 4.5 MiB, at any length.

Full attention is not dead.
MiniMax-M2, GLM-4.x and Ministral 3 shipped all-global in 2025 (why).

Source:
each `config.json`; arithmetic in the research notes.

02 What each lever guarantees

pick the failure you can afford: recall lost ↔ bytes per token

choose by what you cannot afford to lose: recall vs bytes per token

	exact recall far back	decode cost flat in N	nothing discarded	exposed by
Full attention (GQA, MLA) ch. 03 04	yes	no	yes	cost at 128K+
Window / chunk layers alone ch. 05	no	yes	no	needle beyond W
Sparse reads (top-k) ch. 06	if found	reads only	yes	multi-needle, aggregation
Eviction ch. 07	if kept	yes	no	question after the context
Sequence compression ch. 08	blurred	N / r	no	verbatim copy
Recurrent state ch. 09–11	capacity	yes	no	associative recall, copy
Hybrid (few full layers) ch. 12	yes	mostly	yes	cost of the full layers

Yellow cells need measuring.
"If found", "if kept" and "capacity" depend on training and budget, not on the formula.

Test the failure, not the average.
Needle-in-a-haystack is easy for most levers; multi-key RULER tasks, aggregation and questions asked after the context are not.

Multi-turn matters.
A prompt-only eviction can lose what turn 3 asks for (07).

03 Three 2026 recipes

stack = positions + stability + thin records + fewer reads or a state + fewer bytes

a 2026 stack = positions + stability + thin records + fewer reads + state + bytes

DeepSeek-V4-Flash
2026-04 · 43 layers

- 01 partial RoPE 64 + YaRN ×16
- 02 learned sinks
- 03 MQA: 1 KV head, K = V (512)
- 05 SWA W 128 branch in every layer
- 06 indexer top-512 over compressed
- 08 CSA r 4 / HCA r 128 alternate
- 14 FP8 entries, BF16 RoPE dims

reported: 10% of V3.2 KV at 1M

Qwen3.8-Flash-Next
2026-08 · 48 layers

- 01 partial mRoPE 64/256
- 02 sigmoid output gate
- 03 GQA 24 Q / 2 KV, d_h 256
- 06 QSA: pooled-key indexer, 2,048 budget
- 10 Gated DeltaNet, 36 layers
- 12 12 × [3 GDN + 1 attention]

24,576 B/token growing KV

Gemma 4 31B
2026-04 · 60 layers

- 01 local θ 1e4, global p-RoPE 25%
- 02 QK-norm
- 03 global: 4 KV heads, K = V, d_h 512
- 05 5 SWA (W 1,024) : 1 global

51,200 B/token (K=V: –37.5%, reported)

Nobody pulls one lever.
Each stack combines 4–7 chapters of this book.

Converging defaults:
partial RoPE, a sink or output gate, very few KV heads, a small local window, a learned top-k reader.

Open questions:
where the indexer cost goes at 10M tokens, how to evaluate learned forgetting, and whether state tracking (10) matters at scale.